MINISTERUL EDUCAȚIEI ȘI CERCETĂRII

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA
FACULTATEA DE AUTOMATICĂ ŞI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

Sistem de detecție a exercițiilor fizice bazat pe învățare automată

LUCRARE DE LICENŢĂ

Absolvent: **Iulian STRATAN**

Coordonator științific: **Ș.l. dr. ing. Robert VARGA**

2026

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA
FACULTATEA DE AUTOMATICĂ ŞI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

DECAN,
Prof. dr. ing. Vlad MUREȘAN

DIRECTOR DEPARTAMENT,
Prof. dr. ing. Rodica POTOLEA

Absolvent: **Iulian STRATAN**

Sistem de detecție a exercițiilor fizice bazat pe învățare automată

1. **Enunțul temei:** Acest proiect are ca scop dezvoltarea unuei arhitecturi neuronale unificate capabile să proceseze date de intrare sub formă de secvențe video și să realizaze simultan recunoașterea tipului de activitate fizică și numararea automată a repetițiilor efectuate de utilizator

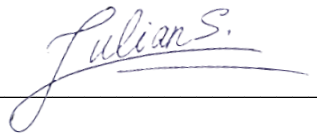
2. **Conținutul lucrării:** Pagina de prezentare, Introducere, Obiectivele proiectului, Studiul Bibliografic, Analiză și fundamentare teoretică, Proiectare de detaliu și implementare, Testare și validare, Manual de instalare și utilizare, Concluzii, Bibliografie, Anexe.

3. **Locul documentării**: Universitatea Tehnică din Cluj-Napoca, Departamentul Calculatoare

4. **Consultanți**:

5. **Data emiterii temei:** 1 Noiembrie 2025

6. **Data predării:** 10 Iulie 2026

Absolvent:  _____

Coordonator științific: ____Ș.l. dr. ing. Robert Varga____

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA
FACULTATEA DE AUTOMATICĂ ŞI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

Declarație pe propria răspundere privind autenticitatea lucrării de licență

Subsemnatul(a)_____Stratan_Iulian_____ _____, legitimat(ă) cu _____carte de identitate_____ seria ___ROU___ nr. _____000160663_____ CNP _____7030703120027_____, autorul lucrării _____ _____SISTEM_DE_DETECȚIE_A_EXERCIȚIILOR_FIZICE_BAZAT_PE_ÎNVĂȚARE _AUTOMATĂ_____elaborată în vederea susținerii examenului de finalizare a studiilor de licență la Facultatea de Automatică și Calculatoare, Specializarea _____Calculatoare_____ din cadrul Universității Tehnice din Cluj-Napoca, sesiunea _____Iulie_____ a anului universitar ___2025-2026____, declar pe propria răspundere că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate, în textul lucrării, și în bibliografie.

Declar că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de licență.

In cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative, respectiv, *anularea examenului de licență*.

Data

_____10.07.2026_____

Stratan Iulian

_____

Semnătura

Cuprins

Capitolul 1. Introducere 1

Capitolul 2. Obiectivele proiectului 4

2.1. Enunțul temei de proiectare 4

2.2. Obiectivul general al proiectului 4

2.3. Obiective specifice 4

2.3.1. Procesarea și analiza datelor 4

2.3.2. Extracția și modelarea scheletului 5

2.3.3. Generarea reprezentărilor spațio-temporale 5

2.3.4. Proiectarea și antrenarea rețelei multi-task 5

2.3.5. Testare si validarea performantei 5

2.4. Cerințe 5

2.4.1. Funcționale 5

2.4.2. Non-funcționale 5

2.5. Metodologia cercetării 6

Capitolul 3. Studiu bibliografic 7

3.1. Metode pentru estimarea poziției umane 8

3.1.1. Stacked Hourglass Networks 8

3.1.2. High-Resolution Representation Network 8

3.1.3. You Only Look Once 8

3.2. Metode pentru recunoașterea acțiunilor umane 9

3.2.1. Rețele convoluționale 3D 9

3.2.2. Spatial Temporal Graph Convolutional Networks 10

3.2.3. Pose MoTion Representation 11

3.3. Metode pentru numărarea repetițiilor 12

3.3.1. Procesarea de semnal 12

3.3.2. Învățarea automată 13

3.4. Sisteme multi-task pentru analiza exercițiilor fizice 13

3.4.1. GymCam 14

3.4.2. Sisteme bazate pe unghiurile articulațiilor 14

3.4.3. RepPenn 14

Capitolul 4. Analiză și fundamentare Teoretică 16

4.1. Arhitectura logică a sistemului 16

4.2. Fundamentele estimării poziției umane 18
4.2.1. Paradigme abstracte de procesare a spațiului vizual 19
4.2.2. Teoria detecției Anchor-Free 19
4.2.3. Modelarea matematică a coordonatelor 20
4.2.4. Funcția de pierdere pentru repere anatomice 21
4.3. Modelarea matematică a hărților de probabilitate 22
4.3.1. Fundamentarea probabilistică prin distribuții Gaussiene 23
4.3.2. Rolul teoretic al parametrului de dispersie 23
4.3.3. Structura de tensor și reprezentarea multi-canal 24
4.4. Fundamentarea teoretică a hărților de proiecție 24
4.4.1. Problema dimensionalității și limitările modelelor 3D 24
4.4.2. Conceptul proiecției ortogonale și colapsarea axelor spațiale 25
4.4.3. Unificarea canalelor și conversia temporalității în textură 25
4.4.4. Invarianța temporală și normalizarea spațiului de trăsături 26
4.5. Teoria învățării Multi-Task 27
4.5.1. Formularea matematică a problemei Multi-Task 28
4.5.2. Topologia rețelei: Partajarea Rigidă a Parametrilor 28
4.5.3. Optimizarea și funcția de pierdere compusă 29
4.5.4. Mecanisme abstracte ale generalizării 30
4.6. Modele abstracte pentru clasificare și regresie 30
4.6.1. Problema degradării și dispariția gradientului 31
4.6.2. Principiul Învățării Reziduale și conexiunile Skip 31
4.6.3. Analiza comparativă: ResNet-34 și ResNet-18 32
4.6.4. Diferențierea teoretică a capetelor de decizie 33

Capitolul 5. Proiectare de detaliu și implementare 35

5.1. Tehnologii și biblioteci utilizate 35
5.2. Schema generală a aplicației 37
5.3. Sinteza setului de date RepPenn 38
5.3.1. Normalizarea spațială și extragerea cadrelor 38
5.3.2. Crearea artificială a ciclicității 39
5.3.3. Augmentarea temporală 39
5.3.4. Codificarea și stocarea 39
5.4. Preprocesare și gestionare a datelor 40
5.4.1. Arhitectura clasei de gestionare a datelor 40
5.4.2. Diagrama de clasă 41
5.5. Modulul de extracție a reperelor anatomice 42

5.6. Modulul de procesare spațio-temporală 43
5.6.1. Normalizarea spațială a coordonatelor 43
5.6.2. Generarea hărților de probabilitate 43
5.6.3. Proiecția în hărți P-map 43
5.7. Proiectarea modelului Multi-Task 44
5.7.1. Inițializarea arhitecturii 44
5.7.2. Fluxul de inferență 44
5.7.3. Diagrama de clasă a modulului Multi-Task 45
5.8. Interfața utilizator 46
5.8.1. Funcția centrală de procesare 46
5.8.2. Construcția Interfeței Web 46
5.9. Caz de utilizare: Procesarea unui video a unui antrenament fizic 47
5.10. Diagrama de secvență a sistemului 48
5.10.1. Descrierea modulelor 48
5.10.2. Pașii efectuați 49

Capitolul 6. Testare și validare 50

6.1. Metodologia de antrenare 50
6.1.1. Alegerea optimizatorului 51
6.1.2. Regularizarea prin degradarea ponderilor 51
6.1.3. Planificatorul ratei de învățare 52
6.2. Metricile de evaluare a performanței 52
6.2.1. Metrica pentru Clasificare: Acuratețea 52
6.2.2. Metricile pentru Regresie 53
6.3. Analiza Etapei 1: Antrenarea ramurii de recunoaștere 54
6.3.1. Analiza convergenței și a performanței 54
6.3.2. Discuție asupra fenomenului de generalizare 54
6.4. Analiza Etapei 2: Antrenarea ramurii de regresie 55
6.4.1. Analiza convergenței și a metricilor specifice 55
6.4.2. Discuție asupra performanței izolate de regresie 55
6.5. Analiza Etapei 3: Reglaj fin 56
6.5.1. Funcția de pierdere compusă 56
6.5.2. Strategia de optimizare 56
6.5.3. Analiza convergenței și a performanței 56
6.6. Analiza timpului de execuție 57
6.6.1. Specificațiile mediului hardware 57
6.6.2. Evaluarea costului antrenării stratificate 57

6.6.3. Analiza timpului de răspuns 57

Capitolul 7. Manual de instalare si utilizare 59

7.1. Cerințe de sistem 59

7.1.1. Cerințe hardware 59

7.1.2. Cerințe software 59

7.2. Ghid de instalare 59

7.3. Manual de utilizare 60

Capitolul 8. Concluzii 63

8.1. Performanța generală 63

8.2. Contribuții personale 63

8.3. Posibile dezvoltări și îmbunătățiri 64

Bibliografie 65

Capitolul 1. Introducere

Odată cu revoluția industrială și digitalizarea masivă a serviciilor cotidiene, precum livrarea mâncării, comerțul online, automatizarea comfortului casnic și tranziția globală către platformele de telemunacă, nivelul de activitate fizică a omului a scăzut considerabil. Acest efect de sedentarism a fost amplificat și de pandemia de Covid-19, când restricțiile de mișcare au limitat accesul la facilitățile sportive și au impus izolarea la domiciliu [1][2]. Având în vedere consecințele avansării tehnologice, care elimina treptat necesitatea efortului fizic în sarcinile de zi cu zi, și a izolării din timpul pandemiei, se accentuează importanța și necesitatea mișcării omului contemporan pentru a asigura un stil de viață sănătos.

Avantajele activității fizice regulate [3] sunt foarte bine cunoscute, cercetate și documentate în literatura medicală de specialitate, având un impact pozitiv nu doar asupra sănătății fizice și mentale a omului, ci și asupra economiei globale, prin reducerea, de exemplu, a presinuii pe sistemele de sănătate publică. La fel, exercițiile fizice reprezintă principala metodă de prevenție a bolilor noncomunicabile (cronice) cum ar fi bolile cardiovasculare, anumite tipuri de cancer, bolile respiratorii cronice și diabetul. Mai mult ca atât, antrenamentul de forță și rezistență poate juca un rol important în longevitatea corpului uman. Acesta diminuează stresul mecanic de lungă durată aplicat asupra încheieturilor prin echilibrarea forțelor interne și externe la nivel articular, și întărește întregul organism prin stimularea hipertrofiei musculare și creșterea densității osoase [4]. În plan psihologic, activitatea fizică susținută reglează secrecția hormonilor fericirii, reducând semnificativ apariția episoadelor de anxietate și depresie, factori de risc tot mai des întâlniți în societatea actuală.

Cum a fost observat anterior, pandemia a avut un impact considerabil asupra societății, printre care și creșterea numărului de utilizatori a platformelor digitale, care s-au dovedit a fi utile în promovarea activă a sportului și angajarea de lungă durată a utilizatorilor în activități fizice intense și moderate [5]. Prin urmare, aceasta a dus la apariția a multor soluții digitale pentru efectuarea antrenamentelor și a activității fizice în general, de la distanță, monitorizând și analizând acestea prin intermediul dispozitivelor inteligente și a asistenților virtuali [6], precum procesarea activității utilizând Machine Learning (ML sau învățare automată).

Învățarea automată este un domeniu deosebit de amplu ce include în sine multe direcții posibile de cercetare și dezvoltare, însă în contextul analizei activității fizice, cea mai rapidă ascensiune o cunoaște subdomeniul de Computer Vision (viziunea artificială). Bazele viziunii artificiale au ca fundament descoperirile din matematică, geometrie și statistică. Această popularitate se datorează capacităților sale înalte și versatile de procesare și de extragere a informațiilor complexe și utile din date vizuale ce nu sunt structurate, care pot avea mai multe forme și surse de proveniență : imagini statice, fluxuri video, secvențe sincronizate de la mai multe camere sau date 3D, oferind în același timpt comoditate utilizatorului, deoarece nu necesită dispozitive sau senzori auxiliari incomozi, fiind destul faptul că în ziua de azi majoritatea oamenilor au posibilitatea de a capta fotografii sau de a înregistra fișiere video utilizând camera telefonului său. Subdomeniul viziunii artificiale a înregistrat o dezvoltare rapidă de la experimente de cercetare sau laborator, la aplicații utilizate în viața cotidiană de către un număr mare de oameni. Astăzi, aceste sisteme sunt capabile să ofere feedback în timp real și, de exemplu, să analizeze performanța sportivă.

Recunoașterea formelor (pattern recognition) reprezintă detecția automată, clasificarea, și interpretarea regulilor, tendințelor, sau a structurilor din date, utilizând, deseori, algoritmi din învățarea automată. În contextul analizei corpului uman, acest domeniu se traduce prin estimarea poziției (pose estimation). Aceasta presupune detectarea și urmărirea unor structuri anatomice specifice, precum articulațiile, pentru a le diferenția de alte obiecte din scenă sau de fundal. Metodele care exploatează aceste repere pot funcționa eficient pentru a evidenția arhitectura mișcării, oferind soluții în diverse situații practice, de la supraveghere video până la interacțiunea om-calculator.

Algoritmii de reprezentare geometrică sunt utilizați frecvent în procesarea imaginilor umane pentru analizarea și interpretarea relațiilor spațiale dintre elementele corpului. Aceștia se bazează pe principii matematice pentru a transforma pixelii în repere spațiale (keypoints) și schelete digitale. Reprezentările geometrice au o utilitate ridicata deoarece permit identificarea structurilor acțiunilor prin analizarea exclusiv a mișcării articulațiilor. Avantajul major al acestor metode este reprezentat de rezistenta lor fata de factorii externi perturbatori, cum ar fi hainele purtate de utilizator, condițiile de iluminare sau detaliile inutile din fundal, fiind ideale pentru sisteme care necesita o procesare rapida și curata a informațiilor.

Pe lângă recunoașterea tipului de exercițiu, este relevantă, în timpul antrenamentului, pentru o stimulare eficienta a organismului, și monitorizarea numărului de repetări efectuate. În mod tradițional, modelele sunt antrenate pentru a îndeplini o singura sarcina principala. De exemplu, un sistem poate fi învățat să realizeze clasificarea (identificarea tipului de mișcare), o sarcina în care algoritmul alege o etichetă dintr-un set predefinit, sau regresia (numărarea repetițiilor), o sarcină în care modelul prezice o valoare numerică continuă. Desi abordarea individuala pe sarcini este eficientă, ea presupune un efort dublu de procesare și un consum ridicat de resurse atunci când sistemul trebuie să ofere răspunsuri multiple în același timp.

În acest context, învățarea mulți-task (mulți-task learning) a devenit o tehnică de învățare supervizată extrem de populară, utilizată pentru a rezolva automat și simultan mai multe probleme corelate. Scopul principal este identificarea unor trăsături comune în datele de intrare și utilizarea acestora într-un mod partajat. Elementele extrase din imagini sunt folosite atât pentru a deduce natura acțiunii, cât și frecventa ei. Învățarea mulți-task este aplicată pe scară largă în sisteme complexe și în prelucrarea datelor video. Printre beneficiile sale se numără reducerea complexității arhitecturale, scăderea timpului de inferență, automatizarea extragerii informațiilor relevante și generalizarea mai bună a modelului.

Pe lângă abordările clasice, dezvoltarea viziunii artificiale a fost accelerată în ultimele decenii de rețelele neuronale convoluționale (convolutional neural networks - CNN) și de variantele lor tridimensionale (3D CNN). Rețelele 3D CNN sunt o versiune avansată concepută pentru a extrage atât trăsături spațiale, cât și temporale din fluxurile video. Ele sunt alcătuite din volume de straturi convoluționale și complet conectate. Aceste rețele sunt utilizate frecvent pentru recunoașterea acțiunilor, oferind performanțe ridicate în înțelegerea contextului video. Totuși, aceste metode necesită putere computațională masivă și memorie extinsă pentru a procesa sute de cadre simultan. În contrast, metodele care comprimă informația video în reprezentări statice 2D oferă soluții mai rapide, mai flexibile și mult mai ușor de rulat, fiind ideale pentru aplicații în timp real.

Datele video continue aduc provocări suplimentare în procesarea automată. Mișcarea umană în timpul activităților fizice este complexă și predispusă la oscilații. Crearea unui tipar digital stabil necesita, în contextul metodele care comprimă informația video, transformarea cadrelor brute în harți spatio-temporale detaliate.

Aceste metode de reprezentare, cum ar fi hărțile de probabilitate (heatmaps), sunt importante pentru a modela distribuția și probabilitatea localizării articulațiilor pe parcursul mișcării. Deoarece datele video brute nu includ în mod direct informații semantice evidente despre numărul de repetiții sau tipul de efort, este necesară această etapă de conversie vizuală pentru a putea extrage comportamentul de interes.

Pentru a înțelege biomecanica umană într-un mod detaliat, sistemele inteligente trebuie să fie capabile să identifice și să urmărească o varietate de mișcări rapide, pauze și poziții specifice din timpul antrenamentelor. Această capacitate este asigurată adesea de echiparea utilizatorilor cu diverși senzori corporali (wearables). Cu toate acestea, comparativ cu senzorii fizici, camerele video standard furnizează date mai bogate și neintruzive, neafectand confortul utilizatorului, fiind considerate ideale atât în medii controlate, cât și în scenarii din viața reala, unde accesibilitatea echipamentelor hardware este limitată.

Detecția tipului de exercițiu și numărarea repetițiilor reprezintă o provocare semnificativă datorită variabilității modului de execuție de la un individ la altul, apariției mișcărilor repetitive, variației în viteza, precum și obturării parțiale a corpului din cauza unghiurilor diferite ale camerei. Modelele potrivite pentru aceasta problemă trebuie să fie rapide, să extragă cu acuratețe datele necesare și să nu depindă excesiv de memorarea secvențială cadru cu cadru, pentru a preveni durata de așteptare mare în timpul procesării.

În acest context, proiectul își propune să rezolve problema recunoașterii acțiunilor fizice și a numărării repetițiilor folosind exclusiv date video procesate prin viziune artificială. Această direcție este motivată de necesitatea unei soluții eficiente și neintruzive, care să poată fi integrată cu ușurință în cadrul sistemelor de antrenament digital și asistență virtuală. Analiza simultană a mișcărilor este un element important pentru oferirea de feedback în timp real și monitorizarea corecta a performanței. Prin combinarea tehnicilor de extracție a scheletului cu strategii eficiente de comprimare temporală și arhitecturi mulți-task de tip CNN, proiectul urmărește să contribuie la dezvoltarea unor soluții performante pentru aplicațiile de fitness și telemedicină din viitor.

Capitolul 2. Obiectivele proiectului

Capitolul dat oferă o viziune detaliată asupra obiectivelor urmărite în cadrul acestui proiect, definind direcțiile de cercetare și criteriile de succes a sistemului. Accentul este pus pe transformarea unei probleme teoretice de viziune artificială într-o soluție software funcțională.

2.1. Enunțul temei de proiectare

Tema de proiectare se definește ca o problemă complexă de viziune artificială ce necesită dezvoltarea unui model capabil să proceseze date de intrare sub formă de secvențe video într-un timp util, utilizând un volum de resurse redus. Provocarea principală constă în balansarea unei precizii adecvate în identificarea exercițiilor fizice cu o eroare minimă în numărarea repetițiilor, totul în cadrul unei singure execuții a modelului. Sistemul trebuie să fie capabil să ignore elementele irelevante din cadre, cum ar fi fundalul sau variațiile de lumină și să se concentreze exclusiv pe mișcarea omului. Această abordare necesită o arhitectură hibridă care să combine extracția trăsăturilor spațiale cu interpretarea lor temporală.

Astfel, această lucrare propune o strategie de cercetare axată pe eficientizarea fluxului de date prin utilizarea reprezentărilor de tipul harților proiectate (projected maps sau P-maps). Această metodă permite sistemului să trateze informația video ca pe o imagine statică, reducând complexitatea matematică față de modelele clasice 3D. Prin enunțarea acestei teme, se urmărește nu doar replicarea unor performante existente, ci și optimizarea pipeline-ului de procesare cu scopul de a utiliza un set de date mai mic, respectiv și utilizarea a unui spatiu de stocare mai redus și timp de antrenare mai rapid.

2.2. Obiectivul general al proiectului

Obiectivul general al proiectului constă în integrarea unor tehnologii existente și proiectarea și implementarea unei arhitecturi neuronale unificate care să realizaze simultan recunoașterea tipului de activitate fizică și numărarea automată a repetițiilor efectuate de utilizator.

Viziunea de ansamblu a sistemului presupune crearea unui flux automatizat care pornește de la un videoclip brut și se finalizeaza cu un feedback despre exercițiul fizic efectuat, fără a necesita intervenția manuală a utilizatorului.

2.3. Obiective specifice

2.3.1. Procesarea și analiza datelor

Acest obiectiv vizează ingineria datelor necesare pentru antrenarea și validarea modelelor. Sarcina principală este sintetizarea și prelucrarea setului de date RepPenn. Se urmărește standardizarea datelor prin conversia videoclipurilor în colecții de matrici normalizate, ceea ce va permite o viteză de citire și procesare mult mai mare în timpul antrenării. Analiza datelor include si implementarea unor tehnici de augmentare pentru a crește stabilitatea modelului față de unghiuri diverse, viteze diferite de execuție și alte perturbații posibile.

2.3.2. Extracția și modelarea scheletului

Utilizarea unui model deja antrenat reprezintă obiectivul tehnic central pentru modificarea pipeline-ului inițial în faza de preprocesare. Acest pas este foarte important, deoarece calitatea scheletului extras determină succesul tuturor etapelor ulterioare. Obiectivul include și filtrarea datelor pentru a elimina cadrele în care persoana nu este detectată.

2.3.3. Generarea reprezentărilor spațio-temporale

Acest obiectiv este partea cheie a păstrării informației temporale și se concentrează pe transformarea coordonatelor scheletului dintr-un spațiu 3D al secvenței de imagini de intrare în spațiul 3D al hărților de probabilitate gaussiene de dimensiuni mici, iar apoi comprimarea lor în hărți de dimensiuni fixe.

2.3.4. Proiectarea și antrenarea rețelei multi-task

În centrul acestui obiectiv se află construcția arhitecturii de tip Y, care folosește straturi comune pentru extracția trăsăturilor și două ramuri paralele de decizie. Se vizează integrarea unei rețele pentru sarcina de clasificare și a unei rețele mai mici pentru sarcina de regresie a numărului de repetiții.

2.3.5. Testare si validarea performantei

Ultimul obiectiv specific vizează evaluarea sistemului folosind metrici standardizate din literatura de specialitate. Se urmărește atingerea unei acurateți de peste 90% în clasificarea exercițiilor. De asemenea, validarea include măsurarea metricii OBO (Off-By-One) și o evaluare subiectivă pe exemple din afara setului de date.

2.4. Cerințe

2.4.1. Funcționale

1. Sistemul trebuie să permită încărcarea fișierelor video.
2. Sistemul trebuie să poată afișa vizual scheletul detectat suprapus peste imaginea originală, oferind utilizatorului certitudinea că mișcarea este monitorizată corect.
3. Rezultatele finale (tipul de exercițiu și numărul de repetiții) trebuie prezentate într-un format textual clar si ușor de citit.

2.4.2. Non-funcționale

1. Portabilitate – sistemul trebuie sa poată fi rulat atât în medii de dezvoltare locale, cât și prin intermediul unor interfețe web fără configurări complexe.
2. Eficiența resurselor este prioritară, modelul rulând la capacitate maximă pe hardware echipat cu unitate de procesare grafică (GPU), dar trebuie sa fie capabil de inferențe și pe procesoare standard (CPU).
3. In final, codul trebuie să fie documentat și modular, permițând extinderea ulterioara a sistemului cu noi tipuri de exerciții fizice sau funcționalități adiționale.

2.5. Metodologia cercetării

Etapele realizării proiectului trebuie să urmeze un flux logic și iterativ, structurat pe parcursul întregii perioade de dezvoltare.

1. Studiul teoretic aprofundat, analizând arhitecturile de viziune artificială și metodele de prelucrare a scheletului uman.
2. Ingineria datelor, incluzând colectarea, curățarea și sintetizarea setului de date RepPenn.
3. Experimentarea și analiza rezultatelor de detecție a scheletului a modelelor existente, obținând inferențele pentru toate cadrele din setul de date și procesându-le pentru a le include în formatele intermediare pentru o dezvoltare mai eficientă și flexibilă.
4. Implementarea arhitecturii multi-task.
5. Antrenarea modelului și reglarea hiperparametrilor (finetuning).
6. Validarea rezultatelor și dezvoltarea interfeței pentru demonstrație a execuției sistemului

Capitolul 3. Studiu bibliografic

Acest capitol prezintă o parte din soluțiile obținute în urma cercetărilor realizate în domeniul viziunii artificiale aplicate pe analiza comportamentului uman, punând accentul pe avantajele, dezavantajele și limitările fiecărei metode.

Cum a fost subliniat în introducere, au apărut multe soluții digitale pentru efectuarea antrenamentelor și a activităților fizice și monitorizarea acestora prin intermediul asistenților virtuali [6], ceea ce permite o gamă nouă și foarte variată de produse software precum Smart Coaching, care pot fi bazate pe tehnologia de Digital Twin [7] (geaman digital), inițial concepută pentru îmbunătățirea proceselor industriale, astfel încât subiectul real să fie replicat în cadrul sistemului informatic pentru a putea monitoriza, înțelege și optimiza funcțiile tuturor entităților fizice și de a oferi un feedback analitic și exact. În acest sens în articolul [8] este prezentat conceptul de Digital Twin Coaching (antrenarea prin intermediul geamanului digital) care combină exact aceste două tehnologii ce au avantajul unui răspuns personalizat pentru fiecare individ în parte și fiecare activitate separată cu o ghidare eficientă pentru progrese continue, însă articolul prezintă un ecosistem complex, care poate fi vizualizat în Figura 3.1, cu anumite cerințe tehnice și sugestii de standardizare. Astfel, documentarea bibliografică este structurată doar pe cele trei direcții fundamentale ale viziunii artificiale aplicate pe analiza activității fizice, care pot fi găsite în modulul inteligent a sistemului Digital Twin Coaching: estimarea poziției umane, recunoașterea acțiunilor și numărarea repetițiilor, finalizând cu analiza sistemelor hibride multi-task.

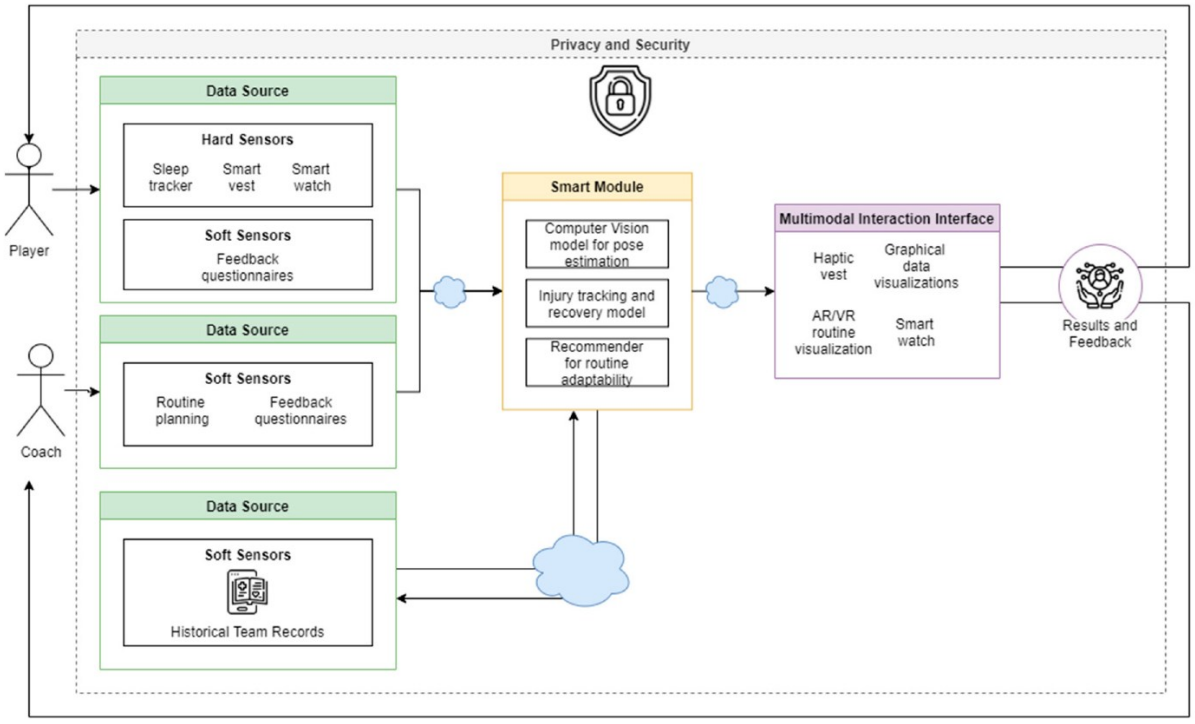


Figura 3.1: Sistemul Digital Twin Coaching. Imagine preluată din [8].

3.1. Metode pentru estimarea poziției umane

Detecția poziției umane este una dintre obiectivele fundamentale din viziunea artificială. Deși au fost propuse soluții bazate pe procesare de imagini, totuși, rețelele neuronale cunosc o deosebită popularitate și sunt larg utilizate în realizarea acestei sarcini, prin urmare tranziția de la metodele clasice de procesare a imaginilor la cele de învățare automată a permis obținerea unei precizii ridicate în medii mai complexe.

3.1.1. Stacked Hourglass Networks

În articolul științific [9] este prezentată o arhitectură fundamentală, care stă la baza multor sarcini înrudite cu estimarea poziției corpului uman. Aceasta are la bază ideea de utilizare repetată și succesivă a unui modul de rețea convoluțională ce realizează pooling, urmat de upsampling, obținându-se o formă de clepsidră, ceea ce se poate observa în Figura 3.2. Astfel se reduce rezoluția și se combină trăsături din mai multe rezoluții, ceea ce permite înțelegerea contextului global al corpului chiar și când unele membre sunt ascunse, iar împreună cu supervizarea inremediară, ce reprezintă aplicarea funcției de pierdere pe rezultatele intermediare obținute între modulele clepsidră, se ajunge la un rezultat precis. Însă dezavantajul acestei arhitecturi este volumul mare de calcule și viteza de inferență, ceea ce determină considerarea altor arhitecturi.

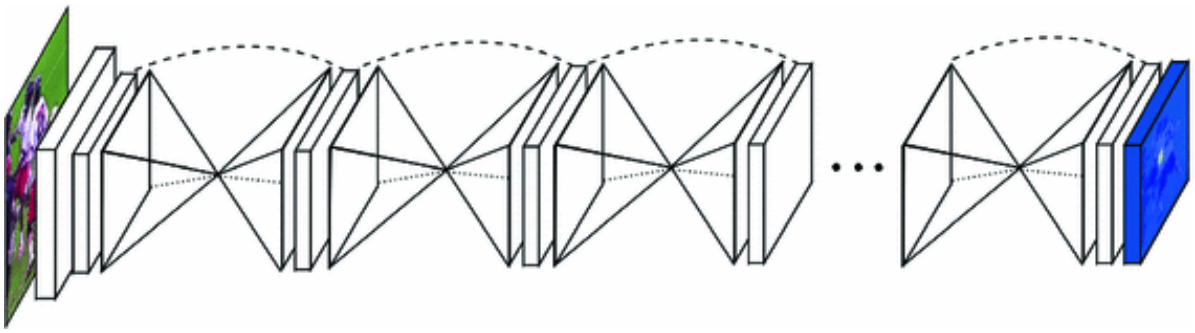


Figura 3.2: Rețea formată din module succesive în forma clepsidrei. Figură preulată din [9]

3.1.2. High-Resolution Representation Network

O altă abordare este prezentată în lucrarea [10], care și-a propus ca scop păstrarea rezoluției pe parcursul întregului de proces de inferență. Autorii acestui articol au realizat o arhitectura ce contine în prima etapa subrețele de rezoluție înaltă și apoi sunt adaugate gradual subretele de la rezolutii mari la rezolutii mici pentru a forma mai multe etape și de a conecta mai multe rezoluții în paralel, obținând astfel o formă piramidală, precum se poate observa în Figura 3.3. În consecință a fost obținut un model precis, ce reține repere spațiale exacte, aspect care ar fi esențial în analiza performanței sportive, însă această precizie vine le costul înalt de calcule și memorie necesară.

3.1.3. You Only Look Once

Pentru a depăși limitările de viteză și respectiv volum de calcul, autorii articolului [11] au prezentat o nouă arhitectură (YOLO) ce prezice direct cutiile de încadrare (bounding boxes), cât și coordonatele articulațiilor într-o singură trecere prin rețea. Aceasta este formată din 24 de straturi convoluționale urmate de 2 straturi complet conectate, alternând cu straturi de convoluție de 1x1, pentru reducerea spațiului de trăsături din straturile precedente, ceea ce se poate vedea în Figura 3.4. Astfel, această arhitectură are marele avantaj de un timp rapid de inferență și utilizare relativ scăzută de resurse, pe când dezavantajul în formă de scădere a

preciziei în scenarii cu multe persoane suprapuse, poate fi neglijat în contextul antrenamentelor individuale.

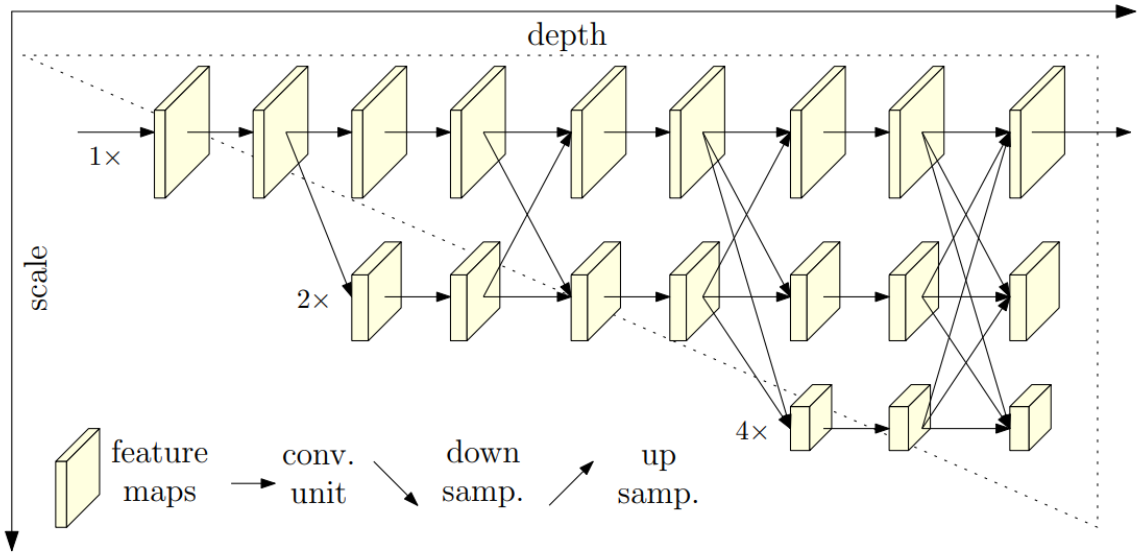


Figura 3.3: Arhitectura HRNet. Figură preluată din [10]

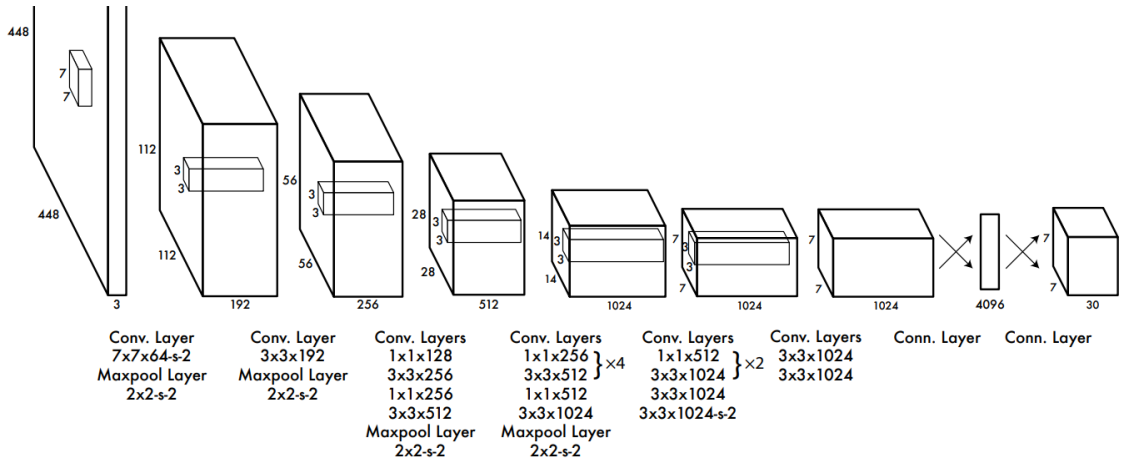


Figura 3.4: Arhitectura YOLO. Figură preluată din [11].

3.2. Metode pentru recunoașterea acțiunilor umane

Detecția acțiunilor umane într-un mediu din viața reală poate găsi un număr vast de aplicații. Aceasta este o sarcină de bază în domeniul viziunii artificiale, dar care aduce și un set de dificultăți în obținerea unui rezultat precis. Aceste dificultăți sunt cauzate de suprapunerea mai multor obiecte sau persoane în imagine, un fundal plin de detalii sau elemente neclare, variații în unghiurile de vizualizare a scenei și altele.

3.2.1. Rețele convoluționale 3D

În vederea soluționării problemelor menționate anterior, autorii lucrării [12] propun o nouă arhitectură, comparativ cu cele existente la timpul publicării, care este reprezentată în Figura 3.5. Această arhitectură a fost proiectată cu scopul de a recunoaște acțiunile efectuate de către om și reprezintă rețele neuronale convoluționale 3D ce extind soluțiile clasice 2D. Acestea extrag atât trăsăturile spațiale cât și cele temporale simultan, capturând informația de

mișcare codificată în mai multe cadre consecutive. Ulterior, această abordare a fost îmbunătățită de către autorii articolului [13], schimbând preprocesarea sub formă de segmentare a volumului video cu procesarea brută a datelor de intrare și modificând puțin arhitectura generală, care este formată din 8 straturi convoluționale, 5 straturi de max-pooling, și 2 straturi complet conectate, urmat de un strat de ieșire cu funcția de activare softmax. Numărul de filtre și structura conceptuală poate fi observat în Figura 3.6.

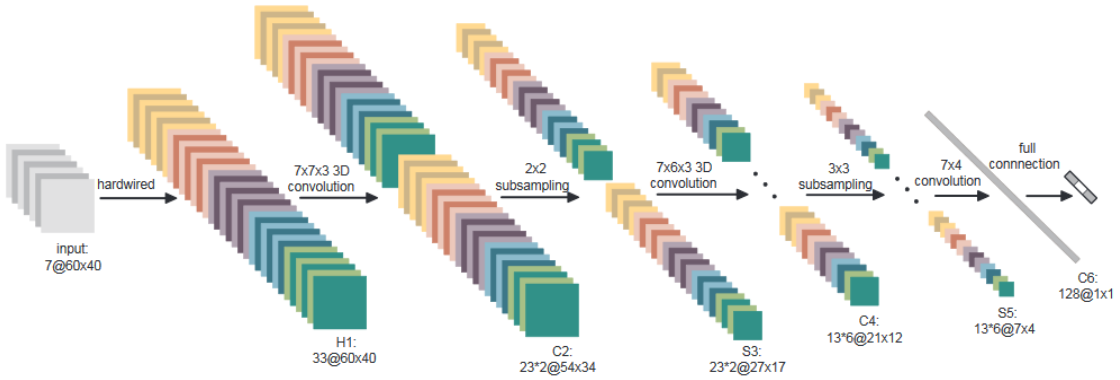


Figura 3.5: Arhitectura clasică a unei rețele convoluționale 3D.

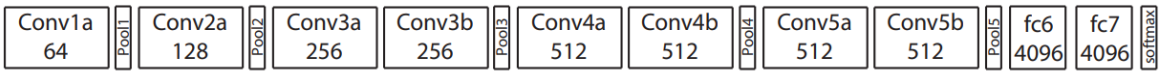


Figura 3.6: Arhitectura rețelei convoluționale 3D modificată (C3D). Figură preluată din [13]

3.2.2. Spatial Temporal Graph Convolutional Networks

În căutarea unui model pentru clasificare mai exact și rapid, în lucrarea [14] s-au studiat rețelele graf-neuronale (Graph Neural Networks sau GNN), care sunt o generalizare a rețelelor neuronale convoluționale (CNN) la grafuri de structuri arbitrare. Autorii acestui articol au proiectat o reprezentare generică a secvențelor scheletului uman pentru recunoașterea acțiunilor efectuate de om prin a extinde GNN-urile la un model de graf spațio-temporal. Precum este ilustrat în Figura 3.7, acest model este construit pe baza mai multor grafuri mai mici ce cuprind scheletul uman în fiecare cadru separat, nodurile grafurilor fiind articulațiile omului. La fel, precum orice graf, acest model are, pe lângă noduri, și muchii care pot fi de două feluri: muchii spațiale care sunt reprezentarea logică a oaselor scheletului, deci, prin urmare, acestea unesc nodurile (sau articulațiile) ce aparțin unui singur cadru, iar al doilea tip de muchii sunt muchiile temporale care unesc aceleași noduri din două cadre consecutive. Această structură ierarhică oferă posibilitatea de înglobare a informațiilor poziției corpului în timp, fără asocierea manuală a părților corpului cu părți corespunzatoare ale grafului. Asta duce la o generalizare bună, întrucât se procesează doar scheletul propriu-zis, fără fundal, dar și la o performanță ridicată, deoarece se iau în calcul doar datele numerice din noduri și perechile de noduri (muchiile), însă, în același timp, asta reprezintă și un mare dezavantaj și anume faptul ca stocarea acestui graf și procesarea sa necesită multă memorie, iar datele de intrare la această rețea graf convoluțional spațio-temporal sunt coordonatele încheieturilor și nu imaginea propriu-zisă, ceea ce înseamnă că sunt necesare tehnici sau alte modele pentru estimarea poziției fiecărei articulații, care împreună cu spațiul ocupat de modelul de reprezentare a dinamicii scheletului fac această soluție mai puțin atractivă în cadrul implementării unui sistem cu un cost redus din punct de vedere al resurselor, atât în timpul dezvoltării (antrenarea modelelor), cât și în timpul utilizării (inferențe pe secvențe video din viața reală).

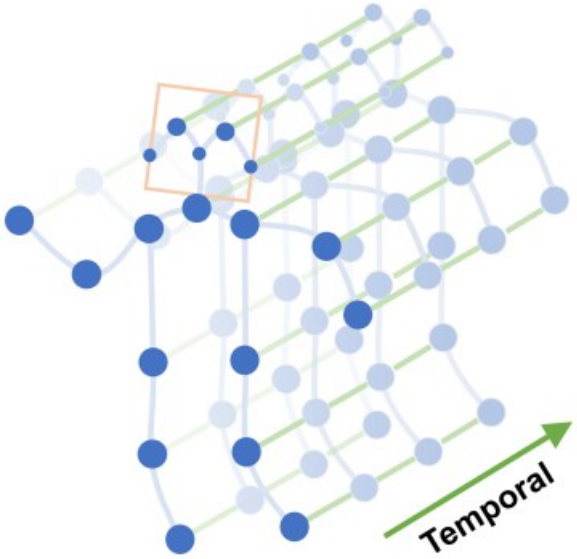


Figura 3.7: Graful spațio-temporal a unei secvențe de mișcare a scheletului uman. Punctele albastre reprezintă articulațiile corpului. Segmentele gri reprezintă muchiile spațiale, iar segmentele verzi – muchiile temporale. Figură preluată din [14].

3.2.3. Pose MoTion Representation

În capitolele de cercetare ale articolului [15], se prezintă o abordare intermediară între o modelare cât mai eficientă și cuprinzătoare a scheletului uman și extragerea informațiilor spațio-temporale direct din imagini. Metoda propusă utilizează hărțile de probabilitate (heatmaps), care sunt mai mult o reprezentare probabilistică de poziționare a articulațiilor în spațiu, pe care apoi le codifică color în dependență de momentul în timp al cadrului din secvența video corespunzător hărții de probabilitate. Aceste hărți sunt ulterior agregate într-o singură hartă ce înglobează atât informațiile spațiale (probabilistice) și temporale (codificate) ale articulațiilor. Acest proces, descris anterior, este ilustrat în Figura 3.8. În continuare, o singură hartă este oferită ca intrare într-o rețea convoluțională 2D, arhitectura căreia este reprezentată în Figura 3.9. Astfel această tehnică (care stă la baza articolului de referință a acestei lucrări) care oferă ca intrare unei rețele clasice de convoluție 2D o singură "imagine" (defapt o hartă de probabilitate) dă dovadă de o soluție promițătoare, deoarece combină beneficiile metodelor și arhitecturilor descrise anterior și anume păstreaza atât topologia (forma) corpului uman, cât și traiectoria temporală (poziția în timp) a acheletului în întregime, folosind spațiu redus de memorie și oferind posibilitatea de utilizare a unei arhitecturi mai puțin costisitoare din punct de vedere al calculelor și dimensiunii. Totuși, această reprezentare inițială a scheletului are și un dezevantaj în forma complexității algoritmilor de colorizare a fiecarei hărți în parte și sensibilitatea acestora. La fel, în vederea faptului că trebuie ținut cont de poziția relativă în timp a fiecarui cadru și respectiv a hărții de probabilitate corespunzătoare acelui cadru, se pierde mult timp în etapa de modelare a scheletului. Astfel această abordare merită studiată mai în detaliu pentru a găsi o soluție ce ar putea oferi echilibru între rapiditatea rezultatului produs în urma efectuării tuturor etapelor de modelare și inferență, precizia acestui rezultat și, în final, a resurselor necesare pentru realizarea acestei sarcini.

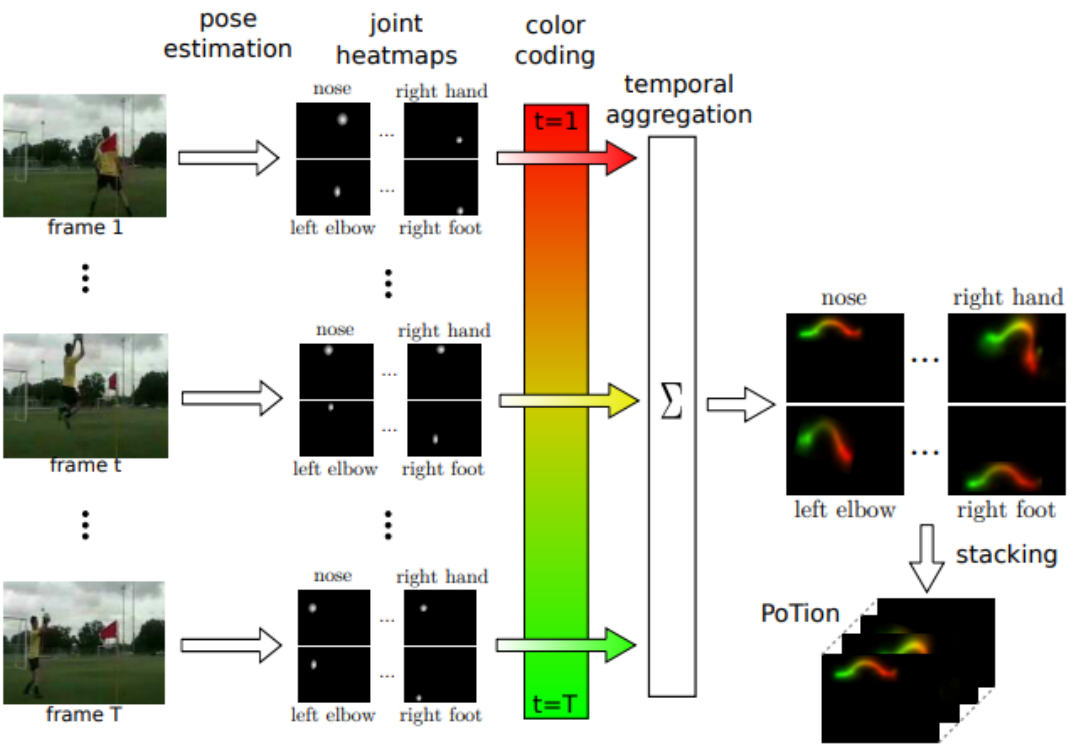


Figura 3.8: Reprezentarea PoTion. Imagine preluată din [15]

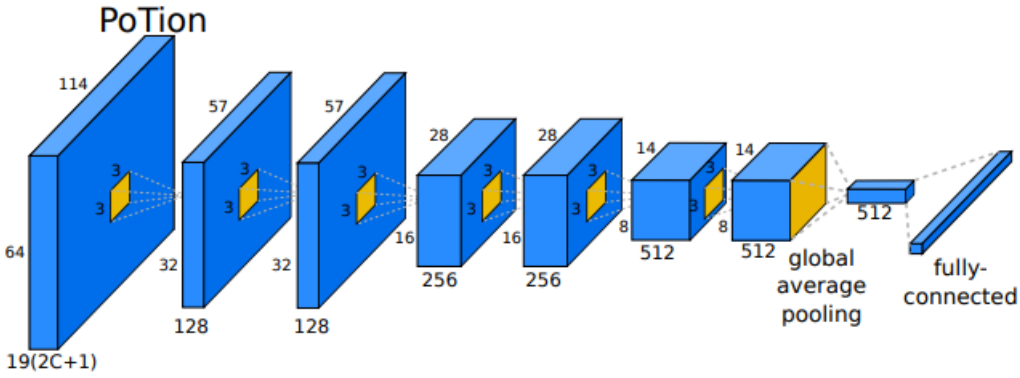


Figura 3.9: Arhitectura rețelei convoluționale 2D de clasificare utilizată pentru procesarea reprezentării PoTion. Figură preluată din [15].

3.3. Metode pentru numărarea repetițiilor

Pe langă recunoașterea tipului de exercițiu, este relevantă, în timpul antrenamentului, pentru o stimulare eficientă a organismului, și monitorizarea numărului de repetări efectuate.

3.3.1. Procesarea de semnal

În mod tradițional, numărarea acțiunilor recurente a fost abordată ca o problemă clasică de procesare a semnalelor. Abordările incipiente extrăgeau un semnal unidimensional din secvența video și aplicau Transformata Fourier pentru a identifica frecvența dominantă a mișcării. Totuși, această metodă prezintă limitări majore în scenariile din lumea reală, deoarece se bazează pe presupunerea restrictivă că mișcarea este perfect statică și staționară (cu o periodicitate absolut constantă). În timpul unui antrenament fizic real, viteza de execuție fluctuează inevitabil pe măsură ce apare oboseala musculară, iar camera poate suferi mișcări fine, aspecte ce diminuează eficacitatea instrumentelor bazate strict pe analiza Fourier.

Pentru a depăși aceste neajunsuri, lucrarea [16] propune o metodă avansată care abordează direct problema estimării repetițiilor în condiții realiste (non-statice și non-staționare). Inovația principală a autorilor constă în înlocuirea analizei Fourier cu Transformata Wavelet. Această alegere matematică permite sistemului să gestioneze variațiile de frecvență în timp, adaptându-se la schimbările de ritm ale exercițiului.

Din punct de vedere tehnic, metoda prezentată nu utilizează repere anatomice (cum ar fi scheletul uman), ci se bazează pe estimarea densă a fluxului optic (optical flow) aplicată peste o mască de segmentare a prim-planului, izolând astfel persoana de zgomotul de fundal. Analizând matematic câmpul de mișcare prin intermediul operatorilor de divergență (Div), gradient (Grad) și rotor (Curl), autorii au derivat o teorie prin care mișcările periodice 3D pot fi clasificate în 18 cazuri fundamentale de percepție în planul 2D al camerei video. Ulterior, sistemul analizează spectrul timp-frecvență și selectează automat cel mai intens semnal pentru a contoriza repetițiile.

Avantajul major al acestei abordări analitice este capacitatea de a extrage repetiții din videoclipuri fără a necesita antrenarea pe seturi masive de date etichetate. De altfel, metoda a demonstrat rezultate superioare în fața primelor modele de deep learning pe seturi de date complexe și neregulate (precum dataset-ul QUVA). Dezavantajul principal rezidă însă în dependența de calculul fluxului optic, un proces intensiv din punct de vedere computațional și sensibil la modificările bruște de iluminare sau la artefacte vizuale, neavând nivelul de abstractizare semantică oferit de sistemele bazate pe extragerea directă a poziției corpului uman.

3.3.2. Învățarea automată

O abordare modernă de tip deep learning, independentă de categoria acțiunii observate (class-agnostic), este propusă în lucrarea [17] prin modelul denumit RepNet. Spre deosebire de sistemele tradiționale antrenate să recunoască un set fix și limitat de exerciții, această soluție se concentrează pe estimarea perioadei cu care orice mișcare se repetă într-un flux video. Nucleul arhitecturii constă în utilizarea unei matrice de auto-similaritate temporală ca punct intermediar obligatoriu de procesare, acționând ca un canal restrictiv de informație. Practic, rețeaua compară caracteristicile fiecărui cadru cu toate celelalte cadre din secvență pentru a identifica structuri ciclice, comprimând datele spațiale într-o reprezentare cu un singur canal.

Principalul avantaj al acestei metode este capacitatea de generalizare, sistemul putând număra cu precizie repetițiile pentru acțiuni complet noi pe care nu le-a întâlnit în faza de antrenament (de la mișcări sportive, la activități cotidiene). Pe de altă parte, printre dezavantaje se numără riscul de supra-învățare (overfitting) cauzat de dependența inițială de datele sintetice, care nu reușesc să capteze mereu fluctuațiile și asimetriile naturale ale mișcărilor umane reale. În plus, generarea matricei de auto-similaritate presupune calcularea distanțelor între absolut toate perechile de cadre dintr-un videoclip, un proces cu o complexitate pătratică ce consumă cantități mari de memorie video atunci când sunt procesate înregistrări de lungă durată.

3.4. Sisteme multi-task pentru analiza exercițiilor fizice

Trecând la sisteme integrate care realizează simultan mai multe sarcini (multi-task), au fost studiate soluții deja existente care îndeplinesc în mare parte obietivele lucrării curente. Acestea prezintă metode, arhitecturi și abordări diverse, dar care au avantajele și dezavantajele lor. În continuare vor fi prezentate două lucrări pentru comparație și articolul de referință pe care este bazat studiul dat.

3.4.1. GymCam

În articolul [18] autorii prezintă GymCam, un sistem de viziune artificială conceput pentru a detecta și monitoriza mai mulți utilizatori simultan într-o sală de forță. Metoda se bazează pe identificarea mișcării prin extragerea traiectoriilor fluxului optic dens (dense optical flow), apoi un perceptron multistrat (Multilayer Perceptron sau MLP) clasifică traiectoriile ca fiind exercițiu sau non-exercițiu, utilizând un vot majoritar pentru a uniformiza rezultatele. În continuare punctele (ce reprezintă articulațiile) sunt grupate împreună dacă apar în același timp și în cadrul unor zone delimitate predifinite pentru fiecare stație de lucru, apoi pentru a separa persoanele aflate una lângă alta, sistemul verifică faza mișcării, traiectoriile care nu sunt sincronizate sunt împărțite în clustere diferite. Toate traiectoriile dintr-un cluster sunt mediate și netezite într-o singură traiectorie reprezentativă, care ulterior este folosită pentru executarea ultimelor două sarcini: numărarea repetițiilor cu ajutorul unui regresor MLP ce analizează caracteristicile de frecvență ale traiectoriei combinate pentru a estima numărul de repetiții și recunoașterea exercițiului prin intermediul unui perceptron multistrat ce clasifică tipul de exercițiu prin votul majoritar al predicțiilor efectuate pe ferestre glisante ale datelor de mișcare.

Avantajul major este aplicabilitatea în scene aglomerate (cu multe persoane). Dezavantajul este acuratețea relativ scazută în recunoaștere (aproximativ 85%) și imposibilitatea de a distinge exerciții vizual similare din cauza că nu folosesc o analiză profundă a posturii întregului corp.

3.4.2. Sisteme bazate pe unghiurile articulațiilor

În lucrarea [19], este propus un sistem integrat ce folosește OpenPose pentru a extrage punctele cheie 3D, pe care le convertește în coordonate 2D pentru a servi drept date de calcul a unghiurilor articulațiilor principale (folosind produsul scalar al vectorilor) în funcție de tipul de mișcare (împins sau tras). La fel, aceste ungiuri sunt folosite pentru a detecta repetițiile corecte prin monitorizarea dacă unghiul trece de punctul critic (mijlocul razei de mișcare) și atinge ambele limite (superioară și inferioară) ale exercițiului.

Avantajul acestui sistem este o precizie ridicată în numărarea repetițiilor și capacitatea de a rula în timp real, oferind și un feedback în formă de clasificare a repetițiilor ca fiind corecte sau incorecte. Ca dezavantaj, metoda clasificării repetițiilor este rigid codificată, iar sistemul impune ca, înainte de începerea numărării, să se cunoască deja ce fel de exercițiu este efectuat, pentru ca algoritmul să știe ce unghiuri să urmărească, limitând astfel gradul de automatizare reală.

3.4.3. RepPenn

Cel mai relevant sistem în contextul acestei lucrări este prezentat de Yu et al. în [20]. Metoda se bazează pe preluarea hărților de probabilitate generate de o rețea pentru detecția poziției cu mai multe etape (Multi-Stage Pose Network sau MSPN) și comprimarea acestora pe axele verticale și orizontale pentru a obține o hartă proiectată (Projected map sau P-map). Această hartă 2D statică este introdusă într-o rețea multi-task cu două ramuri, una pentru clasificarea exercițiului efectuat și alta pentru regresia numărului de repetări.

Avantajul remarcabil este că rețeaua învață concomitent atât recunoașterea cât și regresia pe aceleași date spațio-temporale extrem de comprimate, economisind resurse. Dezavantajele metodei propuse de autori vizează dependența de modelul MSPN (care este greu de implementat sau cel puțin de accesat în medii web/mobile față de YOLO) și limitarea la videoclipuri de antrenament continue și statice.

În urma analizei literaturii de specialitate, se constată o evoluție de la metodele lente de prelucrare secvențială a imaginilor către reprezentări comprimate ale scheletului uman. Deși arhitecturile 3D CNN și modelele bazate pe serii de timp precum memorii de lungă și scurtă

durată (Long Short-Term Memory sau LSTM) oferă rezultate bune, ele necesită putere de calcul industrială. Pe de altă parte, metodele analitice care folosesc doar unghiurile articulare suferă de o lipsă de flexibilitate și necesită intervenție umană prealabilă. Prezenta lucrare de licență preia paradigma propusă de Yu et al., încercând să îmbunătățească și să eficientizeze stadiul actual prin înlocuirea modelului MSPN cu arhitectura YOLO. Astfel, sistemul propus în capitolele următoare îmbină avantajele detecției ”single-stage” de o înaltă performanță cu avantajele compresiei de tip P-map și ale învățării multi-task, oferind o alternativă rapidă și automatizată la soluțiile existente.

Capitolul 4. Analiză și fundamentare Teoretică

Scopul acestui capitol este de a detalia principiile fundamentale care guvernează funcționarea sistemului propus, oferind o perspectivă teoretică asupra modului în care informația brută este transformată în cunoaștere semantică. În paginile ce urmează, soluția va fi analizată prin prisma modelelor abstracte de procesare, a algoritmilor de viziune artificială și a paradigmei de învățare multi-task, punând accent pe justificarea logică a fiecărei componente arhitecturale. Astfel este prezentată fundamentarea teoretică a următoarelor tehnici:

- Estimarea poziției umane (Pose estimation)
- Modelarea matematică a hărților de probabilitate (Heatmaps)
- Modelarea matematică a hărților proiectate (P-maps)
- Învățarea cu mai multe sarcini (multi-task)
- Modele abstracte pentru clasificare și regresie

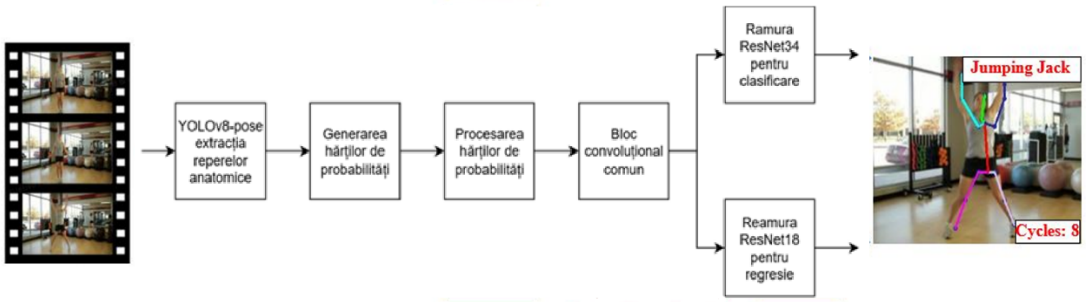


Figura 4.1: Schema bloc a pipeline-ului logic de procesare. Ca intrare sistemul are o secvență video, iar ieșirea reprezintă video-ul original peste care este suprapus scheletul. Este oferit în format de text, tipul exercițiului și numărul de repetări detectate.

4.1. Arhitectura logică a sistemului

Arhitectura logică a sistemului este concepută sub forma unui flux modular de procesare (pipeline), în care fiecare componentă are rolul de a crește gradul de abstractizare al datelor de intrare. Din punct de vedere teoretic, sistemul nu tratează videoclipul ca pe o simplă succesiune de pixeli colorați, ci ca pe o manifestare dinamică a unui model cinematic uman. Această perspectivă permite împărțirea arhitecturii în patru blocuri logice fundamentale: modulul de extracție a reperelor anatomice, modulul de modelare a distribuției spațiale (Heatmaps), modulul de compresie spațio-temporală (P-maps) și, în final, motorul de inferență multi-task bazat pe rețele neuronale convoluționale. Această structură modulară prezentată în Figura 4.2, asigură o separare a responsabilităților, facilitând optimizarea individuală a fiecărei etape, atât în perioada de dezvoltare, cât și în timpul utilizării, fără a compromite integritatea fluxului global.

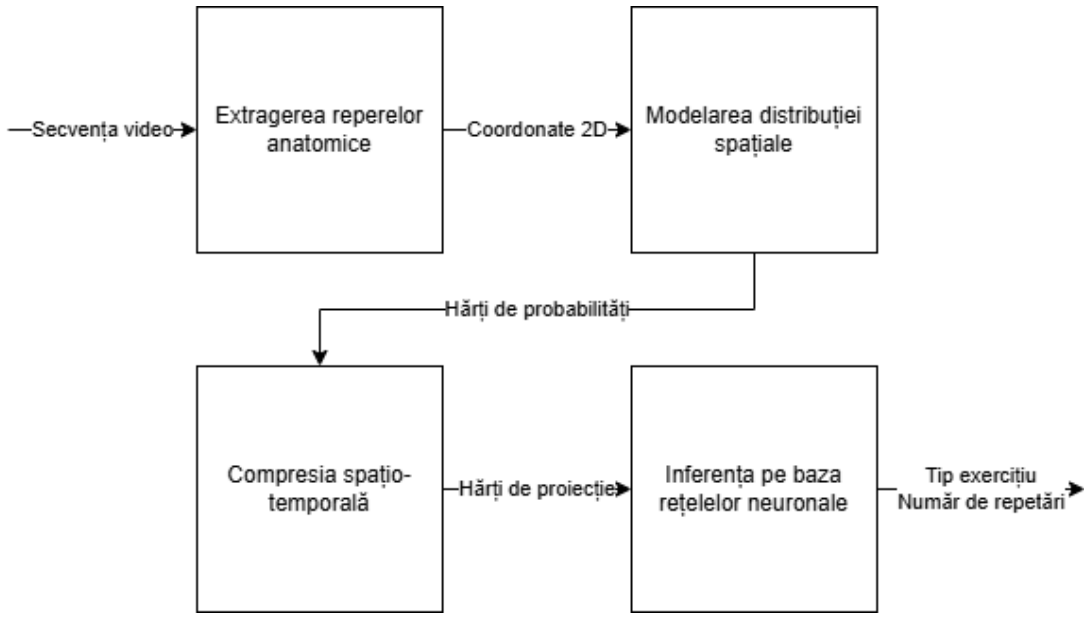


Figura 4.2: Arhitectura logică și modulele de bază a nucleului de procesare

Logica de bază a soluției propuse se fundamentează pe principiul filtrării informației redundante în primele etape ale procesării. În viziunea artificială clasică, procesarea video este adesea penalizată de prezența zgomotului vizual (fundal variabil, condiții de iluminare oscilante, elemente statice irelevante). Alegerea de a extrage scheletul uman într-o fază incipientă reprezintă o decizie de design menită să transforme un spațiu de intrare mare (milioane de valori RGB) într-un spațiu vectorial compact (coordonate carteziene). Din punct de vedere teoretic, acest pas acționează ca un mecanism de atenție biologică, forțând modelul de învățare să se concentreze exclusiv pe trăsăturile dinamice care contează. Astfel, sistemul devine rezistent la variațiile de mediu, deoarece întreaga analiză ulterioară se bazează pe geometria mișcării, nu pe aspectul vizual al cadrelor.

Tranziția de la coordonatele geometrice discrete la reprezentările de tip ”Projected Maps” (P-maps) constituie inovația logică centrală a arhitecturii. În loc să utilizeze volume de date 4D (lățime, înălțime, timp și canale), care ar necesita o putere de calcul mare, sistemul propune colapsarea dimensiunii temporale într-un plan bidimensional. Din punct de vedere funcțional, acest proces transformă o problemă de analiză secvențială într-o problemă de recunoaștere a tiparelor de textură. Logica acestei alegeri rezidă în faptul că o mișcare periodică (precum o flotare sau o genuflexiune) lasă o ”urmă” vizuală distinctă atunci când este proiectată în spațiu, un exemplu al acestui efect poate fi văzut în Figura 4.3. Astfel, arhitectura reușește să păstreze corelațiile temporale (ordinea mișcărilor) fără a suporta costul computațional al convoluțiilor 3D sau al rețelelor recurente.

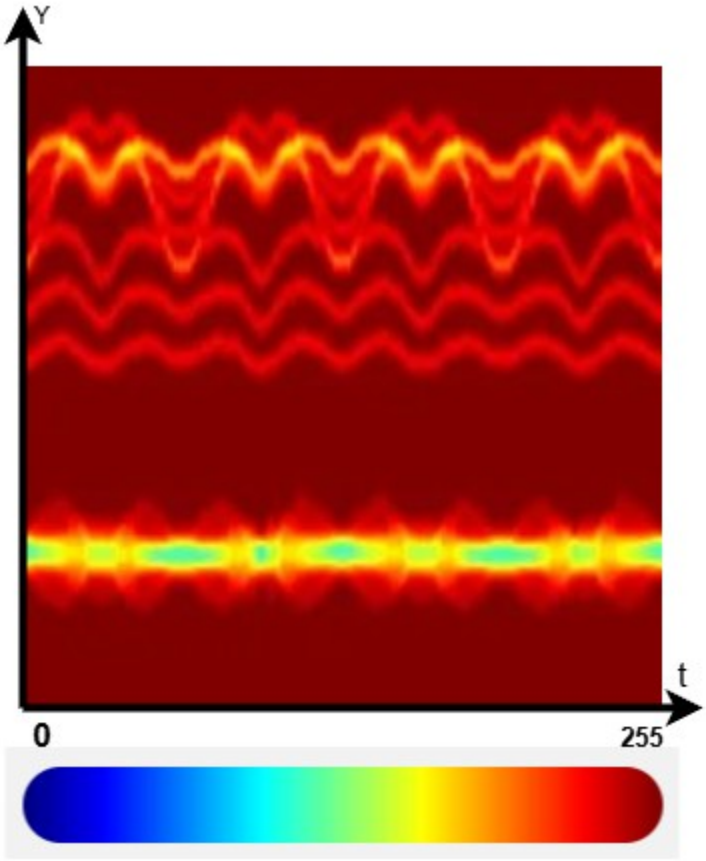


Figura 4.3: Harta de probabilitate (heatmap) suprapusă a celor 17 articulații detectate ale unui om ce efectuează exercițiul „jumping-jacks”, realizând 4 repetări. Axa t reprezintă timpul, axa Y – deplasamentul vertical, iar codificarea „COLORMAP_JET”[1] din grayscale în culori este prezentată sub imagine.

În etapa finală a arhitecturii, modulul de inferență adoptă paradigma învățării multi-task (Multi-Task Learning), reprezentând structura funcțională a deciziei. Această componentă este proiectată să exploateze combinația dintre clasificarea acțiunii și numărarea repetițiilor. Din perspectivă teoretică, cele două sarcini sunt strâns corelate: pentru a număra corect ciclurile unei mișcări, sistemul trebuie să înțeleagă mai întâi natura acelei mișcări. În cadrul arhitecturii propuse, acest lucru este realizat printr-o structură de tip ”Y”, unde straturile inițiale ale rețelei (backbone) învață o reprezentare abstractă comună a mișcării, care alimentează ulterior două ramuri specializate. Această abordare funcțională nu doar că reduce timpul total de execuție, dar îmbunătățește și capacitatea de generalizare a modelului, deoarece învățarea simultană a celor două task-uri forțează rețeaua să ignore trăsăturile specifice zgomotului și să se concentreze pe trăsăturile biomecanice universale.

4.2. Fundamentele estimării poziției umane

Estimarea poziției umane reprezintă o problemă fundamentală în viziunea artificială, al cărei scop teoretic este maparea unui spațiu de intrare vizual, de înaltă dimensionalitate, într-un spațiu topologic de dimensionalitate redusă. Formal, considerând o imagine de intrare $I \in \mathbb{R}^{W \times H \times 3}$ (unde W și H reprezintă lățimea și înălțimea, iar 3 reprezintă canalele de culoare RGB), obiectivul sistemului este de a

[1] OpenCV colormaps - site

aproxima o funcție neliniară $F(I)$ care să returneze un graf structural $G = (V, E)$. În acest graf abstract, setul de vârfuri $V = \{p_1, p_2, \dots, p_k\}$ reprezintă cele K puncte cheie anatomice (articulațiile), iar E reprezintă conexiunile cinematice dintre acestea (structura scheletică).

În cadrul arhitecturii propuse, modelarea acestui graf se concentrează pe predicția precisă a setului de vârfuri V. Pentru a înțelege principiile funcționale ale soluției alese, este necesară analiza modelelor abstracte care guvernează extracția acestor repere spațiale.

4.2.1. Paradigme abstracte de procesare a spațiului vizual

Din punct de vedere teoretic, problema estimării poziției umane a fost abordată istoric prin două paradigme clasice: Top-Down (de sus în jos) și Bottom-Up (de jos în sus).

Modelul Top-Down tratează problema ca pe un proces secvențial în două etape: inițial, un detector localizează persoana extrăgând o regiune de interes (Bounding Box), iar ulterior, un al doilea model estimează poziția articulațiilor exclusiv în interiorul acelei regiuni trunchiate. Deși această metodă oferă o acuratețe ridicată prin izolarea subiectului de fundal, ea prezintă o vulnerabilitate teoretică majoră: complexitatea sa temporală este liniar dependentă de numărul de subiecți din cadru, notată ca $O(N)$. Mai mult, orice eroare de încadrare generată în prima etapă se propagă ireversibil în estimarea scheletului.

Modelul Bottom-Up, pe de altă parte, inversează fluxul logic. Algoritmul detectează mai întâi toate articulațiile din imagine, indiferent de apartenența lor (de exemplu, identifică toate mâinile și toți genunchii), urmând ca un algoritm euristic complex de grupare (precum Part Affinity Fields) să asocieze aceste puncte pentru a forma schelete individuale. Complexitatea acestui model este teoretic constantă în raport cu numărul de persoane, $O(1)$, însă procesul de grupare este un algoritm de optimizare globală extrem de costisitor (deseori rezolvat prin corelarea grafurilor bipartite), fiind sensibil la zgomot și suprapuneri (ocluziuni).

Soluția teoretică adoptată în acest proiect se bazează pe o a treia paradigmă, mult mai modernă: modelul **Single-Stage (într-o singură etapă)** de tip Anchor-Free. Această arhitectură abstractizează întregul proces, realizând detecția globală și regresia punctelor cheie simultan, eliminând complet necesitatea unor algoritmi euristici de grupare sau a unei pre-procesări a regiunilor de interes.

4.2.2. Teoria detecției Anchor-Free

Metodele tradiționale de detecție se bazează pe conceptul de ancore (prior boxes) – o grilă de cutii cu dimensiuni predefinite suprapuse peste imagine, pe care rețeaua încearcă să le ajusteze. Din perspectivă teoretică, ancorele introduc o rigiditate matematică ineficientă pentru obiecte non-rigide, cum este corpul uman aflat în mișcare. Corpul uman suferă deformări topologice extreme (de exemplu, o persoană care face o genuflexiune are un raport de aspect complet diferit față de o persoană care stă în picioare).

Algoritmul propus utilizează o logică de tip Center-Based Representation (Reprezentare bazată pe centru). În acest model teoretic, prezența unei persoane nu mai este definită de o cutie, ci de un singur punct central al masei sale în spațiul trăsăturilor. Imaginea de intrare este divizată într-o rețea de celule (grid). Dacă centrul teoretic al unei persoane cade în interiorul unei anumite celule, acea celulă devine „responsabilă” pentru predicția completă a topologiei acelei persoane.

Astfel, problema se transformă dintr-o ajustare geometrică într-o problemă de clasificare spațială și regresie directă a vectorilor. Din punctul central detectat, rețeaua calculează simultan K vectori de deplasare care indică direcția și distanța exactă către fiecare articulație în parte. Această abstractizare matematică permite sistemului să înțeleagă corpul ca pe o entitate unificată radiind dintr-un punct central, rezolvând problema asocierii articulațiilor la individ.

4.2.3. Modelarea matematică a coordonatelor

Odată ce centrul subiectului este localizat, sistemul trebuie să prezică natura structurală a celor K repere (în cazul acestui proiect, $K = 17$, reprezentând standardul formatului COCO pentru corpul uman). Din punct de vedere matematic, fiecare punct cheie p_i este modelat ca un tuplu tridimensional:

$$p_i = (x_i , y_i , v_i)$$

Aici, x_i și y_i reprezintă coordonatele continue în planul bidimensional al imaginii, iar $v_i \in [0, 1]$ este o variabilă de stare ce definește probabilitatea de vizibilitate a acelei articulații.

Valoarea v_i este de o importanță majoră în analiza activității fizice. Dinamica antrenamentelor presupune adesea obturarea anumitor membre (de exemplu, un braț ascuns în spatele trunchiului sau ieșirea parțială din unghiul de vizualizare). Abordarea teoretică presupune modelarea lui v_i printr-o funcție de activare sigmoidă:

$$v_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}$$

unde z_i este ieșirea liniară netransformată a rețelei pentru canalul de vizibilitate. Această funcție asigură maparea rezultatului într-un spațiu probabilistic strict delimitat între 0 (absent sau invizibil) și 1 (vizibil cu certitudine maximă). Dacă sistemul determină că v_i scade sub un anumit prag teoretic, punctul respectiv este marcat ca invizibil, prevenind astfel introducerea unor valori (x_i , y_i) ironate în etapele ulterioare de transformare spațio-temporală. Un exemplu în care este utilă filtrarea acestei valori se poate observa în Figura 4.4, care ilustrează o utilizare din lumea reală: o persoană își fixează telefonul pentru înregistrarea antrenamentului și apare, doar după câteva secunde, în cadru, astfel, în prima parte a secvenței video nu se detectează valori false.

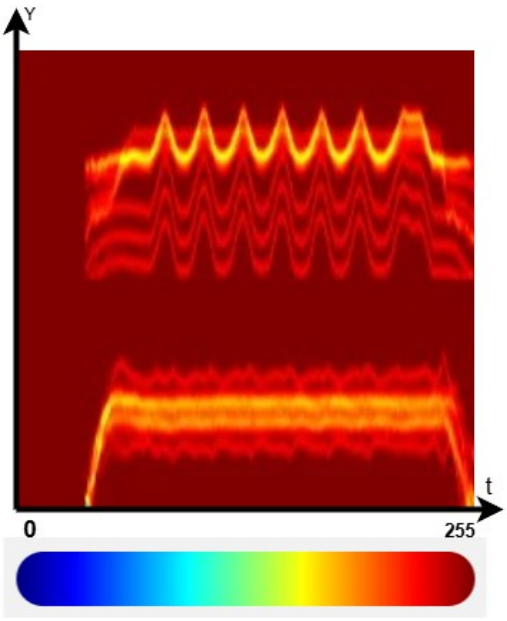


Figura 4.4: Harta de probabilitate (heatmap) suprapusă a celor 17 articulații detectate ale unui om ce efectuează exercițiul „pull-up" (tracțiuni la bară), realizând 7 repetări, cu pauză în poziția de sus, la ultima repetare. La început omul nu este în cadru și apare peste câteva secunde. Axa t reprezintă timpul, axa Y – deplasamentul vertical, iar codificarea "COLORMAP_JET" din grayscale în culori este prezentată sub imagine.

4.2.4. Funcția de pierdere pentru repere anatomice

Valoarea teoretică a oricărui model de viziune artificială rezidă în modul în care acesta „învață" din erori, proces guvernat de funcția de pierdere (Loss Function). Pentru estimarea poziției umane într-o arhitectură single-stage, funcția de pierdere trebuie să echilibreze optimizarea coordonatelor geometrice cu predicția vizibilității.

Funcția globală de pierdere pentru estimarea posturii, notată L_{pose}, este o combinație liniară de funcții individuale de cost:

$$L_{pose} = \lambda_{coord} L_{coord} + \lambda_{vis} L_{vis}$$

unde λ_{coord} și λ_{vis} sunt factori de ponderare care controlează importanța relativă a fiecărei componente în timpul calculării gradientului (Backpropagation).

Calculul erorii coordonatelor (L_{coord}):

Din punct de vedere teoretic, utilizarea unei simple distanțe Euclidiene (L2 Loss) sau L1 Loss pentru penalizarea coordonatelor prezintă un defect logic: nu este invariantă la scară. O eroare de 5 pixeli în estimarea umărului pentru o persoană aflată foarte aproape de cameră este acceptabilă, dar aceeași eroare de 5 pixeli pentru o persoană aflată la distanță mare (unde întreaga persoană ocupă doar 50 de pixeli) reprezintă un procent mare de eroare.

Pentru a corecta această deficiență teoretică, sistemul folosește metrica bazată pe Similaritatea Punctelor Cheie ale Obiectului (Object Keypoint Similarity - OKS). OKS este o măsură de similaritate non-liniară, definită matematic astfel:

$$OKS = \frac{\sum_i \exp\left(-\frac{d_i^2}{2s^2k_i^2}\right)\delta(v_i > 0)}{\sum_i \delta(v_i > 0)}$$

Unde:

- $\boldsymbol{d_i}$ este distanța Euclideană dintre punctul cheie prezis și punctul cheie real.
- $\boldsymbol{s}$ este factorul de scară al persoanei (calculat, de regulă, pe baza ariei cutiei de încadrare), asigurând invarianța la dimensiune.
- $\boldsymbol{k_i}$ este o constantă de normalizare per-articulație (per-keypoint constant). Din punct de vedere anatomic, ochiul uman tolerează erori mai mari la nivelul trunchiului sau șoldului, dar este foarte sensibil la erori în localizarea ochilor sau a nasului. Parametrul k_i modelează matematic această toleranță anatomică.
- $\boldsymbol{\delta(v_i > 0)}$ este funcția indicator (Kronecker delta) care asigură că doar punctele vizibile contribuie la calculul erorii.

Funcția de pierdere derivată din această metrică este definită pur și simplu ca $L_{coord} = 1 - OKS$. Prin minimizarea acestei funcții, modelul este forțat teoretic să reducă distanța dintre predicție și realitate într-un mod proporțional cu scara subiectului și cu importanța anatomică a articulației.

Calculul erorii de vizibilitate (L_{vis}):

Sarcina de a prezice vizibilitatea v_i este formulată teoretic ca o clasificare binară independentă pentru fiecare punct cheie. Astfel, funcția de pierdere aplicată este Binary Cross-Entropy (BCE):

$$L_{vis} = -\frac{1}{K}\sum_{i=1}^{K}[y_i \log(\hat{v}_i) + (1 - y_i)\log(1 - \hat{v}_i)]$$

unde $y_i \in \{0,1\}$ este starea reală de vizibilitate a articulației, iar $\hat{v}_i$ este probabilitatea prezisă de model.

Această separare logică și matematică a funcțiilor de pierdere permite arhitecturii single-stage să deducă o formă structurală coerentă a corpului uman, tratând cu succes atât variațiile de scară, cât și lipsa informației vizuale cauzate de ocluziuni. Rezultatul final al acestui bloc teoretic este un set purificat de vectori topologici, pregătiți pentru a fi transformați în distribuții gaussiene în etapa următoare a sistemului.

4.3. Modelarea matematică a hărților de probabilitate

Transformarea datelor dintr-un format discret, reprezentat de coordonatele carteziene (x, y), într-un format continuu și dens reprezintă una dintre cele mai importante decizii arhitecturale în proiectarea sistemelor moderne de viziune artificială. Această etapă, cunoscută sub denumirea de modelare prin hărți de probabilitate (heatmaps), acționează ca un pod conceptual între geometria brută a punctelor cheie și capacitatea de procesare spațială a rețelelor neuronale convoluționale (CNN). Din punct de vedere teoretic, utilizarea directă a coordonatelor într-o problemă de regresie prezintă limitări severe de neliniaritate și o dificultate ridicată în convergența modelului, deoarece o mică variație în pixeli nu se traduce întotdeauna printr-o schimbare netedă în spațiul funcțiilor de activare.

Hărțile de probabilitate rezolvă această problemă prin redefinirea localizării unei articulații nu ca pe un punct infinit de mic, ci ca pe o distribuție spațială de probabilitate. Această abordare se fundamentează pe conceptul de ”Confidence Map” (hartă de încredere), unde valoarea fiecărui pixel din matrice reflectă certitudinea matematică a prezenței reperului anatomic în acea locație specifică (care reprezintă, de fapt, o hartă de probabilități). Această tranziție permite rețelei neuronale să opereze într-un domeniu în care trăsăturile spațiale sunt corelate local, profitând de operația de convoluție pentru a extrage modele de textură și formă care sunt intrinsec legate de postura umană.

4.3.1. Fundamentarea probabilistică prin distribuții Gaussiene

Din punct de vedere analitic, pentru fiecare dintre cele K puncte cheie identificate (în cazul standardului COCO, $K = 17$), sistemul generează o matrice bidimensională independentă $H_k \in \mathbb{R}^{W \times H}$. Această matrice nu este una binară, ci este construită prin aplicarea unei funcții de densitate de probabilitate. Alegerea teoretică dominantă și cea utilizată în acest proiect este distribuția Gaussiană izotropă bidimensională. Motivația acestei alegeri rezidă în proprietățile matematice de netezire și derivabilitate ale funcției exponențiale, care facilitează un flux stabil al gradientului în timpul etapei de propagare înapoi (backpropagation).

Modelarea matematică a valorii unui pixel situat la coordonatele (x, y) în harta de probabilitate corespunzătoare articulației k, centrată la coordonatele (μ_{xk}, μ_{yk}) este definită prin următoarea expresie:

$$H_k(x, y) = \exp\left(-\frac{(x - \mu_{xk})^2 + (y - \mu_{yk})^2}{2\sigma^2}\right)$$

Această formulă transformă distanța euclidiană pătratică dintre orice punct din imagine și centrul prezis al articulației într-o valoare scalară cuprinsă în intervalul $(0, 1]$. Teoretic, în centrul exact al articulației, valoarea hărții este maximă, adică 1, scăzând exponențial pe măsură ce distanța crește. Această descreștere controlată este de ajutor pentru antrenarea rețelelor adânci: ea oferă modelului un ”semnal de proximitate”. Chiar dacă predicția rețelei este ușor decalată față de centrul real, ea va primi totuși un feedback numeric (un gradient non-nul) care să o ghideze către poziția corectă, spre deosebire de o reprezentare de tip ”punct” unde orice eroare, oricât de mică, ar rezulta într-o pierdere totală a semnalului de corecție.

4.3.2. Rolul teoretic al parametrului de dispersie

Parametrul σ (deviația standard) nu este doar o variabilă de scară, ci un element de reglaj fin al preciziei teoretice a sistemului. El definește ”raza de influență” a fiecărui punct cheie în spațiul hărții de probabilitate. Din punct de vedere logic, valoarea lui σ trebuie să reflecte un echilibru între localizarea precisă și toleranța la erori.

Dacă valoarea σ este aleasă prea mică, distribuția devine extrem de îngustă, apropiindu-se de funcția delta a lui Dirac. În acest scenariu, spațiul de căutare al rețelei devine prea rigid, iar modelul va avea dificultăți majore în a ”găsi” articulația în etapele inițiale ale antrenamentului, ducând la o convergență extrem de lentă sau chiar la blocarea în minime locale. Pe de altă parte, o valoare prea mare pentru σ introduce o ambiguitate spațială excesivă. Deoarece corpul uman este o structură articulată, puncte cheie apropiate (cum ar fi încheietura mâinii și cotul în timpul unei flexii maxime) ar putea genera hărți de probabilitate care se suprapun semnificativ. Acest fenomen de

interferență ar putea împiedica ramura de clasificare să distingă corect detaliile fine ale mișcării, ducând la o scădere a acurateței a sistemului.

4.3.3. Structura de tensor și reprezentarea multi-canal

Rezultatul procesului de modelare pentru un singur cadru video este un tensor tridimensional de dimensiuni $K \times W \times H$. Din punct de vedere funcțional, acest tensor poate fi vizualizat ca o stivă de hărți de probabilitate, unde fiecare porțiune a canalului poartă informația despre o singură articulație specifică. Această structură este fundamentală pentru arhitecturile de tip CNN, deoarece permite nucleelor de convoluție să învețe nu doar localizarea individuală, ci și relațiile spațiale relative dintre canale (de exemplu, relația geometrică fixă dintre umăr, cot și încheietură).

Această reprezentare densă aduce un avantaj teoretic major în gestionarea ocluziilor și a incertitudinii. În cazul în care o articulație este parțial acoperită, harta de probabilitate va reflecta acest lucru printr-o amplitudine scăzută a vârfului Gaussian sau printr-o distribuție mai aplatizată, oferind straturilor ulterioare de decizie o măsură cuantificabilă a încrederii în acea detecție. Mai mult, prin normalizarea acestor hărți la o rezoluție standard (de regulă 64×64 sau 128×128 pixeli), sistemul dobândește o invarianță la rezoluția originală a videoclipului, asigurând că trăsăturile extrase sunt consistente indiferent de sursa de captură.

În concluzie, modelarea prin hărți de probabilitate reprezintă o abstractizare necesară care transformă topologia discretă a scheletului uman într-un câmp continuu de probabilități. Această fundamentare teoretică permite utilizarea întregului arsenal matematic al rețelelor convoluționale pentru a interpreta mișcarea, oferind o suprafață de optimizare netedă și o reprezentare bogată în informații contextuale, pregătind astfel terenul pentru următoarea etapă: comprimarea acestor volume masive de date în hărți proiectate (P-maps).

4.4. Fundamentarea teoretică a hărților de proiecție

Procesarea secvențelor video introduce o nouă dimensiune fundamentală în analiza datelor: axa timpului (T). Din punct de vedere matematic, integrarea acestei dimensiuni transformă o problemă de geometrie plană într-o problemă de analiză a seriilor de timp multi-variate. Un videoclip preprocesat sub formă de hărți de probabilitate, așa cum a fost descris în secțiunea anterioară, este reprezentat teoretic printr-un tensor 4D, definit ca $V \in \mathbb{R}^{T \times K \times W \times H}$, unde T reprezintă numărul de cadre, K numărul de articulații, iar W și H dimensiunile spațiale ale cadrului.

4.4.1. Problema dimensionalității și limitările modelelor 3D

Analiza directă a acestui tensor 4D presupune, în mod convențional, utilizarea convoluțiilor tridimensionale (3D CNN). Deși aceste modele sunt capabile să extragă corelații spațio-temporale de nivel înalt prin glisarea nucleelor (kernels) simultan pe axele spațiale și pe axa temporală, ele sunt afectate sever de ”blestemul dimensionalității” (curse of dimensionality). Funcția de complexitate a unui model 3D crește exponențial odată cu adăugarea dimensiunii temporale, ceea ce determină o amprentă de memorie mare și un timp de inferență restrictiv.

Mai mult, din perspectivă teoretică, tensorul V este extrem de rarifiat (sparse). Deoarece corpul uman acoperă o fracțiune relativ mică din volumul spațiului vizual, majoritatea elementelor din tensorul V conțin valori nule sau extrem de apropiate de zero. Aplicarea unor operații matematice costisitoare pe un spațiu majoritar vid reprezintă o ineficiență arhitecturală majoră. Astfel, problema centrală devine

reducerea dimensionalității acestui spațiu, asigurând conservarea informației de bază despre dinamica mișcării și eliminarea redundanței.

4.4.2. Conceptul proiecției ortogonale și colapsarea axelor spațiale

Soluția teoretică propusă în lucrarea [20] presupune transformarea spațiului 4D într-o reprezentare bidimensională condensată, printr-un proces matematic de proiecție ortogonală. Acest concept, denumit ”Projected Maps” (P-maps), vizează colapsarea succesivă a fiecărei axe spațiale, transferând istoria mișcării pe axa rămasă liberă. Practic, se realizează o translație a axei timpului T într-o axă spațială pură, permițând utilizarea rețelelor convoluționale 2D clasice pentru a analiza o secvență temporală globală.

Proiecția ortogonală se realizează prin aplicarea unui operator de agregare peste o anumită dimensiune a tensorului. În teoria procesării imaginilor, cei mai comuni operatori sunt media ($mean$) și maximul (max). În contextul hărților de probabilitate gaussiene, operatorul teoretic optim este valoarea maximă (max), deoarece acesta reține coordonata cu cea mai mare certitudine (vârful funcției Gaussiene) și ignoră restul spațiului cu probabilitate nulă. Însă, a fost aleasă, ca și în articolul original, utilizarea agregării prin medie, în contextul în care hărțile au fost obținute din coordonate discrete și apoi netezite cu ajutorul distribuției Gaussiane izotrope bidimensionale, pentru a obține o reprezentare mai fluidă a mișcării.

Matematic, proiecția pe axa verticală (agregarea de-a lungul axei orizontale X) a matricii inițiale I de dimensiune $T \times K \times W \times H$, pentru o articulație k la momentul t este definită de ecuația:

$$P_y[t,k,h] = \frac{1}{W}\sum_{w=1}^{W} I[t,k,h,w]$$

Această operație reduce tensorul la o matrice P_y de dimensiune $T \times K \times H$. Din punct de vedere biomecanic, P_y surprinde exclusiv evoluția mișcărilor pe axa verticală (gravitațională) în timp. Aici, axa orizontală a noii imagini reprezintă timpul, iar axa verticală reprezintă deplasarea pe Y. De exemplu, într-un exercițiu de genuflexiuni (squats), mișcarea articulației șoldului va fi proiectată în această matrice ca o formă de undă sinusoidală, reprezentând secvențele repetate de flexie și extensie.

În mod complementar, proiecția pe axa orizontală (agregarea de-a lungul axei verticale Y) se definește teoretic prin:

$$P_x[t,k,w] = \frac{1}{H}\sum_{h=1}^{H} I[t,k,h,w]$$

Această operație generează o matrice de dimensiune $T \times K \times W$. Teoretic, P_x codifică istoria mișcărilor laterale sau de abducție/adducție orizontală. În cazul unui exercițiu de tip ”jumping jacks”, această proiecție va ilustra formarea unor modele divergente și convergente pentru articulațiile gleznelor și încheieturilor, reprezentând depărtarea și apropierea membrelor față de planul de mijloc al corpului.

4.4.3. Unificarea canalelor și conversia temporalității în textură

Următorul pas în modelarea hărților proiectate este unificarea proiecțiilor. Deoarece sistemul monitorizează K articulații, în T cadre și am proiectat mișcarea atât pe axa verticală, cât și pe cea orizontală, se obțin două matrici tridimensionale. Ele pot

fi concatenate pe axa canalelor sau însumate, astfel, formând un tensor global de intrare pentru modelul decizional, definit ca $P \in \mathbb{R}^{T\times K\times(H+W)}$.

În continuare, pentru a obține valori standard pentru o imagine în fiecare pixel (între 0 și 255), valorile din hărți, care sunt în intrevalul $(0, 1]$, sunt normalizate cu următoarea formulă:

$$N = 255 \times \frac{P - \min(P)}{\max(P) - \min(P)}$$

Unde:

- **min**(P) este funcția ce calculează valoarea minimă din matricea P
- **max**(P) este, respectiv, funcția ce calculează valoarea maximă din matricea P

4.4.4. Invarianța temporală și normalizarea spațiului de trăsături

O proprietate fundamentală a oricărui sistem de recunoaștere a acțiunilor este invarianța la durata videoclipului. Un utilizator poate executa 10 flotări în 15 secunde sau în 30 de secunde. Tensorul obținut în etapa anterioară va avea o lățime (dimensiunea T) variabilă, strict dependentă de durata înregistrării. Deoarece rețelele convoluționale (CNN) necesită adesea tensori de intrare cu o dimensiune fixă, este obligatorie o etapă de normalizare temporală.

Această normalizare este realizată matematic printr-o transformare afină de redimensionare (interpolare biliniară sau bicubică), mapând dimensiunea variabilă $T \times (H + W)$ într-un spațiu bidimensional izotrop de dimensiune constantă, de exemplu $(D \times D)$, unde D este rezoluția standard a modelului (frecvent 224 pixeli).

$$P' = F_{resize}(N, D \times D)$$

Efectul teoretic al acestei transformări este că modificarea vitezei de execuție a exercițiului nu alterează topologia generală a undei din imaginea rezultată, ci doar dilată sau comprimă frecvența acesteia în interiorul matricei de $D \times D$. Astfel, sistemul devine imun la fluctuațiile de viteză ale utilizatorului.

Valoarea teoretică a reprezentării P-maps constă în schimbarea de paradigmă pe care o impune procesului de învățare. P-maps transformă recunoașterea spațio-temporală a acțiunilor într-o sarcină clasică de recunoaștere a texturilor. Rețeaua neuronală decizională nu mai este nevoită să ”memoreze” stări trecute (așa cum o fac arhitecturile recurente de tip RNN sau LSTM). O repetiție fizică a unui exercițiu devine, pur și simplu, un tipar geometric vizual repetat în planul P-map-ului.

Din punct de vedere al extragerii de trăsături (feature extraction), identificarea numărului de repetiții se reduce la calcularea densității acestor forme de undă pe lățimea imaginii. Această reformulare matematică a problemei timpului într-o problemă pur spațială aduce o eficiență computațională excepțională, oferind baze teoretice solide pentru utilizarea eficientă a arhitecturilor multi-task. De exemplu, în comparație cu matricea inițială I, matricea finală P', are de $\frac{H\times W}{H+W}$ ori mai puțini parametri, iar presupunând ca $W = H$, numărul de parametri poate fi redus de $\frac{H}{2}$, care în această lucrare [20] este 32. Astfel se obțin k (câte una pentru fiecare articulație) hărți proiectate de dimensiuni 224×224. În Figura 4.5 se poate vizualiza un exemplu de hărți procesate a mai multor articulații în timpul unui exercițiu, partea superioara reprezintând modificarea ordonatelor (valorilor de-a lungul axei X), iar partea

inferioară schimbarea absciselor (valorilor de-a lungul axei *Y*), iar în Figura 4.6 este ilustrată corespondența logică a fiecărei hărți procesate cu articulația detectată.

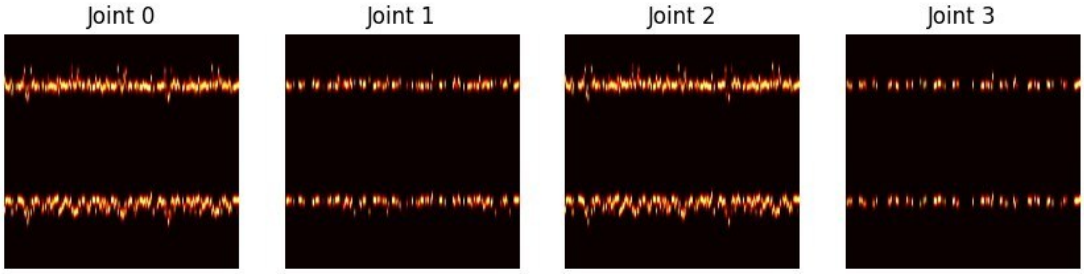


Figura 4.5: Exemplu de hărți proiectate pentru 4 articulații. Partea de sus reprezintă modificarea ordonatelor (valorilor de-a lungul axei *X*), iar partea de jos a fiecărei imagini, schimbarea absciselor (valorilor de-a lungul axei *Y*).

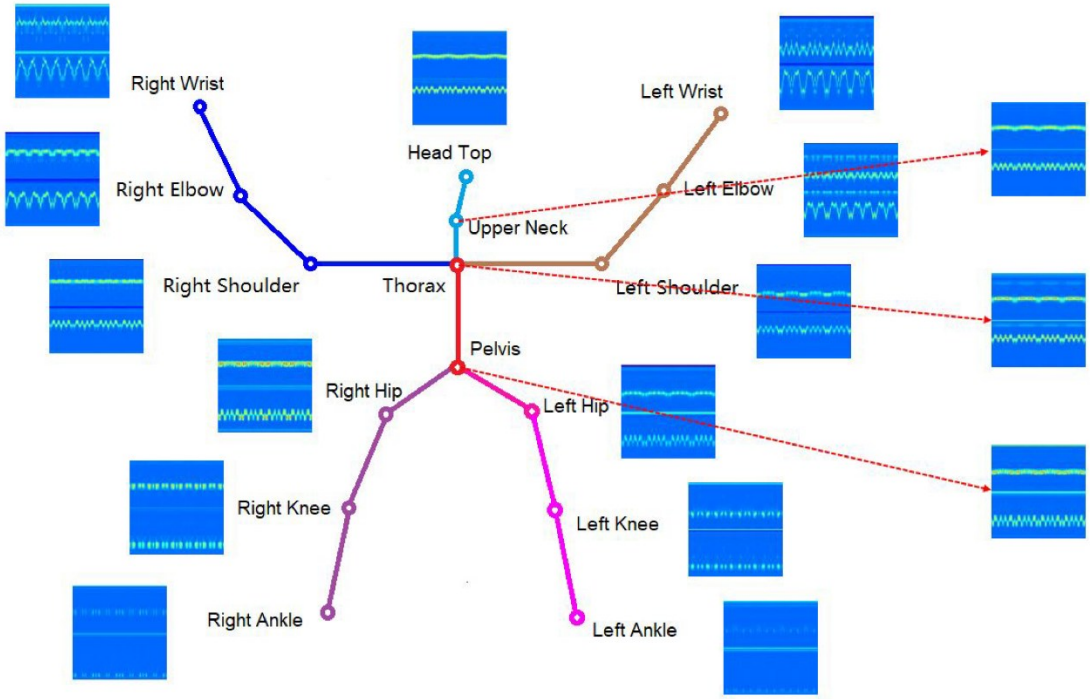


Figura 4.6: Exemplu de hărți proiectate bazat pe un video de 8 repetări a exercițiului „jumping-jack". Fiecare articulație corespunde unei hărți proiectate ce reprezintă mișcarea articulației în tot video-ul. Imagine preluată din [20].

4.5. Teoria învățării Multi-Task

După ce datele video brute au fost abstractizate și comprimate în reprezentări bidimensionale de tip P-map, sistemul necesită un modul decizional capabil să extragă informațiile semantice finale: natura mișcării (clasificarea exercițiului) și cantitatea de efort (numărarea repetițiilor). Din punct de vedere teoretic, abordarea convențională (Single-Task Learning - STL) ar presupune antrenarea a două rețele neuronale independente, fiecare optimizată pentru a rezolva un singur obiectiv. Totuși, această segregare arhitecturală ignoră o realitate matematică și fizică fundamentală: ambele sarcini sunt guvernate de aceleași legi biomecanice și derivă din exact aceeași sursă de date (cinematica scheletului uman).

Pentru a exploata această corelație, arhitectura propusă se fundamentează pe teoria învățării Multi-Task (Multi-Task Learning). MTL este o paradigmă de învățare automată inductivă al cărei scop este îmbunătățirea capacității de generalizare a unui

model prin utilizarea informațiilor de domeniu conținute în semnalele de antrenament ale unor sarcini corelate.

4.5.1. Formularea matematică a problemei Multi-Task

Într-un sistem Single-Task standard, având un set de date $D = \{(x_i, y_i)\}_{i=1}^{N}$, obiectivul este găsirea unui set de parametri θ care minimizează o funcție de pierdere empirică L:

$$\theta^* = \arg\min_{\theta} \sum_{i=1}^{N} L(f(x_i; \theta), y_i)$$

În contrast, teoria MTL definește un spațiu de T sarcini (tasks), notate ca $\{T_1, T_2, \dots, T_T\}$, unde în cazul de față $T = 2$ (T_1 fiind clasificarea, iar T_2 regresia). Pentru fiecare sarcină T_t, sistemul dispune de o funcție de pierdere specifică L_t. Obiectivul matematic global al sistemului devine minimizarea unei funcții de pierdere compuse, definită ca o sumă ponderată a pierderilor individuale, prin ajustarea unui set de parametri partajați θ_{shared} și a unor parametri specifici fiecărei sarcini θ_t:

$$(\theta^*_{shared}, \{\theta^*_t\}) = \arg\min_{\theta_{shared}, \{\theta_t\}} \sum_{t=1}^{T} w_t \sum_{i=1}^{N_t} L_t(f_t(x_{t,i}; \theta_{shared}, \theta_t), y_{t,i})$$

Unde:

- $\boldsymbol{w_t}$ reprezintă greutatea (importanța) sarcinii t în contextul global de optimizare.
- $\boldsymbol{N_t}$ este numărul total de date de intrare pentru sarcina t

Această formulare obligă modelul să găsească o regiune în spațiul parametrilor care satisface simultan constrângerile ambelor probleme matematice.

4.5.2. Topologia rețelei: Partajarea Rigidă a Parametrilor

La nivel arhitectural, MTL poate fi implementat prin mai multe topologii, cele mai cunoscute fiind partajarea ”soft” și partajarea ”hard” a parametrilor. Modelul abstract propus în această lucrare utilizează Partajarea Rigidă a Parametrilor (Hard Parameter Sharing), fiind arhitectura teoretică optimă pentru reducerea riscului de supra-învățare (overfitting).

În această topologie, rețeaua are forma literei ”Y”. Un exemplu de astfel de rețea poate fi obesrvată în Figura 4.7, care reprezintă arhitectura utilizată în lucrarea de referință [20]. Aceasta conține în interiorul straturilor comune cinci blocuri standard de convoluție de dimensinuea $224 \times 224 \times 16$ și utilizează un nucleu de convoluție de dimensiunea de 3×3. Apoi urmează cele două ramuri: prima ramură de recunoaștere a exercițiului este formată dintr-o arhitectură ResNet34, urmată de un strat liniar de dimensiune 128, apoi alt strat liniar de 28 și ultimul strat de ieșire ce conține șapte neuroni, câte unul pentru fiecare clasă de exerciții; a doua ramură începe cu o arhitectură ResNet18, urmată, la fel, de trei straturi liniare de dimensiuni 128, 28 și, respectiv, șapte neuroni, însă mai apare încă un strat pentru regresia unui rezultat numeric care reprezintă numărul de repetări. Straturile inferioare ale rețelei (backbone-ul) conțin parametrii θ_{shared} și funcționează ca un extractor universal de trăsături. Orice tensor de tip P-map de intrare P traversează mai întâi această secțiune comună, generând o hartă de trăsături în spațiul intermediar, definită ca $Z = g(P; \theta_{shared})$.

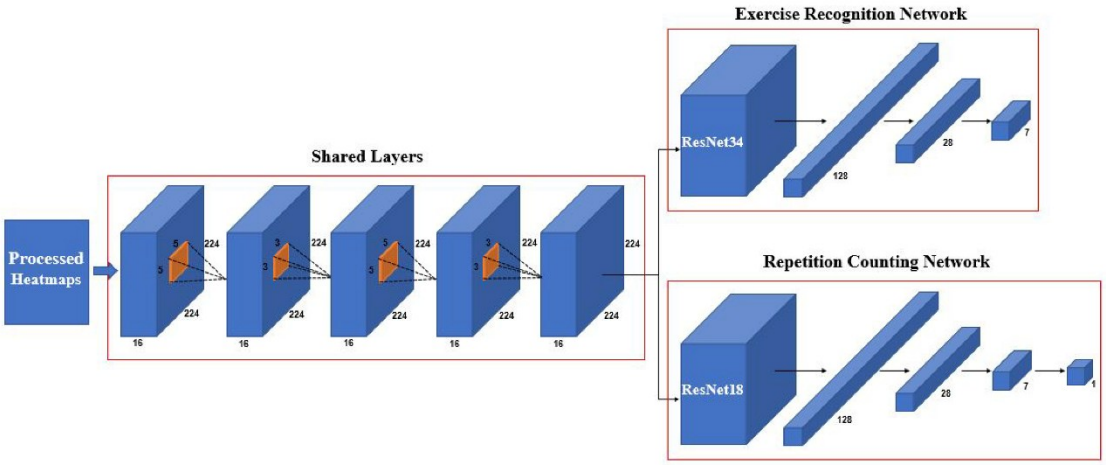


Figura 4.7: Ilustrarea modelului multi-task propus în lucrarea [20], din care a fost preluată această imagine. Conține straturile comune, o ramură pentru recunoașterea exercițiilor și o altă ramură pentru numărarea repetărilor.

Acest spațiu intermediar Z reprezintă o înțelegere abstractă a formelor de undă și a densității mișcării. Din acest punct, fluxul de date se bifurcă. Ramura de clasificare aplică o funcție proprie $h_{cls}(Z;\theta_{cls})$ pentru a obține distribuția de probabilitate a claselor de exerciții, în timp ce ramura de regresie aplică funcția $h_{reg}(Z;\theta_{reg})$ pentru a aproxima o valoare scalară continuă reprezentând numărul de repetiții.

Valoarea teoretică a acestei partajări rezidă în constrângerea spațiului ipotetic (hypothesis space). Într-o rețea Single-Task, modelul poate ”memora” zgomotul specific setului de date, generând un fenomen de supra-învățare. În Hard Parameter Sharing, parametrii θ_{shared} sunt supuși gradientului cumulat din ambele sarcini. O trăsătură falsă (zgomot) care ajută la clasificare, dar încurcă la numărarea repetițiilor, va fi penalizată și ștearsă în timpul propagării înapoi, deoarece nu este universal valabilă. Astfel, Multi-Task Learning acționează intrinsec ca un mecanism matematic de regularizare.

4.5.3. Optimizarea și funcția de pierdere compusă

Una dintre cele mai mari provocări în dezvoltarea unei arhitecturi MTL este optimizarea funcției de pierdere combinate (Joint Loss Function). Deoarece sarcinile sunt fundamental diferite, ele operează în spații metrice diferite, generând gradienți cu magnitudini și direcții potențial conflictuale.

Sarcina de recunoaștere a acțiunii este o problemă de clasificare multi-clasă, a cărei funcție de pierdere L_{cls} este de obicei modelată folosind Categorical Cross-Entropy:

$$L_{cls} = -\sum_{c=1}^{C} y_c \log(\hat{y}_c)$$

Unde:

- $\boldsymbol{y_c}$ este o etichetă binară (one-hot encoding) pentru clasa corectă c
- $\boldsymbol{\hat{y}_c}$ este probabilitatea prezisă de model (obținută printr-o funcție Softmax aplicată ieșirii ramurii de clasificare).

Pe de altă parte, sarcina de numărare este o problemă de regresie, a cărei funcție de pierdere (L_{reg}) este modelată prin Eroarea Medie Pătratică (Mean Square Error -

MSE) sau Eroarea Medie Absolută (Mean Absolut Error - MAE). Presupunând utilizarea MSE:

$$L_{reg} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Unde:

- $\boldsymbol{y_i}$ este numărul real de repetiții
- $\boldsymbol{\hat{y}_i}$ este valoarea scalară estimată de rețea.

Pierderea totală a sistemului devine:

$$L_{total} = w_{cls}L_{cls} + w_{reg}L_{reg}$$

Dacă nu există o strategie de echilibrare teoretică a ponderilor w_{cls} și w_{reg}, modelul se expune fenomenului de Transfer Negativ (Negative Transfer) sau Dominanță a Gradientului. De exemplu, gradientul produs de eroarea de regresie ar putea avea o magnitudine cu un ordin de mărime mai mare decât gradientul entropiei încrucișate. În acest scenariu, vectorul de actualizare a ponderilor θ_{shared} ar fi ”capturat” complet de cerințele sarcinii de regresie, forțând modelul să ignore obiectivul de clasificare. În teorie, optimizarea acestei funcții compuse implică ajustarea dinamică a hiperparametrilor w sau utilizarea unor algoritmi de atenuare a gradientului conflictual, pentru a garanta că ambele suprafețe de decizie converg simultan către un minim global optim.

4.5.4. Mecanisme abstracte ale generalizării

Succesul teoretic al MTL aplicat pe analiza exercițiilor fizice nu este o coincidență, ci se bazează pe două mecanisme cognitive specifice rețelelor profunde:

1. Tragerea cu urechea (Eavesdropping): Unele trăsături abstracte sunt extrem de ușor de învățat pentru o anumită sarcină, dar dificile pentru alta. De exemplu, recunoașterea faptului că un subiect coboară și urcă pe axa Y ajută imens la numărarea repetițiilor (regresie). Ramura de clasificare (care trebuie să deosebească o genuflexiune de o fandare) poate„”trage cu urechea” la această reprezentare oscilantă creată de ramura de regresie în straturile partajate, folosind-o pentru a-și rafina propria predicție semantică.
2. Bias-ul reprezentării (Representation Bias): Din infinitatea de modele matematice pe care rețeaua le-ar putea învăța pentru a rezolva clasificarea, MTL forțează modelul să prefere acele reprezentări care pot explica, simultan, și numărul de cicluri din semnal. Aceasta se traduce printr-un spațiu intern de înaltă calitate, unde trăsăturile geometrice din P-maps sunt decodate pur pe baza frecvenței și formei de undă a mișcării.

În concluzie, abordarea Multi-Task nu este doar o soluție de inginerie pentru a economisi memorie, ci o paradigmă teoretică avansată. Prin impunerea unor constrângeri încrucișate (cross-task constraints), sistemul atinge un nivel de înțelegere structurală a cinematicii umane pe care rețelele izolate matematic nu îl pot egala, validând astfel logic arhitectura aleasă pentru implementarea sistemului.

4.6. Modele abstracte pentru clasificare și regresie

Etapa finală a procesării informației în cadrul arhitecturii propuse este reprezentată de cele două ramuri decizionale care pornesc din trunchiul comun (backbone). Din punct de vedere teoretic, aceste ramuri trebuie să posede capacitatea

de a extrage trăsături de înaltă complexitate din matricele P-map, menținând în același timp un proces de optimizare stabil. În cadrul acestui proiect, s-a optat pentru utilizarea modelelor bazate pe învățarea reziduală (Residual Learning), reprezentate prin arhitecturile ResNet-34 pentru clasificare și ResNet-18 pentru regresie. Această secțiune analizează fundamentele matematice ale acestor modele și rațiunea teoretică a utilizării lor.

4.6.1. Problema degradării și dispariția gradientului

În teoria rețelelor neuronale profunde, o presupunere fundamentală a fost aceea că prin creșterea numărului de straturi (L), rețeaua poate aproxima funcții din ce în ce mai complexe, îmbunătățind astfel performanța. Totuși, cercetările au demonstrat existența unui paradox teoretic cunoscut sub numele de „problema degradării". Pe măsură ce adâncimea rețelei crește, acuratețea tinde să se plafoneze și apoi să scadă rapid. Această degradare nu este cauzată de supra-învățare (overfitting), ci de dificultatea matematică a propagării semnalului de eroare prin numeroase straturi de transformări neliniare.

Principalul obstacol teoretic este „dispariția gradientului" (Vanishing Gradient). În timpul procesului de propagare înapoi, gradientul funcției de pierdere este multiplicat succesiv cu derivatele funcțiilor de activare ale fiecărui strat. Dacă aceste derivate au valori mici (subunitare), produsul lor tinde spre zero exponențial, ceea ce face ca straturile inițiale ale rețelei să nu primească nicio actualizare semnificativă, blocând procesul de învățare.

4.6.2. Principiul Învățării Reziduale și conexiunile Skip

Soluția propusă de autorii lucrării [21] (conceptul ResNet) pentru a depăși problema degradării constă în redefinirea obiectivului de învățare al fiecărui strat. În loc să forțeze un set de straturi să aproximeze direct o funcție dorită $H(x)$, arhitectura reziduală introduce o transformare denumită „mapping rezidual", notată cu $F(x) \coloneqq H(x) - x$. Astfel, funcția originală devine $H(x) = F(x) + x$.

Din punct de vedere matematic, această schimbare de paradigmă este realizată prin introducerea conexiunilor de tip „shortcut" sau „skip connections". Acestea permit informației de la intrarea unui bloc (x) să sară peste unul sau mai multe straturi neliniare, fiind adunată direct la ieșirea acestora. Acest bloc fundamental din care este construită întreaga arhitectură este reprezentat în Figura 4.8.

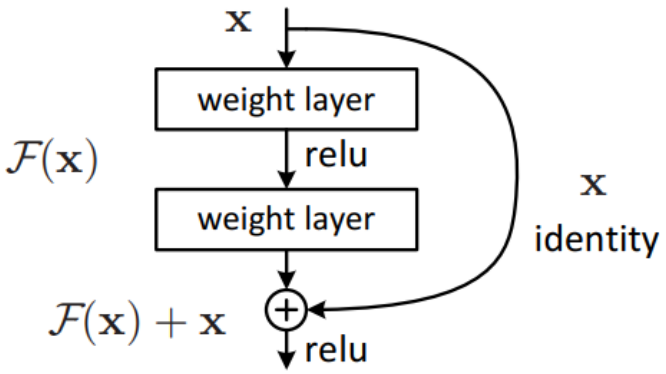


Figura 4.8: Blocul de construcție a arhitecturii cu învățare reziduală. Figură preluată din [21].

Această abordare are avantaje teoretice fundamentale:

1. **Identitatea ca soluție implicită:** Dacă un strat suplimentar este redundant, rețeaua poate învăța foarte ușor să seteze $F(x) = 0$, transformând blocul într-o funcție identitate ($H(x) = x$). Acest lucru garantează teoretic că o rețea mai adâncă nu va avea niciodată o eroare mai mare decât versiunea sa mai puțin adâncă.
2. **Fluxul direct al gradientului:** În timpul propagării înapoi, gradientul poate circula direct prin conexiunea skip, fără a fi diminuat de straturile de convoluție intermediare. Acest lucru permite antrenarea unor arhitecturi foarte adânci fără riscul dispariției semnalului de învățare.

4.6.3. Analiza comparativă: ResNet-34 și ResNet-18

Alegerea unor adâncimi diferite pentru cele două ramuri ale arhitecturii multi-task este fundamentată pe diferența de complexitate teoretică a celor două sarcini.

Ramura de Clasificare (ResNet-34):

Sarcina de a identifica tipul de exercițiu fizic (de exemplu, a distinge între o fandare și o genuflexiune pe baza texturii P-map) necesită un nivel de abstractizare ridicat. Clasificarea acțiunilor umane depinde de înțelegerea relațiilor spațiale globale și a contextului mișcării. Din acest motiv, s-a optat pentru ResNet-34, care utilizează blocuri reziduale de bază, dar cu o adâncime suficientă pentru a extrage trăsături semantice complexe. Arhitectura ResNet-34 și alte arhitecturi (VGG-19 și o rețea convoluțională cu 34 de straturi clasice) pentru comparație sunt ilustrate în Figura 4.9. Teoretic, cele 34 de straturi permit rețelei să învețe ierarhii de trăsături care pornesc de la simple variații de densitate în P-map până la forme de undă specifice fiecărei clase de exerciții.

Ramura de Regresie (ResNet-18):

Numărarea repetițiilor este o sarcină care depinde mai mult de frecvența și densitatea tiparelor repetitive din P-map decât de interpretarea lor semantică profundă. Regresia unui număr scalar necesită o reprezentare mai directă și mai puțin abstractizată a datelor. Utilizarea unei arhitecturi mai simple, precum ResNet-18 (cu doar 18 straturi), oferă câteva avantaje în acest context:

- **Prevenirea supra-învățării pe datele de numărare:** O rețea prea adâncă ar putea încerca să ”memoreze” particularități nesemnificative ale videoclipurilor de antrenament, în loc să învețe conceptul general de ciclu de mișcare.
- **Eficiență computațională:** Regresia este adesea sarcina mai sensibilă la variații fine; o rețea mai compactă tinde să fie mai stabilă în prezicerea valorilor continue.

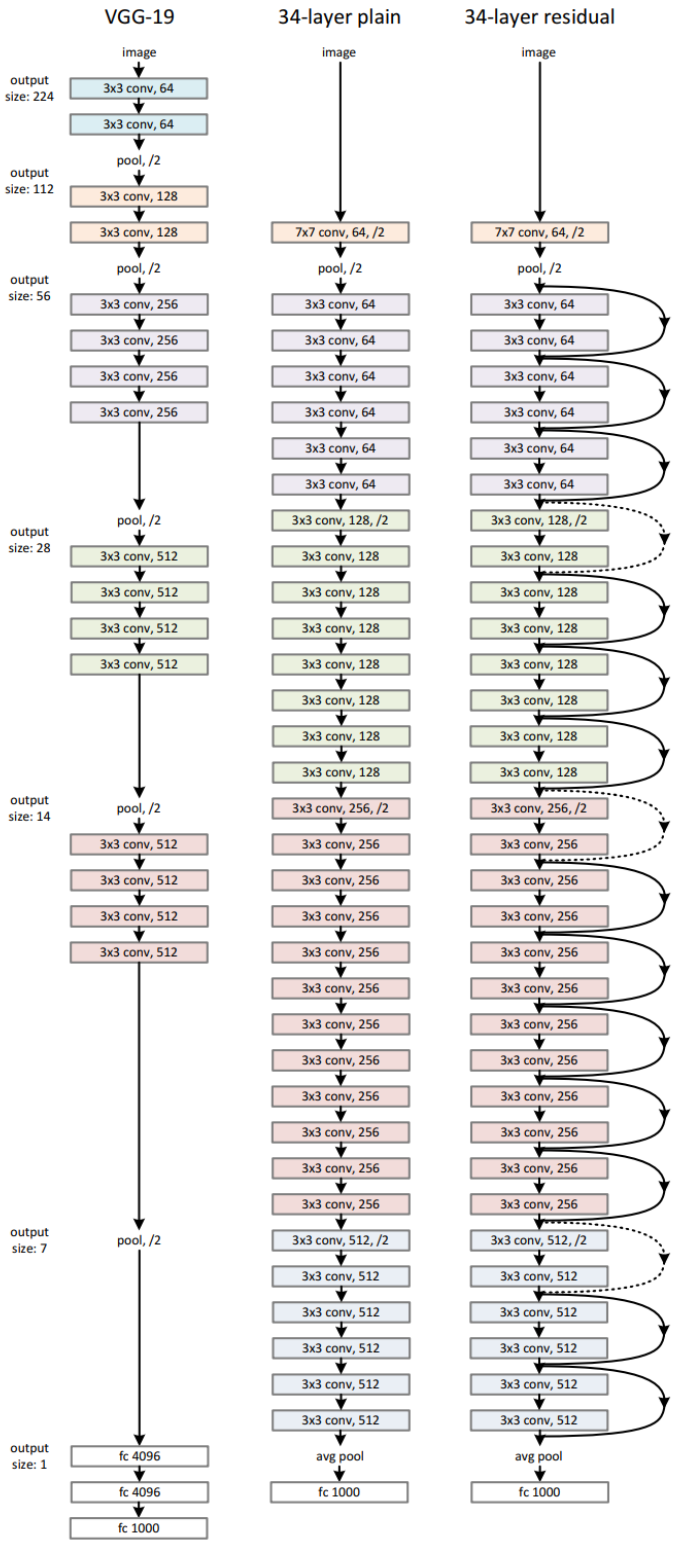


Figura 4.9: Exemplu de arhitecturi a rețelelor pentru ImageNet prezentate în [21]. În această lucrare a fost utilizată arhitectura din dreapta (rețea convoluțională reziduală cu 34 de straturi). Figură preluată din [21].

4.6.4. Diferențierea teoretică a capetelor de decizie

Deși ambele ramuri folosesc blocuri reziduale, obiectivele lor matematice finale impun utilizarea unor funcții de activare și modele de ieșire distincte.

1. **Capătul de Clasificare:** Utilizează o funcție de activare Softmax aplicată peste un vector de dimensiune C (numărul de clase de exerciții). Din punct de vedere teoretic, Softmax transformă ieșirile brute (logits) într-o distribuție de probabilitate, unde suma elementelor este 1. Acest lucru permite interpretarea rezultatului ca fiind probabilitatea de apartenență la o anumită categorie de exercițiu.

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_i}}$$

2. **Capătul de Regresie:** Utilizează o ieșire liniară fără funcție de activare finală (sau, în anumite cazuri, o funcție ReLU dacă se dorește garantarea valorilor pozitive). Obiectivul teoretic este aproximarea unei valori scalare pe axa numerelor reale $\mathbb{R}$. Spre deosebire de clasificare, unde eroarea este măsurată prin distanța între probabilități, în regresie eroarea măsoară distanța directă dintre valoarea prezisă și numărul real de repetiții.

În concluzie, fundamentarea teoretică a modelelor reziduale oferă sistemului capacitatea necesară pentru a învăța trăsături discriminative dintr-o reprezentare comprimată (P-map). Prin utilizarea conexiunilor skip, arhitectura multi-task reușește să mențină un flux informațional eficient, permițând celor două ramuri de ResNet să colaboreze și să se specializeze simultan, maximizând astfel performanța globală a sistemului de detecție și numărare.

Capitolul 5. Proiectare de detaliu și implementare

Scopul acestui capitol este de a translata conceptele teoretice și matematice detaliate anterior într-o arhitectură software concretă și modulară. În secțiunile următoare, vor fi documentate tehnologiile utilizate și structura internă a aplicației dezvoltate, descriind modul în care componentele interacționează pentru a asigura un flux de procesare eficient, de la încărcarea fișierului video până la returnarea rezultatelor analitice. Prezentarea se concentrează pe designul orientat pe obiecte (OOP), schemele logice, diagramele de clase și metodele e care stau la baza funcționării sistemului, oferind astfel o bază solidă pentru viitoarele inițiative de mentenanță sau extindere a platformei.

5.1. Tehnologii și biblioteci utilizate

Dezvoltarea aplicației s-a bazat pe un ecosistem de instrumente software open-source, fundamentat pe limbajul de programare Python. Selecția tehnologiilor a fost realizată având în vedere compatibilitatea, eficiența în procesarea matricelor multidimensionale și suportul pentru accelerarea hardware (GPU). În cele ce urmează sunt detaliate principalele tehnologii utilizate în implementarea sistemului.

- **Python (versiunea 3.10):** Python este un limbaj de programare interpretat, de nivel înalt, recunoscut pentru sintaxa sa ușor de înțeles și pentru cel mai vast ecosistem de pachete dedicate domeniului Data Science și inteligenței artificiale.
  Pentru ce este optim: Este standardul actual în industrie pentru dezvoltarea, dezvoltarea prototipilor și punerea în producție a modelelor de învățare automată, oferind o curbură de învățare lină și o interfațare excelentă cu bibliotecile scrise în C/C++ pentru performanță.
  Rolul în proiect: A reprezentat limbajul de bază în care a fost scris întregul cod sursă al proiectului, de la scripturile de preprocesare a datelor (ingineria setului de date RepPenn), până la arhitectura claselor pentru modelul neuronal și lansarea serverului web.
- **PyTorch (torch, torch.nn, torch.optim):** Este un cadru de lucru (framework) pentru învățare profundă (Deep Learning) dezvoltat de Meta. Oferă suport nativ pentru calculul tensorial accelerat pe plăci video și un sistem de diferențiere automată (Autograd).
  Pentru ce este optim: Construirea și optimizarea arhitecturilor complexe de rețele neuronale datorită grafurilor computaționale dinamice.
  Rolul în proiect: A reprezentat nucleul matematic al modulului AI. Submodulele torch.nn și torch.nn.functional au fost folosite pentru definirea straturilor convoluționale și a funcțiilor de pierdere, în timp ce torch.optim a asigurat actualizarea ponderilor în timpul antrenamentului. Modulul torch.utils.data (prin clasele Dataset și DataLoader) a fost utilizat pentru gestionarea și încărcarea eficientă a datelor în memorie.
- **Torchvision (torchvision.models, torchvision.transforms):** Este o bibliotecă complementară pentru PyTorch, specializată pe domeniul viziunii artificiale, conținând modele pre-antrenate și funcții de procesare a imaginilor.
  Pentru ce este optim: Implementarea rapidă a arhitecturilor de tip "state-of-the-art" și augmentarea datelor vizuale.

Rolul în proiect: A fost utilizat pentru a importa și instanția arhitecturile de bază ResNet-34 și ResNet-18 din torchvision.models, definind ramurile rețelei multi-task. De asemenea, a fost folosit pentru transformările de bază ale matricelor de intrare în tensori compatibili cu modelele PyTorch.

- **Ultralytics YOLO:** Este biblioteca oficială ce implementează familia de modele YOLOv8, oferind un API simplificat pentru estimarea poziției umane (pose estimation) și detectarea obiectelor.
  Pentru ce este optim: Oferă inferență în timp real (real-time processing) cu o precizie a detectării extrem de ridicată, fiind ideală pentru analize video unde latența este critică.
  Rolul în proiect: A fost utilizat ca modul de extracție a reperelor anatomice. Prin instanțierea modelului pre-antrenat yolov8n-pose.pt, biblioteca a procesat cadrele video brute pentru a returna coordonatele spațiale ale celor 17 repere anatomice umane, acționând ca o alternativă digitală la senzorii fizici externi.
- **OpenCV-Python (cv2):** Interfața Python pentru OpenCV (Open Source Computer Vision Library), cea mai populară bibliotecă de algoritmi clasici de procesare a imaginilor.
  Pentru ce este optim: Citirea, scrierea, filtrarea și transformarea geometrică a imaginilor, operând direct și rapid pe matrice.
  Rolul în proiect: Utilizat extensiv pentru preluarea fluxului video (cv2.VideoCapture), redimensionarea cadrelor și pentru generarea feedback-ului vizual (desenarea textului și a scheletului detectat peste videoclipul final destinat utilizatorului).
- **NumPy:** Pachetul fundamental pentru calcul științific în Python, oferind suport pentru tablouri multidimensionale dense și funcții matematice de nivel înalt.
  Pentru ce este optim: Executarea operațiilor matematice vectorizate și agregări de date cu eficiență înaltă.
  Rolul în proiect: Toată logica matematică a transformării coordonatelor YOLO în hărți de probabilități (heatmaps) și ulterior în hărți proiectate (P-maps) s-a bazat pe NumPy. Colapsarea axelor temporale și spațiale prin operatorul de maxim (np.max) a fost realizată folosind aceste structuri de date.
- **SciPy (scipy.io):** Bibliotecă pentru matematică și știință; submodulul io facilitează interacțiunea cu diferite formate de date non-standard.
  Pentru ce este optim: Citirea formatelor de date proprietare, precum cele salvate din mediul MATLAB.
  Rolul în proiect: A fost folosit în etapa inițială de procesare a setului de date PennAction, utilizând funcția loadmat pentru a extrage adnotările, coordonatele și etichetele stocate inițial în fișiere .mat.
- **Matplotlib (matplotlib.pyplot):** O bibliotecă avansată pentru crearea de vizualizări statice, animate și interactive în Python.
  Pentru ce este optim: Generarea de grafice matematice bidimensionale la o calitate înaltă.
  Rolul în proiect: A fost utilizat pentru monitorizarea vizuală a performanței. A facilitat trasarea curbelor de învățare (funcția de pierdere pe epoci) și vizualizarea formelor de undă generate de hărțile P-map pentru diagnosticarea comportamentului rețelei neuronale.
- **TQDM:** O utilitate Python rapidă și extensibilă pentru crearea barelor de progres.
  Pentru ce este optim: Monitorizarea iterațiilor lungi, calculând dinamic viteza de procesare și timpul estimat de finalizare (ETA).

Rolul în proiect: Integrat în buclele de antrenament (training loops) și în scripturile de extragere a cadrelor, permițând o urmărire în timp real a procesării miilor de fișiere video din setul de date.

- **Gradio:** Gradio este un framework pentru crearea rapidă a interfețelor web pentru modele de Machine Learning.
  Pentru ce este optim: Prezentarea intuitivă a rezultatelor AI către utilizatorul final și procesarea non-liniară a rezultatelor multimedia.
  Rolul în proiect: Gradio a stat la baza construirii aplicației web demonstrative (frontend), permițând utilizatorilor încărcarea videoclipurilor și vizualizarea interactivă a predicțiilor. MoviePy a fost utilizat în tandem pentru conversia, asamblarea și formatarea clipurilor video generate de OpenCV, garantând compatibilitatea redării acestora în mediul web.
- **MoviePy:** MoviePy (moviepy.editor) este un modul de manipulare și editare video.
  Pentru ce este optim: Procesarea non-liniară a rezultatelor multimedia.
  Rolul în proiect: MoviePy a fost utilizat pentru conversia, asamblarea și formatarea clipurilor video generate de OpenCV, garantând compatibilitatea redării acestora în mediul web.
- **Biblioteci Standard Python (os, pathlib, glob, time, random):** Module native incluse în instalarea de bază a limbajului Python.
  Rolul în proiect: Aceste instrumente au asigurat arhitectura de sistem a aplicației. Modulele os, pathlib și glob au gestionat parcurgerea și indexarea directoarelor masive de date. Modulul random a fost folosit pentru amestecarea datelor (shuffling) necesară pentru generalizarea modelului, în timp ce time a fost utilizat pentru măsurarea latenței de inferență a sistemului.

5.2. Schema generală a aplicației

Sistemul software este proiectat urmând un șablon arhitectural de tip „conductă” (Pipeline Architecture), împreună cu o decuplare strictă între interfața utilizator și logica de procesare (Backend/Motorul AI). Această decizie de proiectare a fost luată pentru a asigura o testabilitate ridicată și pentru a permite, pe viitor, migrarea ușoară a nucleului de calcul pe o infrastructură cloud (sub formă de API), menținând interfața pe dispozitivul clientului.

La nivel înalt, sistemul este compus din patru subsisteme (module) principale, care colaborează secvențial:

1. **Modulul de Interfață și Control (UI & Inference Controller):** Reprezintă punctul de interacțiune cu utilizatorul. Acest modul preia datele de intrare (videoclipul brut) și randează rezultatele finale (videoclipul adnotat, clasa exercițiului și contorul de repetiții).
2. **Modulul de Viziune Artificială (Pose Extractor):** Este componenta responsabilă cu decodarea vizuală. Preia cadrele individuale furnizate de controller și aplică modelul de detecție (YOLOv8-pose) pentru a genera o listă structurată de coordonate geometrice, corespunzătoare scheletului uman.
3. **Modulul de Procesare Spațio-Temporală (Feature Transformer):** Funcționează ca un intermediar de formatare a datelor. Acesta încapsulează logica matematică a generării hărților de probabilități (heatmaps) și realizează proiecțiile ortogonale pentru a condensa informația din videoclip într-un singur tensor bidimensional (P-map).

4. **Modulul de Învățare Profundă (Multi-Task AI Core):** Reprezintă creierul analitic al sistemului. Este implementat pe baza arhitecturii PyTorch, preluând matricele P-map și rulându-le prin rețeaua cu partajare a parametrilor (ResNet-34 și ResNet-18) pentru a obține predicțiile finale.

La nivel mai jos, structura în detaliu a sistemului este prezentată în Figura 5.1, în care se prezintă și maparea celor patru module descrise anterior.

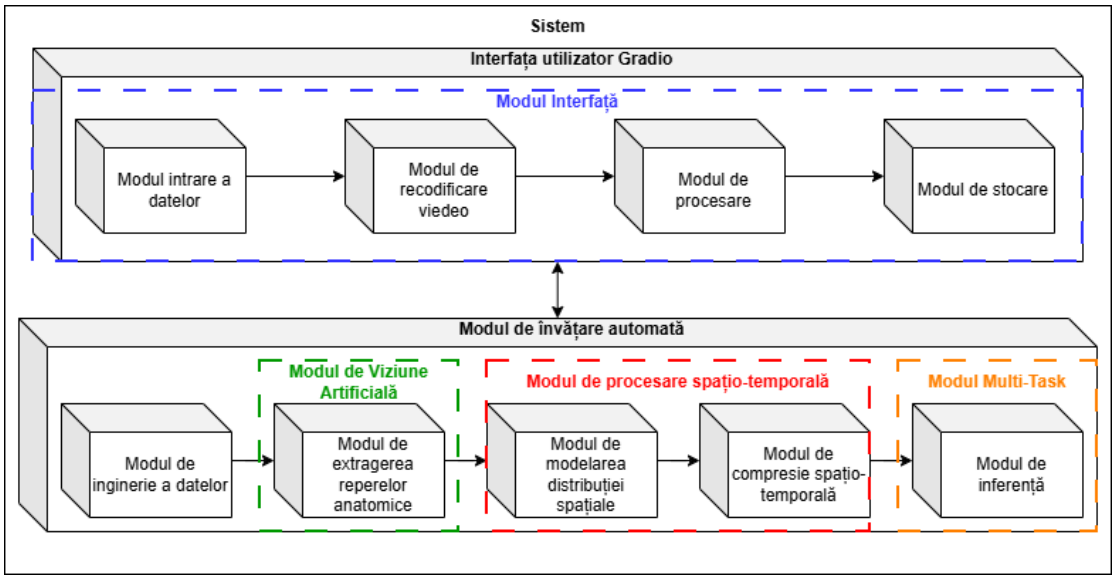


Figura 5.1: Schema bloc a sistemului și maparea principalelor module logice.

Fluxul informațional (Data Flow) al sistemului este unidirecțional. O sesiune de execuție începe prin instanțierea claselor din interfață, care deschide fluxul video și extrage metadatele (FPS, lățime, înălțime). Fiecare cadru valid este trimis către Pose Extractor. După ce întregul videoclip a fost parcurs, memoria sistemului conține o matrice completă a cinematicii umane. Apoi modulul de procesare spațio-temprorală poate converti aceste date brute. În final, tensorul normalizat este pasat metodei de predicție a modulului AI Core, care returnează două primitive de date: un șir de caractere (numele clasei) și o valoare numerică (numărul de repetiții).

5.3. Sinteza setului de date RepPenn

O provocare majoră în dezvoltarea sistemelor de numărare a repetițiilor bazate pe viziune artificială este lipsa seturilor de date adnotate care să conțină videoclipuri cu cicluri multiple de exerciții. Setul de date public PennAction[2] conține înregistrări calitative ale unor activități fizice, însă majoritatea videoclipurilor surprind o singură execuție (un singur ciclu) a unui exercițiu. Pentru a antrena o rețea neuronală capabilă să recunoască și să numere acțiuni repetitive continue, a fost necesară ingineria unui nou set de date derivat, denumit RepPenn, printr-un proces de sinteză și augmentare temporală.

5.3.1. Normalizarea spațială și extragerea cadrelor

Procesul de sinteză debutează cu extragerea și procesarea cadrelor individuale din secvențele originale PennAction. Deoarece videoclipurile sursă pot avea rezoluții și rapoarte de aspect variate, primul pas constă în citirea fiecărei imagini de pe disc și

[2] Setul de date PennAction - site

redimensionarea ei la o rezoluție standardizată de 640×480 pixeli utilizând biblioteca OpenCV. Această normalizare spațială garantează că tensorii generați ulterior vor avea dimensiuni uniforme, prevenind erorile de alocare a memoriei în timpul trecerii prin modelul de detecție YOLO.

5.3.2. Crearea artificială a ciclicității

Pentru a simula o mișcare repetitivă fluidă pornind de la o singură execuție, sistemul aplică o tehnică de compunere secvențială. Simpla concatenare a videoclipului original de mai multe ori ar produce ”salturi” (jumps) nenaturale la punctul de îmbinare (de exemplu, trecerea instantanee de la poziția de final a unei flotări direct la poziția de start).

Pentru a rezolva această discontinuitate fizică, algoritmul prezentat în [20] extrage secvența originală de cadre, notată ca P (Forward Pass), și generează o secvență inversată, notată ca R (Backward Pass sau Reverse). Un ciclu complet de mișcare continuă este obținut prin unirea celor două secvențe ($P + R$), precum este ilustrat și în Figura 5.2. Această secvență compusă este apoi multiplicată de un număr specific de ori (în cazul implementării curente, algoritmul este configurat să genereze variante cu patru, șase, opt și zece cicluri), simulând astfel un antrenament neîntrerupt al utilizatorului.

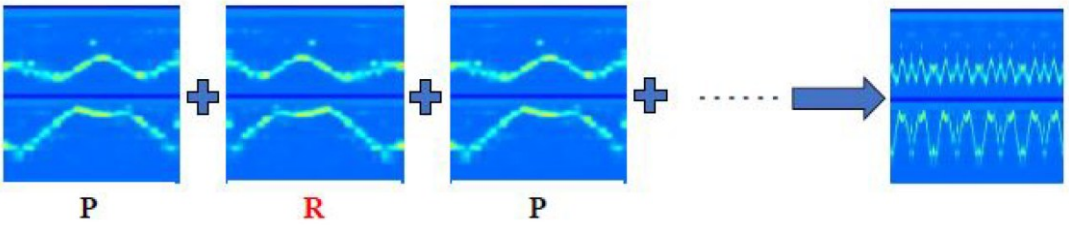


Figura 5.2: Crearea videoclipurilor multiciclu. Imagine preluată din [20].

5.3.3. Augmentarea temporală

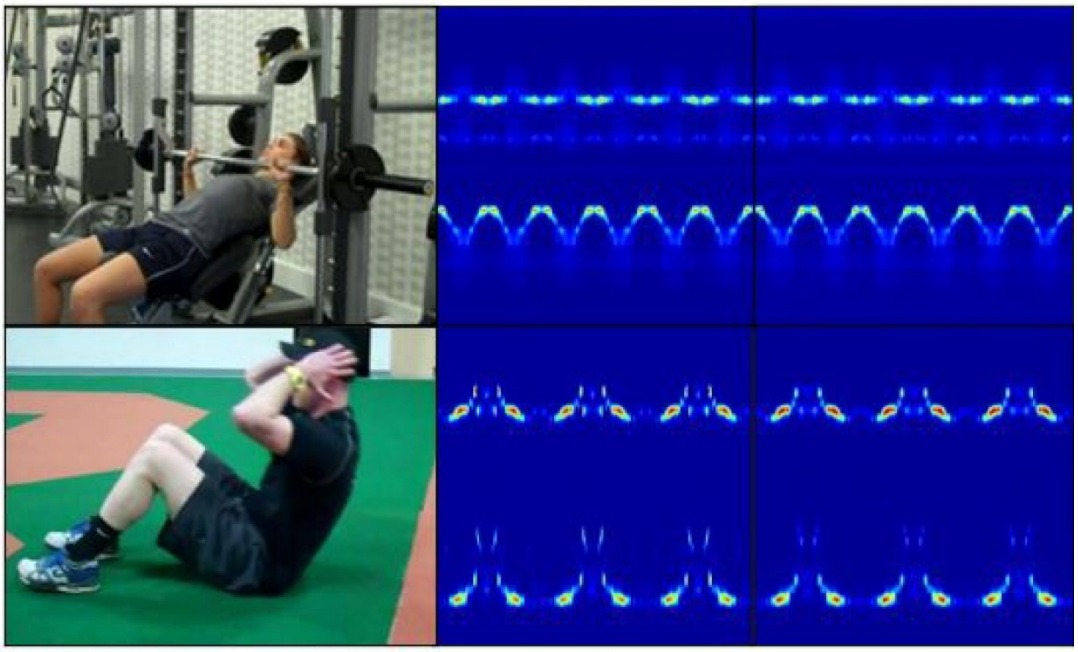
În scenarii reale, utilizatorii execută exercițiile la viteze diferite (tempo diferit). Pentru a preveni supra-învățarea modelului pe o singură frecvență a mișcării, algoritmul aplică o tehnică de augmentare temporală prin eșantionare (subsampling). După generarea cadrelor pentru ciclurile complete, sistemul creează multiple versiuni ale aceluiași videoclip aplicând un pas de eșantionare (stride).

O viteză de ”1x” păstrează toate cadrele, generând o mișcare fluidă la viteza originală. O viteză de ”2x” păstrează doar un cadru din două, înjumătățind durata totală a videoclipului și dublând vizual viteza de execuție a exercițiului. Această variație forțează rețeaua neuronală să devină invariantă la viteza de execuție și să se concentreze strict pe topologia mișcării. La fel, datorită limitării dimensiunii hărților de probabilitate nu se pierde informații spațiale, ceea ce se poate observa în Figura 5.3, în care este prezentată mișcarea încheieturii de la mâna dreaptă în timpul a două exerciții diferite, unde se vede intermitența mișcării, dar în care nu se observă și diferența în viteze, deși coloana **a** prezintă viteza originală, iar coloana **b** prezintă hărțile de probabilitate obținute din procesarea secvenței video în care viteza este de două ori mai mare.

5.3.4. Codificarea și stocarea

Cadrele finale sintetizate sunt asamblate înapoi în fișiere video folosind utilitarul VideoWriter din OpenCV, utilizând codecul mp4v la o rată de 30 de cadre pe secundă (FPS). Fișierele sunt salvate respectând o convenție de denumire strictă (de

exemplu, [ID_Secvență]_c4_s2.mp4), unde ID-ul asigură trasabilitatea la videoclipul original, litera 'c' indică numărul de cicluri, iar litera 's' indică viteza eșantionată. Această procedură de inginerie a datelor a permis creșterea exponențială a volumului de date de antrenament, oferind rețelei o diversitate cinematică utilă pentru generalizare.



(a) (b)

Figura 5.3: Exemplificare hărților de probabilitate a viedoclipurilor cu un diferit număr de repetări și viteze. Mișcarea încheieturii de la mâna dreaptă în timpul a două exerciții: “bech press” și „sit up”. **(a)** Viteza originală. **(b)** Viteza dublă. Imagine preluată din [20].

5.4. Preprocesare și gestionare a datelor

Pentru faza de antrenament și validare a modelului Multi-Task, gestionarea eficientă a memoriei și încărcarea rapidă a datelor reprezintă cerințe importante. Citirea directă a fișierelor video de pe mediul de stocare în timpul fiecărei epoci de antrenare ar crea un blocaj (bottleneck) la nivelul operațiilor de intrare/ieșire (I/O). Prin urmare, sistemul include un modul dedicat ingineriei datelor, care abstractizează setul de date RepPenn sub forma unor obiecte programatice optimizate.

Baza acestui modul este reprezentată de clasa PrecomputedRepPennDataset, care moștenește funcționalitățile clasei de bază Dataset din cadrul bibliotecii PyTorch (torch.utils.data.Dataset). Această moștenire arhitecturală permite integrarea cu mecanismele de încărcare paralelizată (DataLoader).

5.4.1. Arhitectura clasei de gestionare a datelor

Clasa PrecomputedRepPennDataset este proiectată să gestioneze un director pre-procesat în care datele video au fost deja transformate în fișiere de matrice comprimate (de tip .npz). Prin această decizie de proiectare, complexitatea spațială scade semnificativ, iar viteza de acces crește.

Metodele principale care definesc interfața și comportamentul acestei clase sunt:

- **Metoda constructor __init__(self, data_dir):**

Rolul acestei metode este de a inițializa starea obiectului. Primește ca argument calea către directorul care conține datele de antrenament sau testare. În interiorul ei, se realizează maparea sistemului de fișiere: se iterează prin toate directoarele corespunzătoare claselor de exerciții (ex. ”pushup”, ”squat”) și se construiește o listă internă de trasee absolute către fiecare fișier .npz. Tot în această metodă se definește dicționarul de mapare a etichetelor (label mapping), asociind fiecărui șir de caractere (numele exercițiului) un index întreg (de la 0 la 6), necesar funcțiilor de pierdere din rețeaua neuronală.

- **Metoda __len__(self):**

O metodă obligatorie care returnează dimensiunea totală a setului de date (numărul total de fișiere înregistrate în lista internă). Aceasta permite orchestratorului de antrenament să determine numărul exact de iterații necesare pentru a completa o epocă (epoch).

- **Metoda __getitem__(self, idx):**

Aceasta este funcția centrală de extragere a datelor (Data Fetching). Când instanța de tip DataLoader solicită un element de la un anumit index, metoda efectuează următoarele operații secvențiale:

1. Localizează calea fișierului .npz pe baza indexului idx.
2. Încarcă în memoria RAM matricea trăsăturilor, reprezentând harta P-map a videoclipului, folosind biblioteca NumPy.
3. Extrage din același fișier valoarea regresiei (numărul de repetiții pre-etichetat) și numele clasei.
4. Efectuează transformarea tipurilor de date: matricea NumPy este convertită într-un obiect de tip torch.Tensor. O operație importantă efectuată aici este normalizarea datelor: valorile pixelilor, inițial în intervalul $[0, 255]$, sunt divizate la 255 pentru a aduce tensorul în intervalul $[0, 1]$, facilitând convergența rapidă a gradientului. Axa dimensională este rearanjată (folosind metoda permute sau transpose) din formatul (H, W, C) în formatul așteptat de PyTorch (C, H, W).
5. Returnează un tuplu standardizat format din: (tensor_intrare, eticheta_clasei, valoarea_regresiei).

Prin încapsularea acestor operații în cadrul unei clase dedicate, logica de acces la date rămâne decuplată de logica de antrenament a modelului neuronal, respectând principiul responsabilității unice (Single Responsibility Principle) din ingineria software.

5.4.2. Diagrama de clasă

Diagrama este prezentată în Figura 5.4.

Moștenirea: Clasa PrecomputedRepPennDataset moștenește comportamentul de bază al clasei Dataset din PyTorch, fiind obligată să suprascrie (override) metodele __getitem__ și __len__.

Agregarea: Clasa DataLoader (oferită nativ de PyTorch) asamblează loturi de date (batches). Aceasta agregă instanța de PrecomputedRepPennDataset, preluând tensorii generați de metoda __getitem__ și trimițându-i pe placa video (GPU) pentru antrenament, respectând parametrii configurați precum batch_size și shuffle (amestecarea datelor).

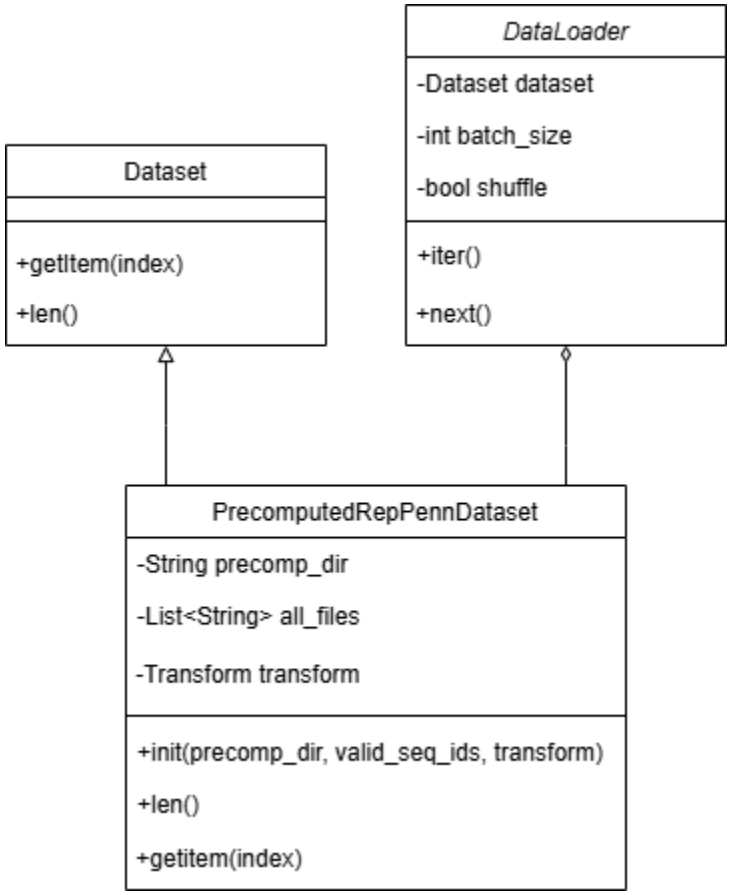


Figura 5.4: Diagrama de clasă pentru PrecomputedRepPennDataset.

5.5. Modulul de extracție a reperelor anatomice

Primul pas în conducta de procesare (pipeline) a aplicației constă în izolarea cinematicii umane de zgomotul vizual din videoclipul brut. Această sarcină este implementată prin intermediul modulului de extracție a reperelor anatomice, care se bazează pe modelul pre-antrenat YOLOv8-pose.

La nivel de implementare, obiectul central al acestui modul este instanțiat folosind biblioteca Ultralytics (**yolo_pose = YOLO("yolov8n-pose.pt")**). S-a optat pentru varianta "nano" (sufixul n) a modelului, care oferă un echilibru optim între viteza de inferență și precizie"", fiind ideală pentru procesarea unor volume mari de date fără a bloca memoria plăcii video (VRAM).

Funcția responsabilă de procesarea videoclipurilor este **extract_kps_yolo**. Pentru a eficientiza timpul de execuție, cadrele video nu sunt procesate individual (unul câte unul), ci sunt grupate în loturi (batches), folosind parametrul batch_size=12. Modelul YOLO primește acest lot de cadre și returnează un obiect cu rezultate pentru fiecare imagine.

O componentă cheie a acestei funcții este gestionarea cazurilor excepționale (ex. cadre în care persoana iese din câmpul vizual sau este obturată complet). Algoritmul verifică dacă au fost detectate puncte cheie validând condiția r.keypoints is not None și asigurându-se că dimensiunea datelor este mai mare de zero. În caz afirmativ, coordonatele și valorile de vizibilitate sunt transferate de pe unitatea de procesare grafică (GPU) în memoria principală (CPU) sub forma unui tablou (array) NumPy. În cazul în care nu se detectează nicio persoană, algoritmul injectează intenționat un tablou de zerouri de dimensiune $(17, 3)$. Această completare (padding)

garantează că dimensiunea temporală a videoclipului rămâne intactă și sincronizată, prevenind eșecul etapelor ulterioare de transformare. La final, toate tablourile sunt concatenate (stivuite) pe prima axă (np.stack), returnând o structură tridimensională de forma $(Nr_{cadre}, 17,\ 3)$.

5.6. Modulul de procesare spațio-temporală

Coordonatele brute obținute din modulul anterior sunt valori discrete, dependente de rezoluția videoclipului original. Modulul de procesare spațio-temporală are responsabilitatea de a standardiza aceste date și de a le condensa în matricele bidimensionale finale (P-maps), pe care rețeaua neuronală le va utiliza pentru decizie. Acest proces implică trei etape de implementare interconectate: redimensionarea, generarea hărților de probabilitate și proiecția spațio-temporală.

5.6.1. Normalizarea spațială a coordonatelor

Deoarece videoclipurile pot avea dimensiuni variate, prima operație, implementată în funcția **scale_keypoints_to_size**, realizează scalarea proporțională a coordonatelor carteziene (x, y). Funcția calculează rapoartele de scalare (scale_x și scale_y, de exemplu **scale_x = Wt / float(Wf)**) între rezoluția originală (**Wf/Hf**) a cadrului și rezoluția țintă (**Wt/Ht**) a hărții de probabilitate (configurată implicit la 64×64 pixeli) și ajustează reperele YOLO în consecință.

5.6.2. Generarea hărților de probabilitate

Funcția **keypoints_to_heatmaps** transformă punctele scalate în distribuții de probabilitate. Se inițializează un tensor nul de dimensiuni (F, K, H, W), unde F reprezintă numărul de cadre, iar K numărul de articulații. Algoritmul creează o grilă de coordonate (np.meshgrid) pentru a evalua rapid funcția de distribuție pe întreaga suprafață a imaginii de 64×64.

Iterând prin fiecare cadru și fiecare articulație, algoritmul aplică un filtru de calitate: dacă vizibilitatea estimată de YOLO este foarte scăzută (vis < 0.1) sau coordonatele sunt nevalide (x<=0 sau y<=0), punctul este ignorat. Pentru punctele valide, se calculează distribuția Gaussiană 2D centrată în coordonatele (cx, cy) cu un parametru de dispersie **sigma = 1.5**. Această implementare vectorizată, folosind operațiile native NumPy (**np.exp(...)**), este mult mai rapidă decât setarea manuală a pixelilor printr-o buclă iterativă, evitând astfel un blocaj de performanță.

5.6.3. Proiecția în hărți P-map

Ultima și cea mai importantă etapă de preprocesare este implementată în funcția **heatmaps_to_processed_feature**. Aici se realizează reducerea tensorului 4D la o formă compactă adecvată modelului ResNet.

1. **Transpunerea axelor:** Tensorul este reordonat (**transpose**) din formatul (F, K, H, W) în (H, W, F, K), pregătind axele pentru operațiile de agregare.
2. **Colapsarea spațială:** Deși teoretic se poate folosi operatorul de maximizare, la nivel de implementare s-a optat pentru calculul mediei (np.mean). Matricea A rezultă din calculul mediei pe axa H, având dimensiunea (W, F, K), iar matricea B rezultă din calculul mediei pe axa W, având dimensiunea (H, F, K). Aceste două matrice reprezintă istoria temporală a mișcărilor pe orizontală și verticală.
3. **Concatenarea:** Matricele A și B sunt unite pe axa 0, rezultând o singură matrice P de dimensiune $(H + W, F, K)$.

4. **Normalizarea Min-Max:** Pentru a stabiliza gradientul rețelei neuronale și a crea o imagine validă, valorile rezultate sunt scalate strict în intervalul $[0, 255]$, folosind ecuația de normalizare **Pn = 255.0 * (P - pmin) / (pmax - pmin)**.

Deoarece numărul de cadre F variază de la un videoclip la altul, matricea finală nu este pătratică. Funcția iterează prin fiecare dintre cele K canale ale tensorului și utilizează **cv2.resize** (cu interpolare liniară) pentru a aduce matricea la dimensiunea fixă de 224×224 pixeli, convertind-o în tipul de date **uint8**. În varianta destinată direct inferenței (**heatmaps_to_Pmap**), tensorul final este convertit într-un format compatibil PyTorch (**torch.Tensor**) și normalizat în intervalul $[0, 1]$ prin divizarea la 255, devenind astfel perfect pregătit pentru trecerea prin straturile modelului Multi-Task.

5.7. Proiectarea modelului Multi-Task

Nucleul decizional al aplicației este încapsulat în modulul de învățare profundă, implementat prin intermediul framework-ului PyTorch. Acest modul definește arhitectura rețelei neuronale capabile să primească tensorii P-map generați anterior și să returneze simultan predicțiile pentru cele două sarcini: clasificarea exercițiului și regresia numărului de repetări.

Din punct de vedere al ingineriei software, modelul este definit printr-o clasă personalizată, diagrama de clasă a căreia este prezentată în Figura 5.5, denumită MultiTaskModel, care moștenește funcționalitățile clasei de bază torch.nn.Module. Această structură orientată pe obiecte permite integrarea nativă cu mecanismele de calcul al gradientului (Autograd) și cu funcțiile de salvare/încărcare a ponderilor (state dictionaries).

5.7.1. Inițializarea arhitecturii

În cadrul metodei constructor, clasa instanțiază cele două ramuri principale ale rețelei folosind modelele pre-antrenate din torchvision.models.

1. **Adaptarea intrării (Input Channels):** Un aspect important rezolvat în constructor este adaptarea primului strat convoluțional. Modelele ResNet standard sunt concepute pentru imagini RGB (3 canale). Deoarece P-map-ul nostru are $K = 17$ canale (corespunzătoare celor 17 repere anatomice), stratul **conv1** al ambelor modele (ResNet-34 și ResNet-18) este înlocuit programatic cu o nouă secțiune de convoluție bidimensională (**nn.Conv2d**) care acceptă 17 canale de intrare, menținând dimensiunea nucleului de 7×7 și pasul (stride) de 2.
2. **Ramura de clasificare (recog_net):** Folosește arhitectura ResNet-34. Stratul final complet conectat (**fc**) este înlocuit cu un nou strat liniar (**nn.Linear**) a cărui dimensiune de ieșire corespunde numărului total de clase de exerciții din setul de date (ex. 7 clase).
3. **Ramura de regresie (count_net):** Folosește arhitectura ResNet-18 pentru a preveni supra-învățarea tiparelor numerice. Similar, stratul său final **fc** este rescris pentru a returna o singură valoare scalară de ieșire (dimensiunea 1).

5.7.2. Fluxul de inferență

Metoda **forward(self, x)** definește traseul pe care tensorul de intrare **x** (având forma [Batch, 17, 224, 224]) îl parcurge prin rețea. Spre deosebire de o rețea secvențială clasică, aici tensorul este procesat în paralel de cele două sub-rețele.

5.7.3. Diagrama de clasă a modulului Multi-Task

Pentru a ilustra arhitectura internă a rețelei neuronale și modul în care aceasta asamblează sub-modelele PyTorch, a fost realizată următoarea diagramă de clase (UML):

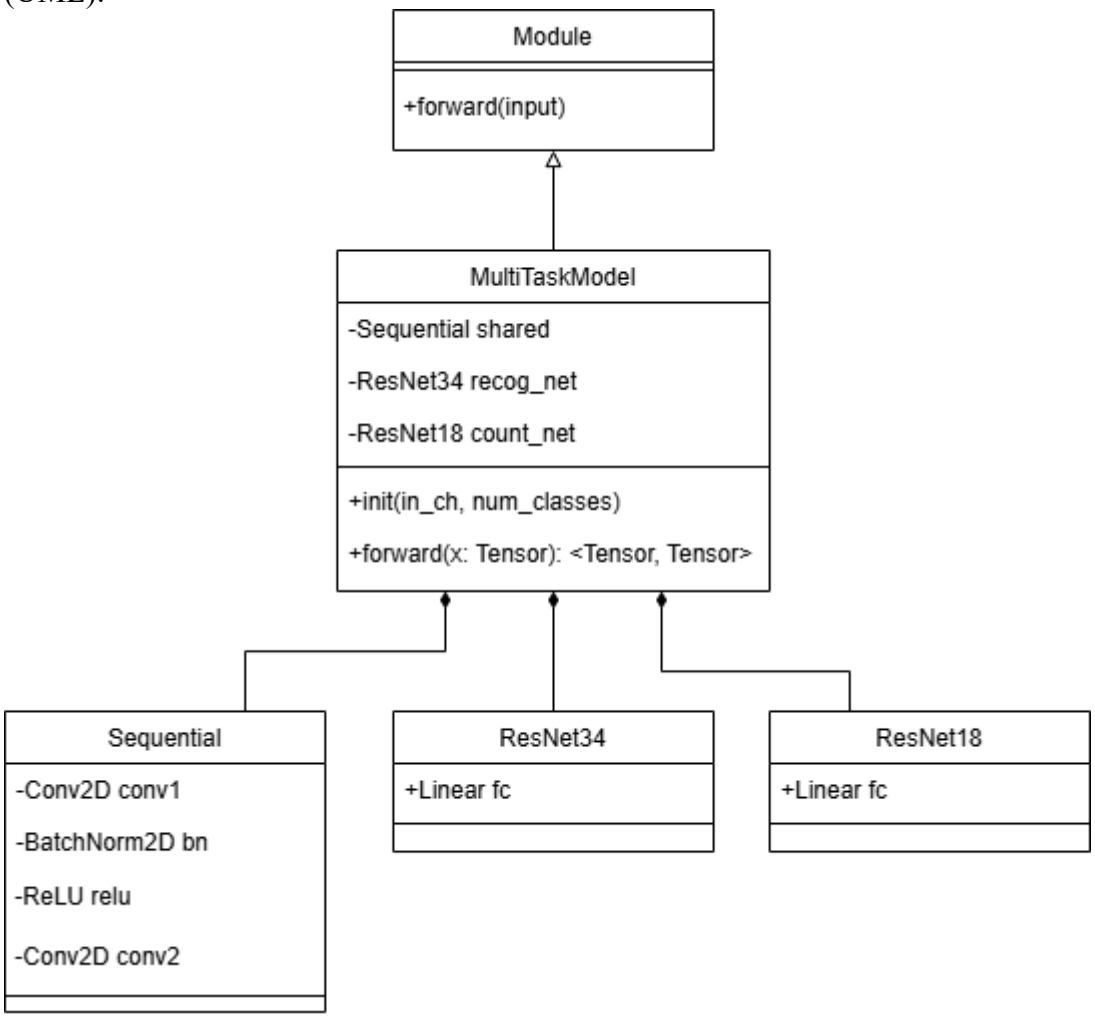


Figura 5.5: Diagrama de clasă pentru MultiTaskModel.

1. **Relația de moștenire:** Clasa MultiTaskModel moștenește clasa de bază Module din ecosistemul torch.nn. Această moștenire este obligatorie în PyTorch pentru a înregistra automat parametrii rețelei și pentru a activa funcționalitățile de propagare înapoi a gradientului (Autograd). Clasa aceasta suprascrie metoda forward a clasei de bază.
2. **Relațiile de compoziție:** MultiTaskModel este o clasă container care asamblează trei blocuri funcționale majore:
    - **Atributul shared:** Este o instanță a clasei Sequential care comprimă numărul de canale de la **in_ch** (de exemplu, 17 canale specifice P-map-urilor) la 3 canale standard. Acest bloc acționează ca un adaptor spațial și extrage primele trăsături comune aplicând succesiv operații de convoluție (Conv2d), normalizare (BatchNorm2d) și activare neliniară (ReLU).
    - **Atributul recog_net:** Reprezintă ramura dedicată clasificării exercițiilor. Utilizează o instanță a clasei ResNet34 (preluată din torchvision.models). Stratul său final complet conectat (**fc**) este

suprascris pentru a mapa trăsăturile către un număr de ieșiri egal cu **num_classes**.

- **Atributul count_net:** Reprezintă ramura dedicată regresiei (numărarea repetițiilor). Utilizează arhitectura mai compactă ResNet18. La fel, capătul său decizional (**fc**) este înlocuit cu un strat liniar care returnează o singură valoare scalară.

3. **Metoda forward:** Definește topologia grafului computațional. Un tensor de intrare **x** trece mai întâi prin blocul **shared**, apoi este pasat în paralel către **recog_net** și **count_net**. Metoda returnează o tuplă conținând atât vectorul de probabilități pentru clase (**cls_out**), cât și valoarea scalară pentru numărare (**cnt_out**, procesată prin funcția **squeeze** pentru a elimina dimensiunile redundante).

Acest design permite calcularea simultană a funcțiilor de pierdere (Cross-Entropy pentru **out_class** și Mean Squared Error pentru **out_count**) în timpul fazei de antrenament, ponderându-le într-o singură pierdere globală (Joint Loss) pe care optimizatorul o va minimiza.

5.8. Interfața utilizator

Pentru a face sistemul accesibil și validabil de către utilizatorii finali, a fost dezvoltat un modul de inferență de tip end-to-end, învelit într-o interfață web interactivă construită cu biblioteca Gradio. Acest modul acționează ca un orchestrator care apelează secvențial toate modulele descrise anterior.

5.8.1. Funcția centrală de procesare

Logica de inferență este consolidată într-o funcție principală care primește ca argument calea către un videoclip încărcat de utilizator și parcurge următorii pași:

1. Decompunerea video: Utilizează cv2.VideoCapture pentru a citi cadrele videoclipului în memorie.
2. Extracția și transformarea: Cadrele sunt trimise către modulul de extragere a reperelor anatomice (**extract_kps_yolo**). Tabloul rezultat este trecut prin funcțiile **scale_keypoints_to_size**, **keypoints_to_heatmaps** și **heatmaps_to_Pmap**, rezultând tensorul final de trăsături.
3. Inferența AI: Tensorul este transferat pe dispozitivul de calcul (GPU/CPU). Calculul gradienților este dezactivat temporar utilizând contextul torch.no_grad() pentru a minimiza consumul de memorie. Tensorul este trecut prin metoda forward, rezultând vectorul de probabilități pentru clasă și numărul de repetiții. Probabilitatea maximă (torch.argmax) determină clasa prezisă.
4. Adnotarea vizuală: Folosind funcțiile de desenare din OpenCV, sistemul suprapune peste fiecare cadru original scheletul digital generat de YOLO, iar într-o fereastră separată este afișat exercițiul detectat și numărul de repetări, oferind un feedback vizual imediat.
5. Reasamblarea video: Cadrele adnotate sunt compilate înapoi într-un fișier video. Pentru a garanta compatibilitatea redării în browserele web, fișierul generat de OpenCV este procesat suplimentar folosind biblioteca MoviePy (VideoFileClip), fiind re-codat cu standardul H.264.

5.8.2. Construcția Interfeței Web

Interfața cu utilizatorul este definită procedural folosind structura gr.Blocks sau gr.Interface din Gradio. Interfața este compusă dintr-o componentă de intrare de tip

gr.Video (care permite încărcarea fișierelor locale sau înregistrarea directă din camera web) și componente de ieșire (un alt element gr.Video pentru rezultatul vizual și un element gr.Textbox pentru afișarea textului brut cu rezultatele). Când utilizatorul apasă butonul de procesare, Gradio pasează automat fișierul temporar către funcția **process_video_for_web** și randează răspunsul asincron.

Acest nivel de abstractizare separă logica complexă de viziune artificială de experiența utilizatorului, transformând codul de cercetare într-o aplicație software funcțională și ușor de testat.

5.9. Caz de utilizare: Procesarea unui video a unui antrenament fizic

Pentru a înțelege interacțiunea directă dintre actorii umani și aplicația software, a fost definit cazul de utilizare principal care descrie procesarea unei înregistrări video a unui antrenament fizic. Acest caz de utilizare evidențiază modul în care sistemul preia datele brute, aplică modelele de inteligență artificială și returnează un feedback vizual și numeric.

- **Actor principal:** Utilizatorul (Pasionat de fitness, sportiv amator sau antrenor personal).
- **Descriere scurtă:** Interacțiunea dintre Utilizator și Sistem pentru a procesa o înregistrare video a unui antrenament fizic. Sistemul extrage reperele anatomice, recunoaște în mod automat tipul de exercițiu și numără repetițiile efectuate, oferind la final rezultatele sub formă de text și un videoclip adnotat vizual cu scheletul utilizatorului.
- **Precondiții:** Aplicația web rulează, iar fișierele cu ponderile rețelelor neuronale (yolov8n-pose.pt și best_joint_f_model.pth) sunt încărcate în memoria sistemului, fiind pregătite pentru faza de inferență.
- **Fluxul de bază (Basic Flow):**
  1. Utilizatorul accesează interfața aplicației (ex. pagina web Gradio) având intenția de a analiza un antrenament.
  2. Utilizatorul încarcă un fișier video în sistem.
  3. Sistemul validează formatul și integritatea fișierului video.
  4. Sistemul extrage cadrele individuale (frame-by-frame) din videoclip.
  5. Sistemul aplică modelul de viziune artificială (YOLOv8) pentru a detecta reperele anatomice ale utilizatorului în fiecare cadru.
  6. Sistemul transformă coordonatele reperelor în hărți de probabilitate (heatmaps) și le comprimă într-o reprezentare statică bidimensională (P-maps).
  7. Sistemul analizează hărțile procesate folosind modelul multi-task (rețelele ResNet) pentru a clasifica tipul de exercițiu și a estima numărul de repetiții.
  8. Sistemul construiește un nou videoclip în care suprapune scheletul detectat peste cadrele originale.
  9. Sistemul afișează pe ecran clasa exercițiului detectat, numărul de repetiții și videoclipul adnotat cu scheletul detectat.
  10. Utilizatorul vizualizează rezultatele, iar cazul de utilizare se încheie cu succes.
- **Fluxuri alternative (Alternative Flows):**
  - **Fișier video invalid (Apare la pasul 3):** Sistemul semnalează o eroare indicând un format nesuportat sau un fișier corupt. Sistemul afișează un mesaj informativ către utilizator. Utilizatorul încarcă un nou fișier video, reluând fluxul de la pasul 2.

- **Persoană nedetectată în cadru (Apare la pasul 5):** Acest fenomen se produce atunci când utilizatorul iese din unghiul camerei sau este acoperit. Sistemul înregistrează o matrice de zerouri pentru coordonatele cadrelor respective. Sistemul continuă analiza cu următorul cadru din videoclip, revenind la fluxul de bază.
- **Descărcarea videoclipului adnotat (Apare la pasul 10):** Utilizatorul selectează opțiunea de descărcare a videoclipului rezultat. Sistemul transferă fișierul video format MP4 în stocarea locală a dispozitivului utilizatorului, iar cazul de utilizare se încheie.

- **Postcondiții:** Sistemul se află într-o stare stabilă. Videoclipul procesat, împreună cu eticheta clasificării și valoarea regresiei (repetițiile), sunt disponibile vizual pentru Utilizator, fără a altera sau distruge fișierul video sursă original.

5.10. Diagrama de secvență a sistemului

Pentru a detalia comunicarea asincronă și sincronă dintre componentele software pe parcursul rulării cazului de utilizare prezentat anterior, a fost elaborată următoarea diagramă de secvență.

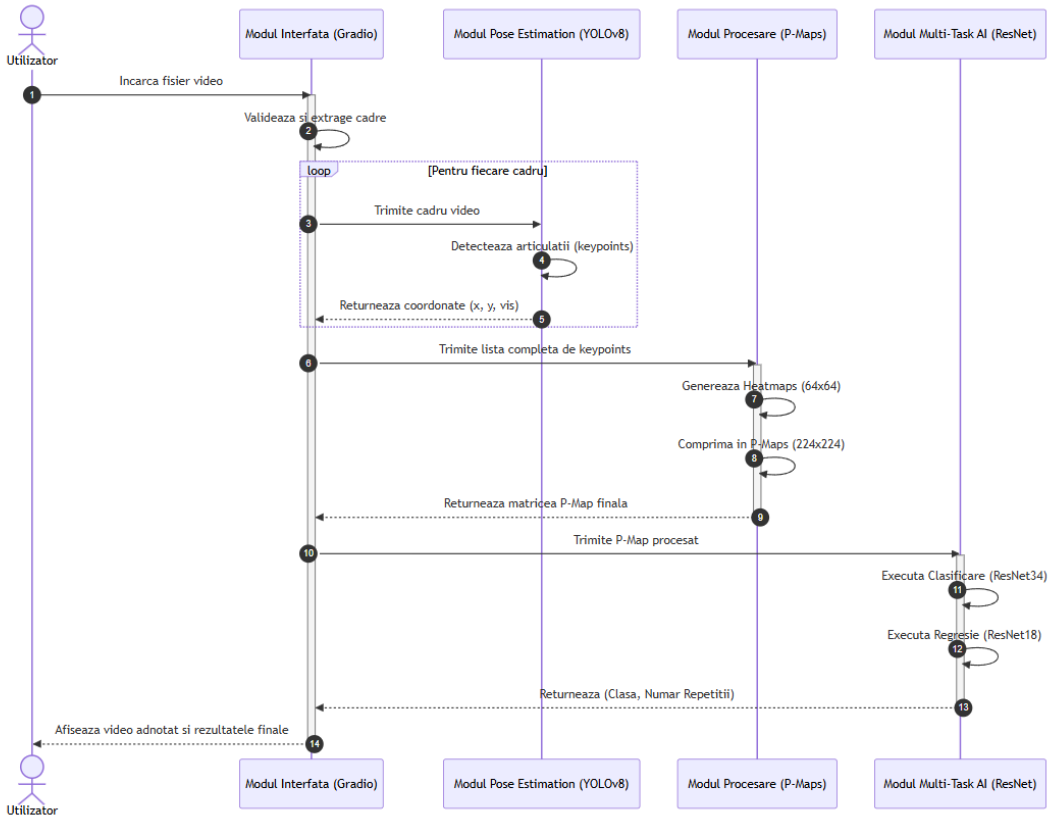


Figura 5.6: Diagrama de secvență a sistemului

5.10.1. Descrierea modulelor

- **Interfața (Gradio):** Este punctul de contact cu utilizatorul. Se ocupă de preluarea fișierului video și de asamblarea rezultatelor într-un format vizual (video cu schelet suprapus și text).
- **Pose Estimation (YOLOv8-pose):** Este responsabil pentru viziunea artificială primară. Analizează fiecare cadru pentru a identifica persoana și a extrage cele

17 puncte cheie (articulații) ale corpului uman, transformând pixelii în date matematice (coordonate).

- **Procesare (P-Maps):** Acest modul implementează logica specifică lucrării RepPenn [20]. Transformă coordonatele brute în hărți de probabilitate (Heatmaps) și apoi comprimă informația temporală într-o singură imagine (Projected Map), făcând datele ușor de înțeles pentru rețelele neuronale 2D.
- **Multi-Task Model (ResNet):** Utilizează o arhitectură partajată care alimentează două ramuri: una care recunoaște tipul de exercițiu (ResNet34) și una care numără repetițiile (ResNet18), oferind predicțiile finale.

5.10.2. Pașii efectuați

Procesul debutează cu încărcarea datelor, unde utilizatorul introduce un fișier video în aplicație prin intermediul interfeței web. Urmează faza de pre-procesare video, unde sistemul validează fișierul și îl împarte în cadre pentru a putea fi analizat secvențial. Extracția scheletului este realizată de modelul YOLOv8-pose, care procesează fiecare cadru și identifică poziția celor 17 articulații ale utilizatorului.

Odată extrase, are loc generarea hărților de probabilitate, unde coordonatele detectate sunt convertite în hărți de probabilitate gaussiene de 64x64 pixeli pentru a modela incertitudinea și distribuția spațială. Ulterior, crearea P-Map asamblează toate hărțile termice din videoclip, agregându-le și comprimându-le pentru a crea o hartă de proiecție de 224x224, care conține întreaga istorie a mișcării.

Harta P-Map este trimisă către ramura de recunoaștere a exercițiului (ResNet34) pentru a clasifica tipul mișcării. Simultan, aceeași hartă este analizată de ramura de regresie (ResNet18) pentru a număra repetițiile. În final, are loc afișarea feedback-ului, sistemul adnotând videoclipul original cu scheletul detectat și prezentând pe ecran rezultatele analizei.

Capitolul 6. Testare și validare

Acest capitol este dedicat evaluării empirice și analizei a arhitecturii software și a modelelor de inteligență artificială proiectate în capitolele anterioare. Scopul validării este de a demonstra performanța sistemului multi-task pe setul de date derivat RepPenn. Se vor detalia metodologia de antrenare partiționată, justificarea matematică a algoritmilor de optimizare, metricile utilizate pentru evaluare și se va realiza o analiză a perfonrmanței modelului pe parcursul celor trei etape fundamentale: antrenarea clasificării, antrenarea regresiei și ajustarea fină comună (Joint Finetuning). Graficile corespunzătoare acestor etape pot fi găsite în Figura 6.1, iar datele numerice în Tabel 6.1.

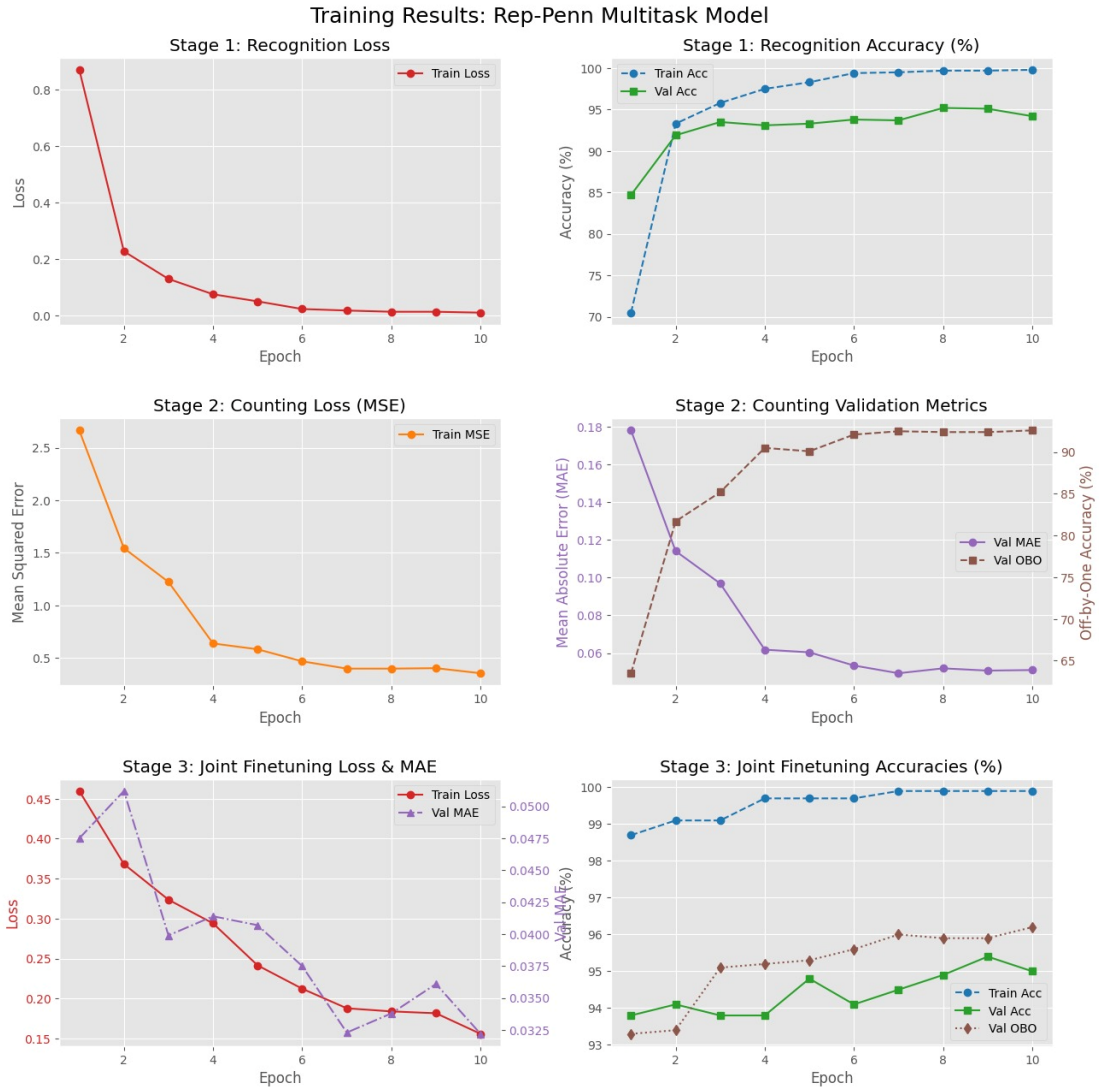


Figura 6.1: Graficile de pierdere (coloana din stânga) și a metricilor de validare (coloana din dreapta) a modelului în cele trei etape de antrenare.

6.1. Metodologia de antrenare

Antrenarea de la zero sau chiar ajustarea fină (finetuning) a unei rețele multi-task (MTL) prezintă provocări majore care nu se regăsesc în arhitecturile cu o singură

sarcină (Single-Task). Cea mai proeminentă problemă este fenomenul de dominanță a gradientului [22] (Gradient Interference sau Negative Transfer). Deoarece sarcinile de clasificare și regresie operează pe scări de eroare complet diferite, actualizarea simultană a ponderilor din trunchiul comun (straturile **shared**) poate duce la o instabilitate severă a procesului de învățare. Pentru a mitiga acest risc, metodologia de antrenare a fost structurată strategic într-un pipeline izolat, în trei etape (Three-Stage Training Pipeline).

Această abordare progresivă permite rețelei să învețe un spațiu intern de bază (feature space) stabil, fără interferențe, unificând sarcinile abia în faza finală. Modificarea controlată a fluxului de gradient este implementată programatic prin manipularea atributului **requires_grad** al tensorilor din PyTorch, permițând ”înghețarea” (freezing) sau ”dezghețarea” (unfreezing) selectivă a anumitor sub-rețele.

6.1.1. Alegerea optimizatorului

În locul algoritmului clasic de coborâre a gradientului stocastic (Stochastic Gradient Descent - SGD), pentru antrenarea acestui sistem a fost ales algoritmul RMSprop (Root Mean Square Propagation). Motivația din spatele acestei alegeri rezidă în natura non-staționară a obiectivelor din învățarea multi-task și în capacitatea algoritmului de a adapta rata de învățare independent pentru fiecare parametru (Adaptive Learning Rate).

Din punct de vedere matematic, RMSprop menține o medie mobilă exponențială (exponential moving average) a pătratelor gradienților anteriori pentru fiecare pondere w:

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1-\beta) g_t^2$$

Unde:

- $\boldsymbol{g_t}$ este gradientul la pasul t
- $\boldsymbol{\beta}$ este rata de micșorare (decay rate), setată implicit la o valoare apropiată de 0.99.

Actualizarea propriu-zisă a ponderii se face divizând rata de învățare globală (η) la rădăcina pătrată a acestei medii:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

Parametrul ϵ este o constantă foarte mică adăugată pentru a preveni împărțirea la zero. Această formulare teoretică asigură că parametrii care primesc gradienți mari și frecvenți (care ar putea cauza oscilații divergente) vor avea o rată de învățare redusă, în timp ce parametrii cu gradienți mici (care învață lent) vor primi un impuls (boost), asigurând o parcurgere mult mai lină a topologiei spațiului de eroare.

6.1.2. Regularizarea prin degradarea ponderilor

Pentru a preveni fenomenul de supra-învățare (overfitting) – un risc inerent având în vedere capacitatea mare a modelelor ResNet (milioane de parametri) raportată la dimensiunea setului de date RepPenn – a fost introdusă o penalizare a complexității modelului, cunoscută sub numele de Weight Decay, setată la valoarea de 10^{-4}. Aceasta echivalează cu o regularizare matematică de tip L2. Funcția de pierdere originală L_0 este modificată prin adăugarea sumei pătratelor tuturor ponderilor din rețea:

$$L = L_0 + \frac{\lambda}{2}\sum w_i^2$$

Prin aplicarea derivatei, optimizatorul va forța valorile ponderilor să se apropie de zero la fiecare pas, încurajând rețeaua să utilizeze doar un set minim de trăsături pentru decizie, distribuind astfel ”responsabilitatea” pe mai mulți neuroni și îmbunătățind capacitatea de generalizare pe date noi.

6.1.3. Planificatorul ratei de învățare

Controlul ratei de învățare (Learning Rate - LR) a fost gestionat dinamic prin intermediul unui planificator în trepte (Step Learning Rate Scheduler). Rata de învățare inițială determină dimensiunea pașilor pe care algoritmul îi face în direcția opusă gradientului. O rată prea mare duce la ocolirea minimului global, în timp ce o rată prea mică duce la blocarea într-un minim local sau la un timp de antrenament excesiv de lung.

Sistemul implementat utilizează strategia MultiStepLR. Aceasta permite modelului să facă pași mari de explorare a spațiului de eroare în primele epoci, când parametrii sunt departe de optim. La atingerea unui numar predefinit de epoci, cum ar fi epocile 1 și 5 pentru etapa de recunoaștere, planificatorul înmulțește rata curentă de învățare cu un factor de diminuare ($\gamma = 0.1$ sau $\gamma = 0.5$). Această reducere bruscă (Decay Step) acționează ca un mecanism de rafinare (similar cu simulated annealing), forțând algoritmul să facă pași din ce în ce mai fini (micro-ajustări) pentru a se ”așeza” exact pe fundul ”văii” corespunzătoare minimului global al funcției de pierdere.

6.2. Metricile de evaluare a performanței

Pentru a măsura obiectiv și cantitativ succesul aplicației, funcția **evaluate_model** implementează o suită de metrici matematice distincte, strict adaptate pentru a capta comportamentul specific al celor două ramuri decizionale (clasificarea categorială și regresia scalară).

6.2.1. Metrica pentru Clasificare: Acuratețea

Deoarece setul de date sintetizat (RepPenn) a fost construit prin augmentarea sistematică a claselor din setul de bază PennAction, distribuția videoclipurilor per categorie de exercițiu este echilibrată. În acest context, Acuratețea globală reprezintă cea mai simplă și intuitivă metrică pentru ramura de recunoaștere (**recog_net**).

Matematic, acuratețea se definește ca raportul dintre numărul de predicții corecte și numărul total de evaluări (mărimea lotului de testare):

$$Accuracy = \frac{\sum_{i=1}^{N} \mathrm{I}(\hat{y}_i = y_i)}{N}$$

Unde:

- $\boldsymbol{N}$ este numărul total de videoclipuri din setul de test
- $\boldsymbol{y_i}$ este clasa reală (Ground Truth)
- $\boldsymbol{\hat{y}_i}$ este clasa prezisă de rețea (obținută aplicând funcția $argmax$ pe vectorul de ieșiri probabile)
- **I** este funcția indicator care returnează 1 dacă condiția este adevărată și 0 în caz contrar.

6.2.2. Metricile pentru Regresie

Evaluarea ramurii de numărare (**count_net**) necesită metrici care măsoară distanța pe axa numerelor reale între predicție și realitate.

- Eroarea Medie Absolută (Mean Absolute Error - MAE) și Deviația Standard (σ)

Cea mai simplă măsurătoare este media diferențelor absolute dintre valoarea scalară prezisă (p_i) și numărul real de repetiții (g_i):

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|p_i - g_i|$$

Pe lângă medie, a fost calculată și deviația standard a diferențelor ($\sigma = std(p - g)$). Această metrică este importnantă pentru analiza de validare, deoarece indică stabilitatea rețelei. O deviație standard mică semnifică faptul că erorile rețelei sunt grupate strâns în jurul mediei (modelul are un comportament previzibil), pe când o deviație mare indică prezența unor erori destul de indepărtate de medie (outliers).

- Eroarea Medie Absolută Normalizată (Normalized MAE)

În contextul antrenamentelor fizice, gravitatea unei erori de numărare depinde direct de volumul total al exercițiului efectuat. O eroare de o repetiție la un videoclip în care utilizatorul execută 20 de repetiții reprezintă o eroare relativă mică (5%). În schimb, aceeași eroare de o repetiție pentru un videoclip scurt în care se execută o singură repetiție reprezintă o abatere masivă (100%).

Pentru a penaliza corect modelul în funcție de contextul efortului, algoritmul implementează o versiune normalizată a erorii absolute (MAE), scalând diferența la valoarea reală a numărului de repetiții:

$$MAE_{norm} = \frac{1}{N}\sum_{i=1}^{N}\frac{|p_i - g_i|}{\max(1.0, g_i)}$$

Utilizarea funcției $\max(1.0, g_i)$ la numitor este un artificiu matematic necesar pentru a preveni erorile de tip ”împărțire la zero” în cazurile rare în care valoarea de adevăr ar fi 0.

- Metrica Off-By-One (OBO Accuracy)

Dacă MAE oferă o perspectivă științifică a erorii, metrica OBO reprezintă valoarea comercială a sistemului, tradusă în calitatea experienței utilizatorului final (User Experience). Deoarece mișcările umane sunt continue și fluide, definirea exactă a ”sfârșitului” unei repetiții poate fi ambiguă (de exemplu, o flotare finalizată doar pe jumătate).

OBO (Off-By-One) relaxează constrângerea de potrivire exactă, considerând o predicție ca fiind un ”succes” dacă distanța absolută față de numărul real este mai mică sau egală cu o repetiție:

$$OBO = \frac{1}{N}\sum_{i=1}^{N}\mathrm{I}(|p_i - g_i| \leq 1.0)$$

Unde:

- $\boldsymbol{p_i}$ este numărul de repetiții prezis
- $\boldsymbol{g_i}$ este numărul de repetiții adevărat (ground truth)
- $\boldsymbol{I}$ este funcția indicator care returnează 1 dacă condiția este adevărată și 0 în caz contrar.

Un procentaj OBO ridicat demonstrează că sistemul este potrivit pentru implementarea într-o aplicație de asistență virtuală pentru fitness, deoarece marja de eroare de ± 1 repetiție este considerată neglijabilă de către majoritatea sportivilor în timpul antrenamentelor de anduranță.

6.3. Analiza Etapei 1: Antrenarea ramurii de recunoaștere

Prima etapă a pipeline-ului de antrenament a avut un obiectiv definit în felul următor: construirea unui spațiu intern de trăsături (feature space) funcțional, capabil să separe semantic cele 7 clase de exerciții. Parametrii ramurii de regresie (**model.count_net**) au fost ”înghețați” programatic (**requires_grad = False**). Astfel, gradientul generat de funcția de pierdere Cross-Entropy a parcurs exclusiv ramura **recog_net** și prin trunchiul comun **shared**. Această decizie arhitecturală împiedică ca trăsăturile spațio-temporale de bază să fie alterate de o sarcină de regresie care încă nu și-a stabilit propriile ponderi.

Antrenamentul s-a desfășurat pe parcursul a 10 epoci, utilizând un set de antrenament (**train_loader**) de 432 de loturi (batches) a câte 16 videoclipuri (reprezentând în total 6912 secvențe sintetizate) și un set de testare (**test_loader**) de 109 loturi (cele 1736 secvențe rămase). Rata de învățare inițială a fost setată la $\eta = 10^{-4}$ și a fost redusă la $\eta = 10^{-5}$ începând cu Epoca 5, conform planificatorului MultiStepLR.

6.3.1. Analiza convergenței și a performanței

- Faza de explorare (Epocile 1-4): Încă de la prima epocă, modelul a demonstrat capacitatea extraordinară a hărților P-map de a codifica natura mișcării. Cu o pierdere de antrenament inițială de 0.8701, acuratețea de testare a atins deja un nivel de 84.7%. Această convergență rapidă este justificată de pre-antrenamentul straturilor ResNet-34 pe ImageNet (transfer learning); rețeaua știe deja să extragă forme geometrice (margini, texturi), necesitând doar o adaptare a capătului decizional la formele de undă specifice P-maps. Până la Epoca 4, eroarea de antrenament a scăzut până la 0.0746, acuratețea la antrenament ajungând la 97.5%.

- Faza de rafinare (Epocile 5-10): Trecerea la rata de învățare de 10^{-5} a permis micro-ajustarea ponderilor. Pierderea la antrenament a tins exponențial spre zero (ajungând la 0.0095 în Epoca 10), iar acuratețea pe datele de antrenament s-a saturat la 99.8%. Acest fenomen indică faptul că modelul a ”memorizat” aproape perfect topologia datelor de antrenament.

Cel mai bun model pentru clasificare a fost salvat la Epoca 8, unde s-a înregistrat acuratețea maximă pe setul de validare: **95.2**%.

6.3.2. Discuție asupra fenomenului de generalizare

Diferența dintre acuratețea de antrenament (99.8%) și cea de validare (95.2%) sugerează o ușoară tendință de supra-învățare (overfitting). Acest fenomen este explicabil și acceptabil. Capacitatea rețelei ResNet-34 este mult mai mare decât complexitatea reală a celor 7 clase din setul RepPenn. Cu toate acestea, atingerea unui

prag de peste 95% pe un set de test complet independent (format din subiecți diferiți, filmați în unghiuri diferite) demonstrează validitatea științifică a abordării: comprimarea mișcării 3D în hărți statice 2D păstrează suficiente caracteristici discriminative pentru o clasificare de înaltă performanță. Ponderile obținute la Epoca 8 au fost izolate și salvate (**best_recog_f_model.pth**) pentru a reprezenta fundația etapei următoare.

6.4. Analiza Etapei 2: Antrenarea ramurii de regresie

Cu un backbone capabil acum să clasifice corect exercițiile, Etapa a 2-a a fost dedicată regresiei (numărării ciclurilor de mișcare). În această fază, logica de antrenament a fost inversată: ponderile din **shared** și **recog_net** au fost blocate (**requires_grad = False**), iar parametrii ramurii **count_net** (ResNet-18) au fost deblocați și antrenați de la zero (dincolo de ponderile de bază ImageNet).

Această izolare previne ceea ce în teoria Machine Learning se numește uitarea catastrofală (Catastrophic Forgetting). Dacă trunchiul comun ar fi fost actualizat exclusiv pe baza erorii de numărare, rețeaua ar fi pierdut abilitatea de a distinge între o flotare și o genuflexiune, concentrându-se doar pe oscilații.

Pentru această etapă de regresie, funcția de pierdere utilizată a fost Eroarea Medie Pătratică (MSELoss). Optimizatorul RMSprop a fost reinițializat cu o rată de învățare superioară ($\eta = 10^{-3}$), menită să scoată ramura ResNet-18 din starea sa inițială, planificatorul fiind configurat să reducă această rată la epocile 3 și 6, la fel ca și în etapa 1, cu un factor de 0.1.

6.4.1. Analiza convergenței și a metricilor specifice

- Convergența rapidă (Epocile 1-2): Datorită ratei mari de învățare, modelul a suferit modificări masive ale ponderilor. De la o eroare MSE de 2.6658, aceasta s-a redus la 1.5445 în doar o epocă. Metrica MAE pe validare a înregistrat 0.1781, indicând o eroare medie de aproximativ 17% față de numărul real de repetiții. OBO a pornit de la 63.5%, sugerând că în stadiul incipient, modelul reușește să ”ghicească” repetițiile doar pentru cazurile foarte simple (ex. un număr mic de cicluri).
- Rafinarea (Epocile 3-5): Odată cu reducerea ratei de învățare la 10^{-4} în Epoca 3, s-a produs un salt calitativ semnificativ. Eroarea MSE de antrenament a scăzut până la 1.2226, iar MAE de validare a coborât brusc la 0.0969. Până la Epoca 5, MSE a ajuns la 0.5811, iar MAE la 0.0604. Metrica OBO, a trecut pragul de 90% (înregistrând 90.5%), dovedind că rețeaua ResNet-18 a învățat să interpreteze corect formele de undă periodice din hărțile P-map.
- Saturația (Epocile 6-10): Scăderea ratei de învățare la 10^{-5} a permis stabilizarea modelului pe minimul local optimizat.

Cel mai bun rezultat, marcat matematic de cea mai mică eroare MAE de validare, a fost obținut la Epoca 7: 0.0493 (reprezentând o eroare medie normalizată sub 5%), însoțit de un OBO de 92.5%.

6.4.2. Discuție asupra performanței izolate de regresie

Obținerea unui OBO de 92.5% antrenând o singură ramură pe un set de date static demonstrează că numărarea repetițiilor din hărțile P-map este o sarcină realizabilă pentru arhitecturile de recunoaștere a texturilor (precum ResNet-18). Modelul nu ”înțelege” biomecanica reală, ci numără densitatea vârfurilor luminoase de pe axa orizontală a tensorului P-map. Totuși, stagnarea erorii MAE în jurul valorii de 0.05 (la

epocile 8-10) sugerează că modelul a atins limita sa teoretică în această configurație izolată. Pentru a depăși acest prag, ramura de regresie are nevoie ca trăsăturile de bază (din blocul **shared**) să devină mai fine și mai adaptate cerințelor sale numerice, aspect care justifică logic tranziția către etapa finală: Finetuning-ul Comun (Joint Finetuning).

6.5. Analiza Etapei 3: Reglaj fin

A treia etapă a procesului de antrenare a reprezentat testul de validare al paradigmei Multi-Task Learning. Dacă în primele două etape ramurile au fost izolate pentru a stabili ponderile de bază, obiectivul acestei faze a fost ajustarea lor. Toate straturile rețelei, inclusiv trunchiul comun **shared**, ramura de clasificare **recog_net** și ramura de numărare **count_net**, au fost deblocate simultan (**requires_grad** = **True**).

Această abordare permite gradientului să circule complet de la ambele funcții de pierdere înapoi spre primele straturi de convoluție, forțând modelul să identifice o reprezentare matematică optimă care să satisfacă ambele sarcini.

6.5.1. Funcția de pierdere compusă

În această etapă, optimizarea s-a realizat prin minimizarea unei funcții de pierdere compuse, definită liniar ca suma ponderată a erorilor individuale:

$$Loss_{total} = L_{crossentropy} + L_{mse}$$

Astfel, ambele sarcini au avut o pondere egală în ajustarea vectorului gradient.

6.5.2. Strategia de optimizare

Deoarece modelul deținea deja ponderi relativ bune din etapele anterioare, aplicarea unei rate de învățare mari ar fi distrus (catastrophic forgetting) aceste informații. Prin urmare, optimizatorul RMSprop a fost inițializat cu o rată de învățare conservatoare de $\eta = 10^{-4}$, cu o penalizare L2 (weight decay) de 10^{-4}. Planificatorul a fost ajustat pentru o coborâre mai lentă ($\gamma = 0.5$ comparativ cu 0.1 din etapele anterioare) la epocile 3 și 6. Această strategie de diminuare în trepte mici este specifică tehnicilor de Finetuning, permițând modelului să coboare lent către minimul global.

6.5.3. Analiza convergenței și a performanței

Impactul imediat (Epoca 1-3): Încă de la prima epocă de antrenament comun, efectul a fost observabil. Acuratețea de validare a ramurii de recunoaștere a crescut imediat la 93.8%, iar eroarea MAE a ramurii de regresie a scăzut brusc la 0.0475, un scor deja mai bun decât recordul absolut din antrenamentul izolat (Etapa 2). Până la Epoca 3 (înaintea primei reduceri a ratei de învățare), eroarea MAE a coborât la 0.0399, iar procentul OBO a atins un prag de 95.1%. Această scădere indică fenomenul de ”transfer pozitiv” (Positive Transfer): ramura de regresie și-a îmbunătățit predicțiile folosind trăsăturile semantice mai fine extrase de ramura de clasificare.

Faza de consolidare (Epocile 4-7): La o rată de învățare de 5×10^{-5}, modelul a continuat să se îmbunătățească, ajungând la Epoca 7 la o acuratețe de clasificare de 94.5% și o eroare MAE de regresie minimă de 0.0323, cu un OBO de 96.0%.

Convergența finală (Epocile 8-10): După ultima ajustare a ratei de învățare (2.5×10^{-5}) modelul s-a stabilizat. Deși acuratețea de clasificare a fluctuat ușor, atingând 95.4% la Epoca 9, ramura de numărare a înregistrat recordul său absolut la Epoca 10, cu un MAE de 0.0322 și o metrică OBO de 96.2%. Pierderea globală (Train

Loss) a încheiat la o valoare scăzută de 0.1557, reflectând o convergență stabilă și reușită a ambelor obiective.

6.6. Analiza timpului de execuție

6.6.1. Specificațiile mediului hardware

Toate experimentele de antrenare, ajustare a hiperparametrilor și testare au fost realizate pe o configurație hardware accesibilă, de nivel de consumator (consumer-grade), centrată în jurul unei plăci video NVIDIA GeForce RTX 4060 echipată cu 8GB de memorie RAM dedicată (VRAM). Această alegere tehnologică are o semnificație semnificativă pentru proiect: unul dintre argumentele centrale ale lucrării este accesibilitatea tehnologiilor de tip ”Smart Coaching” prin eliminarea dependenței de clustere industriale de calcul sau supercomputere costisitoare.

6.6.2. Evaluarea costului antrenării stratificate

Implementarea strategiei de antrenament în trei etape distincte (Three-Stage Pipeline) a necesitat cicluri separate de optimizare, fiecare având un cost temporal specific, în medie de două ore.

Astfel, timpul total de antrenare cumulat pe parcursul celor trei faze a fost de 6 ore. Din punct de vedere al ingineriei software aplicate în inteligența artificială, o durată de 6 ore pentru un model capabil de performanțe multi-obiectiv pe mii de secvențe reprezintă o eficiență remarcabilă.

6.6.3. Analiza timpului de răspuns

În analiza timpului de rulare (runtime analysis), este importantă stabilirea unei distincții clare între latența brută de execuție a modelului neuronal (core inference) și timpul de răspuns global al aplicației end-to-end (system latency) perceput de utilizatorul final.

Timpul de inferență dedicat strict rețelei neuronale propuse (MultiTaskModel) este de **sub 3 secunde**, manifestând o proprietate remarcabilă de independență în raport cu numărul de cadre ale videoclipului încărcat. Această complexitate constantă, notată teoretic ca $O(1)$, este o consecință directă a modului în care au fost proiectate hărțile proiectate (P-maps). Indiferent dacă o secvență video originală are o lungime de 3 secunde (90 de cadre) sau de 15 secunde (450 de cadre), transformarea spațio-temporală realizează o redimensionare și o normalizare a istoriei mișcării într-o matrice statică unică, cu dimensiunea fixă de 224×224 pixeli. Prin urmare, numărul de operații în virgulă mobilă (FLOPs) necesar pentru propagarea înainte (forward pass) prin straturile ResNet rămâne neschimbat, garantând un răspuns aproape instantaneu.

Cu toate acestea, atunci când sistemul este utilizat prin intermediul interfeței grafice web (Gradio), timpul total de răspuns devine liniar proporțional, notat ca $O(N)$, cu durata secvenței multimedia încărcate de utilizator. Acest comportament este guvernat de modulele auxiliare de preprocesare vizuală și post-procesare (randare):

1. Randarea feedback-ului vizual: Pentru a asigura o componentă demonstrativă de înaltă calitate, sistemul reconstruiește un nou fișier video în care suprapune scheletul digital extras peste corpul utilizatorului în fiecare cadru.
2. Recodificarea multimedia: Asamblarea cadrelor adnotate și comprimarea lor folosind standardul H.264 (prin intermediul bibliotecii MoviePy) pentru a asigura compatibilitatea redării în browserele web este un proces dependent direct de volumul total de cadre (N).

Prin urmare, în timp ce execuția de inferență a sistemului se realizează în mai puțin de 3 secunde datorită abstractizării P-map, pipeline-ul complet din interfața grafică scalează cu lungimea fișierului. Această corelație reprezintă un compromis predictibil și bine echilibrat, oferind utilizatorului un feedback vizual bogat în informații în schimbul unei latențe proporționale de procesare.

Tabel 6.1: Rezultatele metricilor în urma etapelor de antrenare

Etapa de antrenare	**Funcția de pierdere (Antrenare)**	**Acuratețe recunoaștere (Antrenare)**	**Acuratețe recunoaștere (Validare)**	**Eroare Medie Absolută (MAE)**	**Performanța Off-by-one (OBO)**
Recunoaștere a exercițiului	0.0126	99.7%	95.2%	-	-
Numărarea repetărilor	0.3972	-	-	0.0493	92.5%
Reglaj fin	0.1557	99.9%	95.0%	0.0322	96.2%

Capitolul 7. Manual de instalare si utilizare

Acest capitol are ca scop oferirea unui ghid complet pentru punerea în funcțiune a aplicației dezvoltate. Prima secțiune detaliază cerințele de sistem și pașii necesari pentru instalarea mediului software. A doua secțiune reprezintă manualul de utilizare propriu-zis, menit să ghideze utilizatorul final în interacțiunea cu interfața grafică a platformei.

7.1. Cerințe de sistem

Pentru ca aplicația de recunoaștere a exercițiilor fizice să funcționeze în parametri optimi, sistemul gazdă trebuie să îndeplinească următoarele cerințe minime și recomandate.

7.1.1. Cerințe hardware

- Procesor (CPU): Intel Core i5 de generația a 8-a / AMD Ryzen 5 (sau echivalent) - minim; Intel Core i7 / AMD Ryzen 7 - recomandat.
- Memorie RAM: Minim 8 GB; Recomandat 16 GB pentru a asigura rularea fluentă a mediului web și procesarea cadrelor video în memorie.
- Placă video (GPU): Sistemul poate rula exclusiv pe procesor (CPU), însă timpul de inferență va fi simțitor mai mare. Pentru performanțe optime și timp de răspuns de sub 3 secunde, se recomandă o placă video NVIDIA cu suport CUDA și un minim de 8 GB VRAM (ex. NVIDIA GeForce RTX 3060 / 4060).
- Spațiu de stocare: Minim 5 GB spațiu liber pe disc pentru instalarea bibliotecilor Python și stocarea modelelor pre-antrenate.

7.1.2. Cerințe software

- Sistem de operare: Windows 10/11, Ubuntu 20.04+ sau macOS.
- Mediul de execuție: Python versiunea 3.10 (sau mai nouă).
- Drive-re grafice: NVIDIA Display Driver și CUDA Toolkit (versiunea 11.8 sau 12.1), doar în cazul utilizării accelerării hardware (GPU).

7.2. Ghid de instalare

Următorii pași descriu modul de instalare și configurare a aplicației pe un sistem de operare Windows, folosind linia de comandă (Command Prompt sau PowerShell).

Pasul 1. Instalarea Python: Dacă Python nu este instalat pe sistem, descărcați programul de instalare de pe site-ul oficial[3]. În timpul procesului de instalare, asigurați-vă că bifați opțiunea ”Add Python to PATH”.

Pasul 2. Preluarea fișierelor aplicației: Copiați directorul cu fișierele sursă ale proiectului pe discul local. Asigurați-vă că arhiva conține structura de bază:

- Fișierul principal de execuție (demo.py).
- Fișierul cu utilitare (demo_utils.txt).
- Fișierele cu ponderile modelelor neuronale (yolov8n-pose.pt și best_joint_f_model.pth).

[3] Site-ul oficial python

Pasul 3. Crearea unui mediu virtual (Virtual Environment): Pentru a nu intra în conflict cu alte pachete instalate pe sistem, se recomandă izolarea aplicației într-un mediu virtual. Deschideți terminalul, navigați în folderul proiectului și executați comanda: **python -m venv reppenn**

Pasul 4. Activarea mediului virtual: După crearea mediului, acesta trebuie activat executând următorul script: **venv\Scripts\activate**. Activarea cu succes va fi indicată de apariția prefixului (reppen) în linia de comandă, precum se vede în Figura 7.1.

Pasul 5. Instalarea bibliotecilor necesare: Cu mediul virtual activat, instalați toate bibliotecile externe necesare rulării aplicației (PyTorch, Ultralytics, Gradio, OpenCV, etc.) prin intermediul managerului de pachete pip: **pip install <numele bibliotecii>**

Pasul 6. Plasarea ponderilor modelului: Verificați ca fișierul best_joint_f_model.pth (rezultat în urma antrenării) și fișierul YOLO (yolov8n-pose.pt) să se afle în același director rădăcină cu fișierul de execuție. Aplicația este acum instalată și gata de utilizare.

```
(reppen) PS D:\user\Work\University\Anul4\SRF\project> python demo.py
* Running on local URL:  http://127.0.0.1:7860
* Running on public URL: https://81e569c246072ffd31.gradio.live
```

Figura 7.1: Execuția din linia de comandă a codului pentru demonstrație.

7.3. Manual de utilizare

Aplicația a fost concepută având în centru principiul simplității. Interfața grafică este intuitivă, minimalistă și se accesează direct dintr-un browser web standard (Google Chrome, Mozilla Firefox, Safari).

Lansarea aplicației: Pentru a porni aplicația, rulați fișierul principal din terminalul unde ați activat mediul virtual: **python demo.py**. În terminal va apărea un mesaj care indică faptul că serverul local a pornit cu succes, însoțit de un link, precum se poate vedea în Figura 7.1. Copiați acest link și deschideți-l în browserul web preferat sau dați diret click pe link.

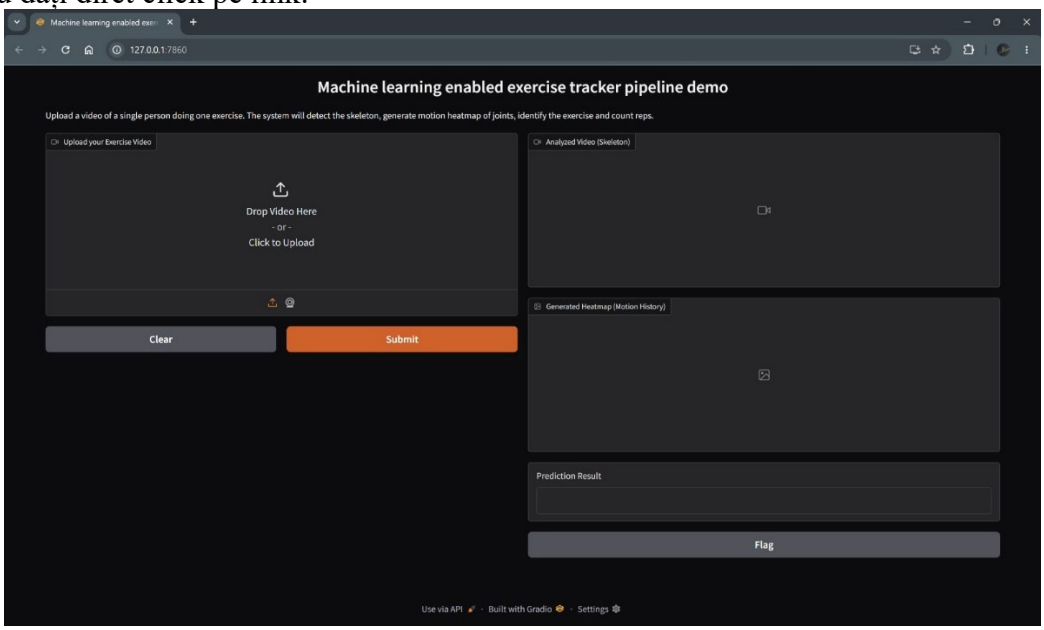


Figura 7.2: Interfața aplicației demonstrative.

Navigarea interfeței: Odată deschisă pagina, se poate vedea interfața principală, ilustrată în Figura 7.2, care este împărțită logic în două zone principale:

1. **Zona de intrare (Input):** Situată de în partea stângă a ecranului. Aici este locul unde se furnizează materialul video pe care doriți să îl analizați.
2. **Zona de ieșire (Output):** Situată în partea dreaptă a ecranului. Aici sistemul va afișa automat rezultatele procesării sub formă de text (tipul exercițiului și numărul de repetiții) și un videoclip demonstrativ.

Pasul 1. Încărcarea videoclipului: Interacțiunea începe prin adăugarea fișierului video ce conține antrenamentul dumneavoastră.

- Faceți click pe zona care indică **"Drop Video Here"** (Trageți videoclipul aici) sau **"Click to Upload"** (Click pentru a încărca).
- Se va deschide fereastra navigatorului de fișiere din sistemul dumneavoastră. Selectați videoclipul dorit și apăsați butonul **"Open"** (Deschide).
- Alternativ, puteți trage direct fișierul video dintr-un folder peste zona de încărcare din browser.

Pasul 2. Pornirea analizei: După ce videoclipul a fost încărcat cu succes în caseta de intrare, tot ce aveți de făcut este să apăsați butonul principal de execuție, etichetat **"Submit"** (Trimite). Veți observa un indicator de progres care vă informează că secvența video este procesată. Nu închideți fereastra browserului în timpul acestui proces.

Pasul 3. Interpretarea rezultatelor: În doar câteva secunde (timpul poate varia ușor în funcție de lungimea videoclipului), zona de ieșire din partea dreaptă va fi populată automat cu rezultatele analizei, cum se poate observa în Figura 7.3.

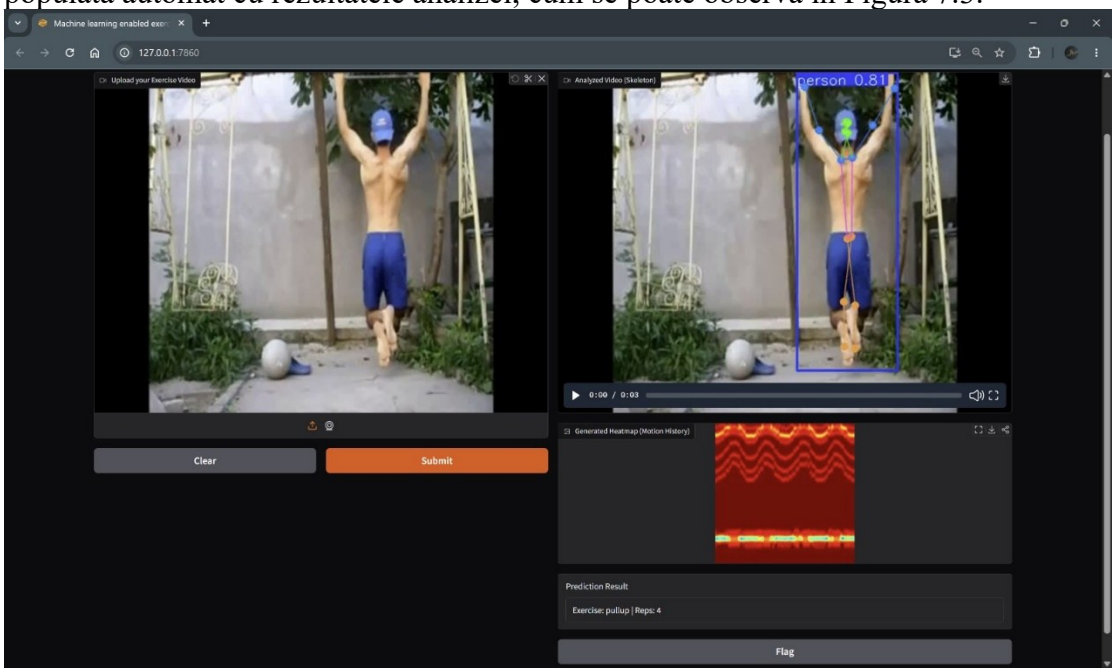


Figura 7.3: Interfața aplicației pentru demonstrație cu un fișier de intrare încărcat și rezultatele procesării afișate.

Rezultatul vă oferă două tipuri de informații:

- **Feedback numeric și textual:** Într-o casetă de text dedicată, aplicația vă va spune cu exactitate ce fel de exercițiu ați executat (de exemplu, „squat", „pullup") și vă va oferi contorul exact al repetițiilor efectuate cu succes.
- **Feedback vizual (Video Adnotat):** Pe ecran va rula un nou videoclip. Acesta este videoclipul original, peste care sistemul a desenat în culori un „schelet" digital. Acest videoclip demonstrativ vă confirmă faptul că sistemul v-a urmărit

corect mișcările pe toată durata antrenamentului. La fel, în altă casetă, este afișată și harta de probabilități suprapusă a tuturor articulațiilor în timp.

Pasul 4. Descărcarea rezultatului (Opțional): Dacă doriți să păstrați o copie a videoului cu scheletul suprapus sau harta de probabilități, puteți interacționa cu videoclipul (sau harta) din secțiunea de rezultate. Majoritatea browserelor permit descărcarea acestuia fie printr-un buton dedicat aflat în colțul ferestrei video (sau butonul **”Flag”**), fie dând click dreapta pe videoclip și selectând opțiunea „Save video as...” (Salvează videoclipul ca...).

Aplicația poate fi utilizată în mod repetitiv; pentru a analiza o nouă sesiune de antrenament, faceți click pe iconița de ștergere (**”Clear”**) din colțul zonei de încărcare video și repetați pașii descriși mai sus.

Capitolul 8. Concluzii

8.1. Performanța generală

Validarea experimentală a sistemului pe setul de date RepPenn a demonstrat succesul integrării tehnicilor de viziune artificială cu paradigma învățării multi-task. Evaluarea empirică a evidențiat faptul că rețeaua neuronală a atins un nivel înalt de performanță academică și aplicabilitate practică, înregistrând o acuratețe de clasificare de peste 95% și o eroare medie absolută (MAE) de regresie de 0.0322, rezultate foarte apropiate de cele din lucrarea de referință [20], însă obținute cu un set de date mult mai mic.

Analiza limitărilor actuale ale sistemului: În ciuda performanțelor ridicate, o analiză critică impune identificarea și punctelor slabe ale proiectului actual. Sistemul prezintă o dependență directă de calitatea inferenței primare realizate de modelul YOLOv8-pose. În scenariile în care utilizatorul suferă ocluziuni severe și prelungite (de exemplu, un membru este ascuns complet în spatele trunchiului pe o perioadă lungă dintr-un unghi nefavorabil al camerei) sau în care condițiile de iluminare sunt extrem de slabe (generând zgomot masiv în pixeli), modelul YOLO tinde să returneze coordonate eronate sau nule. Deși funcția de procesare spațio-temporală tratează aceste cazuri generând matrici de zero pentru a conserva axa timpului, o densitate prea mare de cadre ocluzionate distorsionează undele din P-map, putând conduce la degradarea rezultatului contorului de repetiții. De asemenea, sistemul actual este configurat exclusiv pentru analiza activității unui singur om, neavând implementată logica de urmărire multi-persoană în cazul în care în cadru apar mai mulți sportivi simultan.

8.2. Contribuții personale

Dezvoltarea acestui proiect a presupus rezolvarea unor probleme de inginerie a datelor și proiectare software, conducând la următoarele contribuții:

- **Sinteza și augmentarea setului de date RepPenn:** Pornind de la cadrele statice izolate și limitate ale setului de date PennAction, a fost conceput și implementat un algoritm de generare și augmentare temporală. Prin tehnica de compunere inversată (Forward-Backward Loop) și eșantionare la tempo-uri diferite, s-a reușit crearea artificială a unor cicluri de mișcare fluide, multiplicând volumul de date disponibil și oferind diversitatea cinematică necesară antrenării rețelelor neuronale.
- **Proiectarea fluxului de procesare:** A fost implementat un pipeline modificat față de lucrarea de referință, optimizat matematic prin operații vectorizate NumPy, care preia coordonatele carteziene discrete furnizate de YOLOv8.
- **Construirea unei aplicații:** Logica de cercetare a fost integrată într-un produs software funcțional prin intermediul interfeței web Gradio, cuplată cu module de post-procesare video (OpenCV și MoviePy), demonstrând că un sistem AI complex poate fi transformat într-o utilitate simplă, accesibilă de pe orice browser sau mobil.

8.3. Posibile dezvoltări și îmbunătățiri

Sistemul dezvoltat reprezintă o fundație tehnologică de bază, care deschide numeroase oportunități pentru cercetări și extinderi software viitoare. Direcțiile principale de îmbunătățire pot fi structurate pe trei direcții majore:

1. Evaluarea calitativă a tehnicii de execuție: O extensie naturală și de valoroasă pentru aplicațiile de fitness este trecerea de la simpla numărare cantitativă la evaluarea calitativă a mișcărilor. Arhitectura curentă poate fi extinsă prin adăugarea unei a treia ramuri specializate în rețeaua multi-task. Această nouă ramură ar putea primi ca intrare aceleași hărți termice sau proiecții P-map și, prin compararea lor cu un șablon ideal definit de medici sportivi sau antrenori experți, ar putea returna un scor continuu (de la 0 la 100), reprezentând corectitudinea posturii utilizatorului, prevenind astfel riscul de accidentare.

2. Optimizarea pentru procesare în timp real: Deși inferența modelului este rapidă, pipeline-ul interfeței web actuale suferă o latență cauzată de procesarea video secvențială (unde întregul videoclip trebuie încărcat și descompus înainte de a rula modelul). O dezvoltare ulterioară posibilă presupune reproiectarea sistemului pentru procesarea în flux continuu (cadru cu cadru în timp real) direct din camera web.

3. Extinderea catalogului de exerciții și urmărirea multi-persoană: Pentru a crește versatilitatea platformei, setul de date poate fi îmbunătățit prin tehnici de finetuning pe seturi de date mai vaste, extinzând cele 7 clase actuale la zeci de exerciții fizice complexe. De asemenea, integrarea unui model care ar permite izolarea și monitorizarea simultană a mai multor sportivi aflați în același cadru vizual, transformând aplicația dintr-un asistent personal într-un instrument util pentru gestionarea claselor de antrenament de grup sau a competițiilor sportive.

Bibliografie

[1] K. Wunsch, K. Kienberger, și C. Niessner, „Changes in Physical Activity Patterns Due to the Covid-19 Pandemic: A Systematic Review and Meta-Analysis”, *International Journal of Environmental Research and Public Health 2022, Vol. 19,* vol. 19, nr. 4, feb. 2022, doi: 10.3390/IJERPH19042250.

[2] J. Wilke *et al.*, „A Pandemic within the Pandemic? Physical Activity Levels Substantially Decreased in Countries Affected by COVID-19”, *International Journal of Environmental Research and Public Health 2021, Vol. 18,* vol. 18, nr. 5, pp. 1–12, feb. 2021, doi: 10.3390/IJERPH18052235.

[3] K. Dhuli *et al.*, „Physical activity for health”, *J. Prev. Med. Hyg.*, vol. 63, nr. 2 Suppl 3, p. E150, iun. 2022, doi: 10.15167/2421-4248/JPMH2022.63.2S3.2756.

[4] G. D’Onofrio, J. Kirschner, H. Prather, D. Goldman, și A. Rozanski, „Musculoskeletal exercise: Its role in promoting health and longevity”, *Prog. Cardiovasc. Dis.*, vol. 77, pp. 25–36, mar. 2023, doi: 10.1016/J.PCAD.2023.02.006.

[5] K. Parker *et al.*, „The use of digital platforms for adults’ and adolescents’ physical activity during the COVID-19 pandemic (our life at home): Survey study”, *J. Med. Internet Res.*, vol. 23, nr. 2, p. e23389, feb. 2021, doi: 10.2196/23389.

[6] A. Fabbrizio *et al.*, „Smart Devices for Health and Wellness Applied to Tele-Exercise: An Overview of New Trends and Technologies Such as IoT and AI”, *Healthcare 2023, Vol. 11,* vol. 11, nr. 12, iun. 2023, doi: 10.3390/HEALTHCARE11121805.

[7] A. El Saddik, „Digital Twins: The Convergence of Multimedia Technologies”, *IEEE Multimedia*, vol. 25, nr. 2, pp. 87–92, apr. 2018, doi: 10.1109/MMUL.2018.023121167.

[8] R. Gámez Díaz, Q. Yu, Y. Ding, F. Laamarti, și A. El Saddik, „Digital Twin Coaching for Physical Activities: A Survey”, *Sensors 2020, Vol. 20,* vol. 20, nr. 20, pp. 1–21, oct. 2020, doi: 10.3390/S20205936.

[9] A. Newell, K. Yang, și J. Deng, „Stacked hourglass networks for human pose estimation”, *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 9912 LNCS, pp. 483–499, 2016, doi: 10.1007/978-3-319-46484-8_29/FIGURES/11.

[10] K. Sun, B. Xiao, D. Liu, și J. Wang, „Deep High-Resolution Representation Learning for Human Pose Estimation”, *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, vol. 2019-June, pp. 5686–5696, feb. 2019, doi: 10.1109/CVPR.2019.00584.

[11] J. Redmon, S. Divvala, R. Girshick, și A. Farhadi, „You Only Look Once: Unified, Real-Time Object Detection”, *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern*

Recognition, vol. 2016-December, pp. 779–788, iun. 2015, doi: 10.1109/CVPR.2016.91.

[12] S. Ji, W. Xu, M. Yang, și K. Yu, „3D Convolutional neural networks for human action recognition", *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, nr. 1, pp. 221–231, 2013, doi: 10.1109/TPAMI.2012.59.

[13] D. Tran, L. Bourdev, R. Fergus, L. Torresani, și M. Paluri, „Learning Spatiotemporal Features With 3D Convolutional Networks", *Proceedings of the IEEE International Conference on Computer Vision*, pp. 4489–4497, dec. 2014, doi: 10.1109/ICCV.2015.510.

[14] S. Yan, Y. Xiong, și D. Lin, „Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition", *32nd AAAI Conference on Artificial Intelligence, AAAI 2018*, pp. 7444–7452, ian. 2018, doi: 10.1609/aaai.v32i1.12328.

[15] V. Choutas, P. Weinzaepfel, J. Revaud, și C. Schmid, „PoTion: Pose MoTion Representation for Action Recognition", în *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, Salt Lake City, UT, USA: IEEE, 2018, pp. 7024–7033. doi: 10.1109/CVPR.2018.00734.

[16] T. F. H. Runia, C. G. M. Snoek, și A. W. M. Smeulders, „Real-World Repetition Estimation by Div, Grad and Curl", *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pp. 9009–9017, feb. 2018, doi: 10.1109/CVPR.2018.00939.

[17] D. Dwibedi, Y. Aytar, J. Tompson, P. Sermanet, și A. Zisserman, „Counting Out Time: Class Agnostic Video Repetition Counting in the Wild", 2020. Data accesării: 8 mai 2026. [Online]. Disponibil la: https://sites.google.com/view/repnet

[18] R. Khurana, K. Ahuja, Z. Yu, J. Mankoff, C. Harrison, și M. Goel, „GymCam", *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 2, nr. 4, pp. 1–17, dec. 2018, doi: 10.1145/3287063.

[19] T. Alatiah și C. Chen, „Recognizing Exercises and Counting Repetitions in Real Time", mai 2020, Data accesării: 8 mai 2026. [Online]. Disponibil la: https://arxiv.org/pdf/2005.03194

[20] Q. Yu, H. Wang, F. Laamarti, și A. El Saddik, „Deep Learning-Enabled Multitask System for Exercise Recognition and Counting", *Multimodal Technologies and Interaction 2021, Vol. 5,* vol. 5, nr. 9, sep. 2021, doi: 10.3390/MTI5090055.

[21] K. He, X. Zhang, S. Ren, și J. Sun, „Deep Residual Learning for Image Recognition", *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, vol. 2016-December, pp. 770–778, dec. 2015, doi: 10.1109/CVPR.2016.90.

[22] T. Schaul, D. Borsa, J. Modayil, și R. Pascanu, „Ray Interference: a Source of Plateaus in Deep Reinforcement Learning", apr. 2019, Data accesării: 20 mai 2026. [Online]. Disponibil la: https://arxiv.org/pdf/1904.11455